Tromboneless

v0.1.1

Generated by Doxygen 1.9.8

1 Tromboneless 1
1.1 Table of Contents 1
1.2 Getting Started: 1
1.2.1 Option 1 1
1.2.2 Option 2 2
1.3 Dependencies 2
1.4 Bill of Materials 2
1.4.1 Controller 2
1.4.2 Sensors 3
1.4.3 Additional Components 3
1.4.4 Circuit Diagram 3
1.5 Documentation 3
1.6 Social Media 3
1.7 Authors and Contributions 4
1.8 License 4
1.8.1 ADS1115 4

2 Notes on App 5
2.1 Map of the code 5
2.2 Code tutorials specifically for JUCE: 5
2.2.1 For labels: 5
2.3 For drop down menus 6

3 Notes for the changes made to the firmware 7

4 Notes for the Hardware implementation and design 9
4.1 Details 9
4.1.1 Processing 9
4.1.2 Power 9
4.1.3 Reading data 9
4.1.3.1 ADC 10
4.1.3.2 ToF sensor 10
4.1.3.3 Touch sensor 10
4.1.4 Outputs 10
4.1.4.1 MIDI 10
4.1.4.2 Video 10

5 KiCAD PCB 11

6 Hierarchical Index 13
6.1 Class Hierarchy 13

7 Class Index 15
7.1 Class List 15

8 File Index 19
8.1 File List . . . 19

9 Class Documentation 21
9.1 AmplitudeMapper Class Reference . . . 21
9.1.1 Member Function Documentation . . . 21
9.1.1.1 getGateFactor() . . . 21
9.1.1.2 setGateFactor() . . . 21
9.2 AudioOutput Class Reference . . . 22
9.2.1 Detailed Description . . . 22
9.2.2 Constructor & Destructor Documentation . . . 22
9.2.2.1 AudioOutput() . . . 22
9.2.2.2 ∼AudioOutput() . . . 23
9.2.3 Member Function Documentation . . . 23
9.2.3.1 writeSamples() . . . 23
9.3 AudioRender Class Reference . . . 23
9.3.1 Detailed Description . . . 30
9.3.2 Constructor & Destructor Documentation . . . 30
9.3.2.1 AudioRender() . . . 30
9.3.3 Member Function Documentation . . . 30
9.3.3.1 isRunning() . . . 30
9.3.3.2 noteChangeFromMidi() . . . 31
9.3.3.3 noteOffFromMidi() . . . 32
9.3.3.4 noteOnFromMidi() . . . 32
9.3.3.5 pitchBendFromMidi() . . . 32
9.3.3.6 setDebugTone() . . . 32
9.4 Barometer Class Reference . . . 33
9.4.1 Detailed Description . . . 36
9.4.2 Member Function Documentation . . . 36
9.4.2.1 getBackgroundColour() . . . 36
9.4.2.2 getBoarderColour() . . . 37
9.4.2.3 getEdgeColour() . . . 37
9.4.2.4 getInnerRadius() . . . 37
9.4.2.5 getLabelRadius() . . . 37
9.4.2.6 getOuterRadius() . . . 38
9.4.2.7 getTextColour() . . . 38
9.4.2.8 paint() . . . 38
9.4.2.9 setBackgroundColour() . . . 38
9.4.2.10 setBoarderColour() . . . 39
9.4.2.11 setEdgeColour() . . . 39
9.4.2.12 setInnerRadius() . . . 39
9.4.2.13 setLabelRadius() . . . 39

9.4.2.14 setOuterRadius() . . . 40
9.4.2.15 setTextColour() . . . 40
9.4.2.16 sliderValueChanged() . . . 40
9.5 CalibrateEmbouchure Class Reference . . . 41
9.5.1 Detailed Description . . . 42
9.5.2 Member Function Documentation . . . 42
9.5.2.1 buttonClicked() . . . 42
9.6 CalibrationOnClick Class Reference . . . 43
9.6.1 Detailed Description . . . 44
9.6.2 Member Function Documentation . . . 44
9.6.2.1 resized() . . . 44
9.6.3 Member Data Documentation . . . 44
9.6.3.1 button . . . 44
9.7 CalibrationSlider Class Reference . . . 45
9.7.1 Detailed Description . . . 47
9.7.2 Member Function Documentation . . . 47
9.7.2.1 getMinDifference() . . . 47
9.7.2.2 setMinDifference() . . . 47
9.7.2.3 sliderValueChanged() . . . 47
9.8 CalibrationSliderLookAndFeel Class Reference . . . 48
9.8.1 Detailed Description . . . 49
9.8.2 Member Function Documentation . . . 49
9.8.2.1 drawLinearSlider() . . . 49
9.8.2.2 getEmptyTrackColour() . . . 50
9.8.2.3 getFullTrackColour() . . . 50
9.8.2.4 getMaxThumbColour() . . . 50
9.8.2.5 getMinThumbColour() . . . 51
9.8.2.6 getTrackWidth() . . . 51
9.8.2.7 setEmptyTrackColour() . . . 51
9.8.2.8 setFullTrackColour() . . . 51
9.8.2.9 setMaxThumbColour() . . . 52
9.8.2.10 setMinThumbColour() . . . 52
9.8.2.11 setTrackWidth() . . . 52
9.9 CAP1188 Class Reference . . . 53
9.9.1 Detailed Description . . . 54
9.9.2 Constructor & Destructor Documentation . . . 54
9.9.2.1 CAP1188() . . . 54
9.9.3 Member Function Documentation . . . 55
9.9.3.1 deltaCount() . . . 55
9.9.3.2 getAveraging() . . . 55
9.9.3.3 getCycleTime() . . . 56
9.9.3.4 getSampleTime() . . . 56

9.9.3.5 getSensitivity() . . . 56
9.9.3.6 getThresholds() . . . 56
9.9.3.7 initialise() . . . 57
9.9.3.8 linkLEDs() . . . 57
9.9.3.9 recalibratePins() . . . 57
9.9.3.10 registerCallback() . . . 58
9.9.3.11 setAveraging() . . . 58
9.9.3.12 setCycleTime() . . . 58
9.9.3.13 setInterruptRepeatRate() . . . 58
9.9.3.14 setSampleTime() . . . 59
9.9.3.15 setSensitivity() . . . 59
9.9.3.16 setThresholds() . . . 59
9.9.3.17 touched() . . . 60
9.10 CAP1188Channel Class Reference . . . 60
9.10.1 Detailed Description . . . 60
9.10.2 Member Function Documentation . . . 61
9.10.2.1 getThreshold() . . . 61
9.10.2.2 isPinTouched() . . . 61
9.10.2.3 pinDelta() . . . 61
9.10.2.4 pinRecalibrate() . . . 61
9.10.2.5 setThreshold() . . . 61
9.11 CAP1188Printer Class Reference . . . 62
9.12 CoreWrapper Class Reference . . . 62
9.13 DropDownMenu Class Reference . . . 63
9.13.1 Detailed Description . . . 64
9.13.2 Constructor & Destructor Documentation . . . 64
9.13.2.1 DropDownMenu() . . . 64
9.13.3 Member Function Documentation . . . 64
9.13.3.1 AddItem() . . . 64
9.13.3.2 ChangeLabelText() . . . 65
9.13.3.3 getLabelText() . . . 65
9.13.3.4 OnChange() . . . 65
9.14 DropDownMenus Class Reference . . . 66
9.14.1 Detailed Description . . . 67
9.15 DualRotarySlider Class Reference . . . 67
9.15.1 Detailed Description . . . 69
9.15.2 Member Function Documentation . . . 70
9.15.2.1 getInterval() . . . 70
9.15.2.2 getLabelHeight() . . . 70
9.15.2.3 getMaxAngle() . . . 70
9.15.2.4 getMaxLimit() . . . 70
9.15.2.5 getMaxSliderRadius() . . . 71

9.15.2.6 getMinAngle() . . . 71
9.15.2.7 getMinDifference() . . . 71
9.15.2.8 getMinLimit() . . . 71
9.15.2.9 getMinSliderRadius() . . . 72
9.15.2.10 setInterval() . . . 72
9.15.2.11 setLabelHeight() . . . 72
9.15.2.12 setMaxAngle() . . . 72
9.15.2.13 setMaxLimit() . . . 73
9.15.2.14 setMaxSliderRadius() . . . 73
9.15.2.15 setMinAngle() . . . 73
9.15.2.16 setMinDifference() . . . 73
9.15.2.17 setMinLimit() . . . 74
9.15.2.18 setMinSliderRadius() . . . 74
9.15.2.19 sliderValueChanged() . . . 74
9.15.3 Member Data Documentation . . . 74
9.15.3.1 maxSlider . . . 74
9.15.3.2 minSlider . . . 74
9.15.3.3 rotaryLabel . . . 75
9.16 Envelope Class Reference . . . 75
9.16.1 Detailed Description . . . 76
9.16.2 Constructor & Destructor Documentation . . . 76
9.16.2.1 Envelope() . . . 76
9.16.3 Member Function Documentation . . . 77
9.16.3.1 attackFunction() . . . 77
9.16.3.2 decayFunction() . . . 77
9.16.3.3 getAmplitude() . . . 77
9.16.3.4 getAttack() . . . 78
9.16.3.5 getDecay() . . . 78
9.16.3.6 getRest() . . . 78
9.16.3.7 getStage() . . . 78
9.16.3.8 getSustain() . . . 79
9.16.3.9 restFunction() . . . 79
9.16.3.10 setAttack() . . . 79
9.16.3.11 setDecay() . . . 80
9.16.3.12 setRest() . . . 80
9.16.3.13 setStage() . . . 80
9.16.3.14 setSustain() . . . 80
9.17 Equalizer Class Reference . . . 81
9.17.1 Detailed Description . . . 83
9.17.2 Constructor & Destructor Documentation . . . 84
9.17.2.1 Equalizer() . . . 84
9.17.3 Member Function Documentation . . . 84

9.17.3.1 sliderValueChanged() . . . 84
9.18 EqualizerPanel Class Reference . . . 85
9.18.1 Detailed Description . . . 86
9.18.2 Member Function Documentation . . . 86
9.18.2.1 buttonClicked() . . . 86
9.18.2.2 resized() . . . 87
9.19 EventHandler Class Reference . . . 87
9.19.1 Detailed Description . . . 87
9.19.2 Constructor & Destructor Documentation . . . 88
9.19.2.1 EventHandler() . . . 88
9.19.3 Member Function Documentation . . . 88
9.19.3.1 initialise() . . . 88
9.19.3.2 start() . . . 88
9.19.3.3 waitForEvent() . . . 88
9.20 GuiAppApplication Class Reference . . . 89
9.20.1 Detailed Description . . . 90
9.20.2 Member Function Documentation . . . 90
9.20.2.1 anotherInstanceStarted() . . . 90
9.20.2.2 getApplicationName() . . . 90
9.20.2.3 getApplicationVersion() . . . 90
9.20.2.4 initialise() . . . 90
9.20.2.5 shutdown() . . . 91
9.21 I2CBus Class Reference . . . 91
9.21.1 Detailed Description . . . 92
9.21.2 Constructor & Destructor Documentation . . . 92
9.21.2.1 I2CBus() . . . 92
9.21.3 Member Function Documentation . . . 92
9.21.3.1 openBus() . . . 92
9.21.3.2 readBlock16() . . . 92
9.21.3.3 readBlock8() . . . 93
9.21.3.4 writeBlock16() . . . 93
9.21.3.5 writeBlock8() . . . 94
9.22 Layout Class Reference . . . 94
9.22.1 Detailed Description . . . 95
9.22.2 Member Function Documentation . . . 95
9.22.2.1 paint() . . . 95
9.22.2.2 resized() . . . 95
9.23 GuiAppApplication::MainWindow Class Reference . . . 96
9.23.1 Detailed Description . . . 97
9.23.2 Constructor & Destructor Documentation . . . 97
9.23.2.1 MainWindow() . . . 97
9.23.3 Member Function Documentation . . . 97

9.23.3.1 closeButtonPressed() . . . 97
9.24 MidiCoordinator Class Reference . . . 97
9.24.1 Detailed Description . . . 98
9.24.2 Member Function Documentation . . . 98
9.24.2.1 ChangeNote() . . . 98
9.24.2.2 PressureEdge() . . . 98
9.24.2.3 setBend() . . . 99
9.24.2.4 setDevice() . . . 99
9.24.2.5 setExpr() . . . 99
9.24.2.6 setState() . . . 99
9.25 MidiMessage Class Reference . . . 100
9.25.1 Detailed Description . . . 100
9.25.2 Constructor & Destructor Documentation . . . 100
9.25.2.1 MidiMessage() [1/3] . . . 100
9.25.2.2 MidiMessage() [2/3] . . . 100
9.25.2.3 MidiMessage() [3/3] . . . 101
9.25.3 Member Function Documentation . . . 101
9.25.3.1 bytes() . . . 101
9.25.3.2 empty() . . . 101
9.25.3.3 size() . . . 101
9.26 MidiMessageBuilder Class Reference . . . 102
9.26.1 Detailed Description . . . 102
9.26.2 Member Function Documentation . . . 102
9.26.2.1 noteOff() . . . 102
9.26.2.2 noteOn() . . . 103
9.26.2.3 pitchBend() . . . 103
9.27 NeedleLookAndFeel Class Reference . . . 104
9.27.1 Detailed Description . . . 105
9.27.2 Member Function Documentation . . . 105
9.27.2.1 drawRotarySlider() . . . 105
9.27.2.2 getNeedleColour() . . . 105
9.27.2.3 setNeedleColour() . . . 106
9.28 Notes Class Reference . . . 106
9.28.1 Detailed Description . . . 109
9.28.2 Constructor & Destructor Documentation . . . 109
9.28.2.1 Notes() . . . 109
9.28.3 Member Function Documentation . . . 110
9.28.3.1 getA() . . . 110
9.28.3.2 getA_sharp() . . . 110
9.28.3.3 getAb() . . . 110
9.28.3.4 getB() . . . 111
9.28.3.5 getBb() . . . 111

9.28.3.6 getC() . . . 111
9.28.3.7 getC_sharp() . . . 111
9.28.3.8 getD() . . . 113
9.28.3.9 getD_sharp() . . . 113
9.28.3.10 getDb() . . . 113
9.28.3.11 getE() . . . 113
9.28.3.12 getEb() . . . 114
9.28.3.13 getF() . . . 114
9.28.3.14 getF_sharp() . . . 114
9.28.3.15 getG() . . . 114
9.28.3.16 getG_sharp() . . . 115
9.28.3.17 getGb() . . . 115
9.28.3.18 getNote() . . . 115
9.28.3.19 setA() . . . 115
9.28.3.20 setA_sharp() . . . 116
9.28.3.21 setAb() . . . 116
9.28.3.22 setB() . . . 116
9.28.3.23 setBb() . . . 117
9.28.3.24 setC() . . . 117
9.28.3.25 setC_sharp() . . . 117
9.28.3.26 setD() . . . 118
9.28.3.27 setD_sharp() . . . 118
9.28.3.28 setDb() . . . 118
9.28.3.29 setE() . . . 119
9.28.3.30 setEb() . . . 119
9.28.3.31 setF() . . . 119
9.28.3.32 setF_sharp() . . . 120
9.28.3.33 setG() . . . 120
9.28.3.34 setG_sharp() . . . 120
9.28.3.35 setGb() . . . 121
9.28.3.36 setNote() . . . 121
9.29 Octaves Class Reference . . . 121
9.29.1 Detailed Description . . . 125
9.29.2 Member Function Documentation . . . 125
9.29.2.1 PlayingFrequency() . . . 125
9.29.2.2 PlayingNote() . . . 125
9.30 OctavesWithHarmonics Class Reference . . . 126
9.30.1 Detailed Description . . . 130
9.30.2 Member Function Documentation . . . 130
9.30.2.1 getDecayConstant() . . . 130
9.30.2.2 getHarmonicDecayMax() . . . 130
9.30.2.3 HarmonicDecay() . . . 131

9.30.2.4 PlayingFrequencyWithHarmonics() . . . 131
9.30.2.5 PlayingNoteWithHarmonics() . . . 132
9.30.2.6 setDecayConstant() . . . 132
9.31 Panels Class Reference . . . 133
9.31.1 Detailed Description . . . 134
9.31.2 Member Function Documentation . . . 134
9.31.2.1 getBackgroundColour() . . . 134
9.31.2.2 paint() . . . 134
9.31.2.3 setBackgroundColour() . . . 134
9.32 PitchMapper::PartialDef Struct Reference . . . 135
9.33 PitchMapper Class Reference . . . 135
9.33.1 Detailed Description . . . 136
9.33.2 Member Function Documentation . . . 136
9.33.2.1 mouthpiece_to_MIDI_note() . . . 136
9.33.2.2 tof_to_MIDI_bend() . . . 136
9.34 PressureSensor Class Reference . . . 136
9.34.1 Detailed Description . . . 137
9.35 RawInputEvent Struct Reference . . . 137
9.35.1 Detailed Description . . . 138
9.35.2 Member Enumeration Documentation . . . 138
9.35.2.1 Type . . . 138
9.35.3 Member Data Documentation . . . 139
9.35.3.1 keycontrol . . . 139
9.35.3.2 mouthpieceReading . . . 139
9.35.3.3 pressureReading . . . 139
9.35.3.4 tofDistance . . . 139
9.35.3.5 type . . . 139
9.36 RtMidiSink Class Reference . . . 140
9.36.1 Detailed Description . . . 140
9.36.2 Constructor & Destructor Documentation . . . 140
9.36.2.1 RtMidiSink() . . . 140
9.36.3 Member Function Documentation . . . 141
9.36.3.1 DetectUSBMidiOutput() . . . 141
9.36.3.2 GetDeviceStatus() . . . 141
9.36.3.3 send() . . . 141
9.37 Sliders Class Reference . . . 141
9.37.1 Detailed Description . . . 143
9.37.2 Member Function Documentation . . . 143
9.37.2.1 getDistanceRange() . . . 143
9.37.2.2 getMaximumDistance() . . . 144
9.37.2.3 getMinimumDistance() . . . 144
9.37.2.4 getStepDistance() . . . 144

9.37.2.5 resized() . . . 144
9.37.2.6 setDistanceRange() . . . 145
9.37.2.7 setMaximumDistance() . . . 145
9.37.2.8 setMinimumDistance() . . . 145
9.37.2.9 setStepDistance() . . . 145
9.37.2.10 sliderValueChanged() . . . 146
9.37.3 Member Data Documentation . . . 146
9.37.3.1 distanceSlider . . . 146
9.38 SliderWithLabel Class Reference . . . 146
9.38.1 Detailed Description . . . 148
9.38.2 Member Function Documentation . . . 148
9.38.2.1 CreateLabel() . . . 148
9.38.2.2 getBottomLabelBounds() . . . 149
9.38.2.3 getLeftLabelBounds() . . . 149
9.38.2.4 getRightLabelBounds() . . . 149
9.38.2.5 getTopLabelBounds() . . . 150
9.38.2.6 resized() . . . 150
9.38.2.7 setBottomLabelBounds() . . . 150
9.38.2.8 setLeftLabelBounds() . . . 150
9.38.2.9 setRightLabelBounds() . . . 151
9.38.2.10 setTopLabelBounds() . . . 151
9.39 SynthesiserParameters_t Struct Reference . . . 151
9.39.1 Detailed Description . . . 151
9.40 ToFSensor Class Reference . . . 152
9.40.1 Detailed Description . . . 152
9.40.2 Member Typedef Documentation . . . 152
9.40.2.1 DistanceCallback . . . 152
9.40.3 Constructor & Destructor Documentation . . . 152
9.40.3.1 ToFSensor() . . . 152
9.40.4 Member Function Documentation . . . 153
9.40.4.1 initialise() . . . 153
9.40.4.2 registerCallback() . . . 153
9.41 TromponelessParameters_t Struct Reference . . . 153
9.41.1 Detailed Description . . . 154
9.42 TromboneSynth Class Reference . . . 154
9.42.1 Detailed Description . . . 160
9.42.2 Constructor & Destructor Documentation . . . 160
9.42.2.1 TromboneSynth() . . . 160
9.42.3 Member Function Documentation . . . 160
9.42.3.1 ChangeTromboneNote() . . . 160
9.42.3.2 getAdjustedFrequency() . . . 160
9.42.3.3 getAttackMS() . . . 162

9.42.3.4 getDecayMS() . . . 162
9.42.3.5 getNHarmonics() . . . 162
9.42.3.6 getNote() . . . 162
9.42.3.7 getOctave() . . . 163
9.42.3.8 getPitchBend() . . . 163
9.42.3.9 getRestMS() . . . 163
9.42.3.10 getSustain() . . . 163
9.42.3.11 NewTromboneNoteMIDI() . . . 164
9.42.3.12 ReadTromboneAudio() . . . 164
9.42.3.13 setAttackMS() . . . 164
9.42.3.14 setDecayMS() . . . 164
9.42.3.15 setNHarmonics() . . . 165
9.42.3.16 setPitchBend() . . . 165
9.42.3.17 setRestMS() . . . 165
9.42.3.18 setSustain() . . . 165
9.42.3.19 StartTromboneNote() . . . 166
9.43 verticalMixSlider Class Reference . . . 166
9.43.1 Detailed Description . . . 168
9.43.2 Member Function Documentation . . . 168
9.43.2.1 sliderValueChanged() . . . 168
9.44 VerticalSliderLookAndFeel Class Reference . . . 169
9.44.1 Detailed Description . . . 171
9.44.2 Constructor & Destructor Documentation . . . 171
9.44.2.1 VerticalSliderLookAndFeel() . . . 171
9.44.3 Member Function Documentation . . . 171
9.44.3.1 drawLinearSlider() . . . 171
9.44.3.2 getBackgroundColour() . . . 172
9.44.3.3 getDialColour() . . . 172
9.44.3.4 getDialEdgeColour() . . . 172
9.44.3.5 getFingerHeight() . . . 173
9.44.3.6 getFingerWidth() . . . 173
9.44.3.7 getTextColour() . . . 173
9.44.3.8 getTickColour() . . . 173
9.44.3.9 getTickThickness() . . . 174
9.44.3.10 getTickWidth() . . . 174
9.44.3.11 getTrackColour() . . . 174
9.44.3.12 getTrackWidth() . . . 174
9.44.3.13 setBackgroundColour() . . . 175
9.44.3.14 setDialColour() . . . 175
9.44.3.15 setDialEdgeColour() . . . 175
9.44.3.16 setFingerHeight() . . . 175
9.44.3.17 setFingerWidth() . . . 176

9.44.3.18 setTextColour() . . . 176
9.44.3.19 setTickColour() . . . 176
9.44.3.20 setTickThickness() . . . 176
9.44.3.21 setTickWidth() . . . 177
9.44.3.22 setTrackColour() . . . 177
9.44.3.23 setTrackWidth() . . . 177

10 File Documentation **179**

10.1 CustomStyles.hpp File Reference . . . 179
10.1.1 Detailed Description . . . 181
10.1.2 Function Documentation . . . 181
10.1.2.1 drawCircle() . . . 181
10.1.2.2 PaintArc() . . . 181
10.1.2.3 PaintTrombone() . . . 182
10.1.2.4 RotaryArc() . . . 182
10.2 CustomStyles.hpp . . . 183
10.3 Layout.hpp File Reference . . . 185
10.3.1 Detailed Description . . . 186
10.4 Layout.hpp . . . 186
10.5 Panels.hpp File Reference . . . 187
10.5.1 Detailed Description . . . 188
10.6 Panels.hpp . . . 188
10.7 TromboneIessJuce.hpp File Reference . . . 190
10.7.1 Detailed Description . . . 191
10.8 TromboneIessJuce.hpp . . . 191
10.9 Widgets.hpp File Reference . . . 191
10.9.1 Detailed Description . . . 193
10.10 Widgets.hpp . . . 193
10.11 CustomStyles.cpp File Reference . . . 197
10.11.1 Detailed Description . . . 198
10.11.2 Function Documentation . . . 198
10.11.2.1 drawCircle() . . . 198
10.11.2.2 PaintArc() . . . 199
10.11.2.3 PaintTrombone() . . . 199
10.11.2.4 RotaryArc() . . . 200
10.12 Layout.cpp File Reference . . . 200
10.12.1 Detailed Description . . . 201
10.13 Panels.cpp File Reference . . . 201
10.13.1 Detailed Description . . . 201
10.14 TromboneIessJuce.cpp File Reference . . . 202
10.14.1 Detailed Description . . . 202
10.15 Widgets.cpp File Reference . . . 202

10.15.1 Detailed Description . . . 202
10.16 AmplitudeMapper.hpp . . . 203
10.17 audioRender.hpp File Reference . . . 203
10.17.1 Detailed Description . . . 204
10.18 audioRender.hpp . . . 205
10.19 CoreWrapper.hpp File Reference . . . 205
10.19.1 Detailed Description . . . 206
10.20 CoreWrapper.hpp . . . 207
10.21 audio_output.hpp File Reference . . . 207
10.21.1 Detailed Description . . . 209
10.22 audio_output.hpp . . . 209
10.23 cap1188_registers.hpp . . . 209
10.24 cap_sensor.hpp File Reference . . . 212
10.24.1 Detailed Description . . . 213
10.25 cap_sensor.hpp . . . 214
10.26 i2c_bus.hpp File Reference . . . 215
10.26.1 Detailed Description . . . 216
10.27 i2c_bus.hpp . . . 216
10.28 Pin_Definitions.h . . . 217
10.29 pressure_sensor.hpp . . . 217
10.30 tof_sensor.hpp File Reference . . . 218
10.30.1 Detailed Description . . . 219
10.31 tof_sensor.hpp . . . 219
10.32 USBMidi.hpp File Reference . . . 220
10.32.1 Detailed Description . . . 221
10.33 USBMidi.hpp . . . 222
10.34 envelope.hpp File Reference . . . 222
10.34.1 Detailed Description . . . 223
10.35 envelope.hpp . . . 223
10.36 eventHandler.hpp File Reference . . . 224
10.36.1 Detailed Description . . . 225
10.37 eventHandler.hpp . . . 226
10.38 main.hpp . . . 226
10.39 MidiCoordinator.hpp File Reference . . . 227
10.39.1 Detailed Description . . . 228
10.40 MidiCoordinator.hpp . . . 228
10.41 MidiMessage.hpp . . . 229
10.42 MidiMessageBuilder.hpp . . . 229
10.43 PitchMapper.hpp . . . 230
10.44 synth.hpp File Reference . . . 231
10.44.1 Detailed Description . . . 232
10.45 synth.hpp . . . 233

10.46 tromboneSynth.hpp File Reference 235
10.46.1 Detailed Description 236
10.47 tromboneSynth.hpp 236
10.48 CoreWrapper.cpp File Reference 238
10.48.1 Detailed Description 238
10.49 cap_sensor.cpp File Reference 238
10.49.1 Detailed Description 239
10.50 i2c_bus.cpp File Reference 239
10.50.1 Detailed Description 239
10.51 tof_sensor.cpp File Reference 240
10.51.1 Detailed Description 240
10.52 envelope.cpp File Reference 240
10.52.1 Detailed Description 241
10.53 MidiCoordinator.cpp File Reference 241
10.53.1 Detailed Description 241
10.54 synth.cpp File Reference 241
10.54.1 Detailed Description 242
10.55 tromboneSynth.cpp File Reference 242
10.55.1 Detailed Description 243
10.56 tromboneless_data.hpp 243

Chapter 1

Tromboneless

This GitHub repository contains the open source hardware and firmware for Tromboneless - an open source device capable of measuring oral input to synthesise the output of a brass trombone using a Raspberry Pi 5, implementing MIDI protocol.

Please refer to our Wiki `wiki` to see component, sequence and state diagrams for the MIDI implementation.

1.1 Table of Contents

- Getting Started
- Dependencies
- Bill of Materials
- Social Media
- Acknowledgements
- Authors & Contributions
- License

1.2 Getting Started:

1. Install Raspberry Pi OS onto your Raspberry Pi.
   For set-up guidance for PiOS, please follow [this guide].(https://www.raspberrypi.com/documentation/computers/getting-started.html#installing-the-operating-system)

1.2.1 Option 1

1. clone this git repository and its submodule dependencies using one line one the command line:

```
git clone --recursive https://github.com/RyanMcB8/Tromboneless.git
```

1.2.2 Option 2

1. clone this git repository using the command:

```
git clone https://github.com/RyanMcB8/Tromboneless.git
```

2. To add the necessary submodules into the workspace, you will need to use the following command line commands:

```
git submodules init
```

```
git submodules update
```

1.3 Dependencies

The Tromboneless App utilises the JUCE framework to create all the widgets necessary for operation.

- Juce
- RTMidi
- libgpiod (specifically for interacting with the GPIO pins on the Pi)
- libwebkit2gtk-4.1-dev
- libgtk-3-dev
- libcurl4-openssl-dev

1.4 Bill of Materials

Below contains the materials required to construct the Tromboneless hardware, with a circuit diagram showing how to configure it.

1.4.1 Controller

Microcontroller	Quantity	Cost (£)
Raspberry Pi 5	1	58.98
Active Cooler for Raspberry Pi 5 (recommended)	1	4.80

1.4.2 Sensors

Sensors	Quantity	Cost (£)
VL53L1X Time of Flight (ToF) Sensor Breakout	1	20.40
CAP1188 8-Key Capacitive Touch Sensor	1	7.70
ADS1115 16-bit ADC	1	6.49

Total Cost: £34.59

1.4.3 Additional Components

Additional Components	Quantity
10.0 k ohm Resistor	1
300.0 ohm Resistor	1
390.0 k ohm Resistor	1
USB MIDI Cable	1
Infrared LED	1
Photodiode	1

1.4.4 Circuit Diagram

Circuit diagram showing how to assemble the Tromboneless :

1.5 Documentation

For documentation of the code used, please refer to the `Documentation` directory where there are both LaTex and HTML versions of the documentation for this project.

1.6 Social Media

- #1 Post on `r/Trombone`
- 28.6k+ Views across `r/Embedded`, `r/Trombone` and `r/linuxaudio`

Follow us for additional updates, including showcase videos and demonstrations:

`Reddit`
`Instagram`

1.7 Authors and Contributions

- Ben Allen -
- Aidan McIntosh - High-level event-handling, musical coordination, external MIDI integration
- Ryan McBride -
- Kerr McLaren -
- Ciaran Rogers -

1.8 License

1.8.1 ADS1115

- The ADS1115 library was adopted from `Bernd Porr`, which can be sourced `here`.

Chapter 2

Notes on App

2.1 Map of the code

Within the `main.cpp` file is the code responsible for creating main window. This should not need to be modified as the contents of the window are defined elsewhere.

The code within `Layout.cpp` creates the defines the general structure of the window by creating instances of widgets and panels and arranging them to be in the correct position on the screen.

The `Panels.cpp` file is where each panel on the display is created. This sets the backgorund colour of each panel and calls upon the specific widgets which should be in each panel.

The `Widgets.cpp` file hosts the classes for creating more custom widgets that base library of JUCE does not already provide. These include modifications to shapes of the widgets such as the custom calibration sliders and equalizer.

The `tromboneless_data.cpp` file has the variables which interact with the Tromboneless Core functionality and allow for the Core and App to communicate parameters between each other. If there is a need to add more communication channels between the app and core, this is where t should be added. For example, a flag for if a speaker has been connected.

Each of these files has an associated header file, excluding `main.cpp`, which hosts the declarations and custom types which may be used externally. All documentation for the functions are within the header files with more in-depth step by step explanations within the cpp files.

If the files are thought about as a pipe line, the order would be: JUCE --> Widgets --> Panels --> Layout --> main

2.2 Code tutorials specifically for JUCE:

2.2.1 For labels:

This code will display only uppercase characters input.

```
addAndMakeVisible (uppercaseLabel);
uppercaseLabel.setText ("Uppercase:", juce::dontSendNotification);
uppercaseLabel.attachToComponent (&uppercaseText, true);
uppercaseLabel.setColour (juce::Label::textColourId, juce::Colours::orange);
uppercaseLabel.setJustificationType (juce::Justification::right);
addAndMakeVisible (uppercaseText);
uppercaseText.setColour (juce::Label::backgroundColourId, juce::Colours::darkblue);
```

These functions will allow for the text to be editted by the user when the app is open.

```
addAndMakeVisible (inputText);
inputText.setEditable (true);
inputText.setColour (juce::Label::backgroundColourId, juce::Colours::darkblue);
inputText.onTextChange = [this] { uppercaseText.setText (inputText.getText().toUpperCase(),
      juce::dontSendNotification); };
```

When this label is changed by the user, the `onTextChange` sets a lambda function to be excucted whenever there is any change within the label.

2.3 For drop down menus

The drop down menus within the JUCE library are referred to as a `comboBox`.

The lines of code below display an example which first creates an instance of a combo-box called 'styleMenu' and adds optionsusing the `addItem` method. When the choice is changed, the `styleMenuChanged()` function is called upon. The first option within the list is set as the default to be selected.

```
addAndMakeVisible (styleMenu);
styleMenu.addItem ("Plain", 1);
styleMenu.addItem ("Bold", 2);
styleMenu.addItem ("Italic", 3);
styleMenu.onChange = [this] { styleMenuChanged(); };
styleMenu.setSelectedId (1);
setSize (400, 200);
```

Chapter 3

Notes for the changes made to the firmware

Chapter 4

Notes for the Hardware implementation and design

The image above shows a circuit diagram representing the hardware and connections used to create the prototype for the Tromboneless. The chosen peripherals may be changed to suitable alternatives but the code will need to be adapted to have the correct drivers for the new devices.

4.1 Details

4.1.1 Processing

To process and host the program, the Tromboneless used a Raspberry Pi 5 with a linux build which allowed for easy navigation around the directories and remote access for remote testing purposes. As this system was built using C++, this could should be capable of being adapted to work using another main board which supports the standard language. Unfortunately, this would require changing low level drivers and pin defintions within the code base to allow it to work.

One recommendation from the Tromboneless team is to use a cooler on the device to ensure that it doesn't begin to thermal throttle. The Pi was found to crash when building the JUCE submodule without a cooler. The cooler used in development was the `Active Cooler for Raspberry Pi 5` from Raspberry Pi purchased though `The Pi Hut`.

4.1.2 Power

Each of the peripherals is connected directly the Raspberry Pi for both power and ground. Each of the peripherals receives 5V.

4.1.3 Reading data

To read the data from the peripherals, the I2C protocol was utilised to allow for a reduction in pins for communication compared to SPI. To read the data at an appropriate time, each of the peripherals need to send a data ready interrupt to the processor for it to initiate a transfer. This allowed for a reduction in computation for the processor by utilising blocking I/O as opposed to polling for data continuously. This shifted a large amount of computation to the peripherals from the main processor freeing to complete other threads.

4.1.3.1 ADC

To find the amount of pressue the user is exerting into the mouthpiece (blowing), a balloon was enclosed in a tube restricting its width and reducing the light within the tube. The system developed used an IR transmitter and receiver, both of which were diodes. When the balloon expands, the amount of infra-red being reflected increases due to the light bouncing off the base of the balloon. The IR transmitter (an IR LED), was kept on persistently with the receiver having its cathode pin connected to the ADC. The cathode pin also connected to a 10K pull down resister in parallel which created a potential divider allowing the ADC to receive varying voltages as the amount of IR light is varied.

The specific ADC used was the `ADS1115` on a development board from `the Pi hut`.

An alternative to this method is to use a pressure sensor but from testing, the lower end sensors did not have a large enough range to match that which a trombonist would reach during a performance.

4.1.3.2 ToF sensor

To detect the distance the user's hand is from the device, a Time of Flight (ToF) sensor was used. The specific sensor chosen was the `VL53LOX` on a breakout board from `Pimoroni`. By addressing the devices onboard registers, the sensor could be set have the ideal range and sampling frequency for this device.

There are alternative choices for measuring the hand distance such as sonar or linear variable resistors. For this project, LiDAR was the best option for its speed, low interference, accuracy and being non contact.

4.1.3.3 Touch sensor

To measure the amount of pressure the user puts onto the mouthpiece to simulate the change in embouchure or 'buzz' of the trombonists lips, a capacitive touch sensor was developed. This custom sensor was created wrapping a sheet of copper and an electronically insulating material around a curved surface. This created a capacitor. The amount of pressure the user places on the device changes the distance between each of the copper layers which changes the capacitance. This capacitance may be fed directly into the touch sensor board used, the `CAP1188` by Microchip on a breakout board `from Adafruit`.

As the touch sensor and the ToF sensor both have the same I2C address as default, a 390K resister connected to the AD pin of the touch sensor was pulled to ground changing its I2C address from 0x29 to 0x28.

4.1.4 Outputs

4.1.4.1 MIDI

To read the MIDI data which is being output by the Pi, a MIDI compatible device such as a portable synthesiser may be connected to any of the USB-A connectors on the front of the Pi.

4.1.4.2 Video

The 2 mini HDMI ports on the side of the Raspberry Pi may be connected to a monitor (or monitors) to display the video output. When the app is launched, it may be seen in the video output.

Chapter 5

KiCAD PCB

The KiCAD schematic for the PCB can be found here. This PCB was made to interface between the Raspberry Pi 5 and the sensors.

Chapter 6

Hierarchical Index

6.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

AmplitudeMapper . . . 21
AudioOutput . . . 22
CAP1188 . . . 53
CAP1188Channel . . . 60
CAP1188Printer . . . 62
juce::Component
- CalibrationOnClick . . . 43
  - CalibrateEmbouchure . . . 41
- DropDownMenu . . . 63
- DualRotarySlider . . . 67
  - Barometer . . . 33
- Layout . . . 94
- Panels . . . 133
  - DropDownMenus . . . 66
  - EqualizerPanel . . . 85
  - Sliders . . . 141
- SliderWithLabel . . . 146
  - CalibrationSlider . . . 45
  - verticalMixSlider . . . 166
    - Equalizer . . . 81

CoreWrapper . . . 62
juce::DocumentWindow
- GuiAppApplication::MainWindow . . . 96

Envelope . . . 75
EventHandler . . . 87
I2CBus . . . 91
juce::JUCEApplication
- GuiAppApplication . . . 89

juce::Button::Listener
- CalibrationOnClick . . . 43
- EqualizerPanel . . . 85

juce::Slider::Listener
- CalibrationSlider . . . 45
- DualRotarySlider . . . 67
- Sliders . . . 141

verticalMixSlider 166
juce::LookAndFeel_V4
CalibrationSliderLookAndFeel 48
NeedleLookAndFeel 104
VerticalSliderLookAndFeel 169
MidiCoordinator 97
MidiMessage 100
MidiMessageBuilder 102
Notes 106
Octaves 121
OctavesWithHarmonics 126
TromboneSynth 154
AudioRender 23
PitchMapper::PartialDef 135
PitchMapper 135
PressureSensor 136
RawInputEvent 137
RtMidiSink 140
SynthesiserParameters_t 151
ToFSensor 152
Tromboneless Parameters_t 153

Chapter 7

Class Index

7.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

AmplitudeMapper 21

AudioOutput
Provides a low-level interface for audio playback via ALSA 22

AudioRender
Handles real-time audio generation and output 23

Barometer
A class which can allow for the look and feel of a set of rotary sliders be modified to look like a barometer (pressure gauge) 33

CalibrateEmbouchure
A class which is inherited from the CalibrationOnClick class that adds a dropdown menu next to the button for the user to select which option they wish to calibrate 41

CalibrationOnClick
A class which allows for the additon of a button to the screen without inheriting the button directly. This allows for other widgets to be added to it later without compatibility issues 43

CalibrationSlider
A class that adds more functionality to the slider class from Juce 45

CalibrationSliderLookAndFeel
A class to set the look and feel of a two finger linear slider to look like a trombone where the left most finger is the main body and the right most is the end of the trombones slide 48

CAP1188
Base driver class for the CAP1188 53

CAP1188Channel
Helper class unique to one sensor channel 60

CAP1188Printer 62

CoreWrapper 62

DropDownMenu
A class that creates a drop down menu or 'comboBox' with a label attached to it. This takes up 95% of the available local area. The label is set to have a maxmimum width of 200, but scales as the window size is reduced 63

DropDownMenus
A class which is created to display the drop down menus required by the Tromboneless UI to choose the chosen option for calibration settings 66

DualRotarySlider
A class that adds more functionality to the slider class from Juce 67

Envelope
A class which can allow for the create of an envelope function which outputs a value between 0 and 1 depending on what stage the envelope is in. This uses the ADSR standard in music synthesis where A is the Attack, D is the decay, S is the saturation and R is the rest stage . . . 75
Equalizer
A class which inherits its sliders from `verticalMixSlider` made to act as an equalizer . . 81
EqualizerPanel
A class which may be called to add a panel to the window contains a series of sliders for the equalizer allowing for live editting of the synthesisers sound to ensure it sounds how the Trombonist wishes it to . . . 85
EventHandler
Handles acquisition and dispatch of raw sensor input events . . . 87
GuiAppApplication
Creation of main application class . . . 89
I2CBus
Minimal Linux I2C bus wrapper for register-based communication . . . 91
Layout
Main class that creates the window . . . 94
GuiAppApplication::MainWindow
This class implements the desktop window that contains an instance of the MainComponent class . . . 96
MidiCoordinator
Class to coordinate Midi message construction based on output from sensor mappers. This class is stateful and responsible for translating high-level sensor data into Midi messages. Utilises MidiMessageBuilder for external synths and directly controls internal synth . . . 97
MidiMessage
Represents a raw MIDI message as a sequence of bytes . . . 100
MidiMessageBuilder
Factory class for constructing common MIDI channel messages . . . 102
NeedleLookAndFeel
A class which may be attached to a rotary slider from the juce framework to change its look and feel. This changes the look of the slider to have an arrow rotate about the centre of the slider pointing to the different values rather than a knob moving in an arc as the basic rotary slider . . 104
Notes
A class to store the frequency of the notes which may be referenced. The notes are stored in an enum `Notes_t` which allow for the higher level functions to call upon the specific note as opposed to frequency . . . 106
Octaves
Creation of a class to store the frequencies of notes at different octaves. Inherits from `Notes` class.
121
OctavesWithHarmonics
A class which inherits functions and variables from the `Octaves` class allowing for the notes being output to have a set of harmonic frequencies attached to them too . . . 126
Panels
A class which is inherited from Juce:Component reponsible for setting the general style of a panel within the main window . . . 133
PitchMapper::PartialDef . . . 135
PitchMapper
Takes raw input values from the LIDAR and mouthpiece/embouchure sensors and maps them to appropriate values for the musical coordinator to work with . . . 135
PressureSensor
Wrapper around ADS1115rpi library (https://github.com/berndporr/rpi_↩ ads1115) by Bernd Porr that provides an interface for pressure sensing . . . 136
RawInputEvent
Represents a generic input event produced by hardware sensors . . . 137
RtMidiSink
Handles MIDI output to external devices using RtMidi . . . 140

Sliders
A class which adds all the linear sliders being used within the calibration section to be within the same
panel 141
SliderWithLabel
A class which inherits attributes from the juce::Component class to create a new component containing a 8 possible label locations around the slider 146
SynthesiserParameters_t
A struct to store any of the synthesier parameters that may be changed within the app. A data changed flag variable should be added to this struct to tell the core to read the new values . . 151
ToFSensor
Time-of-Flight sensor interface using I2C and GPIO interrupt handling 152
Tromboneless Parameters_t
A struct which stores all the necessary parameters that may be changed on the app side effecting the core functionality. This struct should add a flag which is triggered whenever a parameter has been changed 153
TromboneSynth
A class which may be used to create a custom synth with an envelope attached to it 154
verticalMixSlider
A class that adds more functionality to the slider class from Juce 166
VerticalSliderLookAndFeel
Creation of a class which can be used to change the look of the sliders used within the equalizer for the Tromboneless project 169

Chapter 8

File Index

8.1 File List

Here is a list of all documented files with brief descriptions:

CustomStyles.hpp
This file holds the declarations of all functions and variables needed by the Tromboneless project to change the looks and feel of various different widgets which are being displayed to the end user 179

Layout.hpp
This file holds the declarations of all functions and variables needed by the Tromboneless project to define the layout of the window which is being displayed to the end user 185

Panels.hpp
A header file to contain the panels used within the Tromboneless user interface window. This will allow for the encapsulation of different areas as opposed to having large amounts of repeat calls in the `Layout.hpp` file 187

TromboneslessJuce.hpp 190

Widgets.hpp
A file which sets the declarations of any custom widgets which have been built off of the juce framework. A large portion of these widgets have been created specifically for the Tromboneless project 191

CustomStyles.cpp
This file holds the defintions of all functions and variables needed by the Tromboneless project to change the looks and feel of various different widgets which are being displayed to the end user 197

Layout.cpp
This file holds the defintions of all functions and variables needed by the Tromboneless project to define the layout of the window which is being displayed to the end user 200

Panels.cpp
This file contains the function defintions for to create the necessary panels that may be used within the Tromboneless user interface 201

TromboneslessJuce.cpp 202

Widgets.cpp
A file which defines the functions which have been declared within the `Widgets.hpp` file . . 202

AmplitudeMapper.hpp 203

audioRender.hpp
Declares the AudioRender class, responsible for managing the real-time audio rendering pipeline and interfacing with the synthesis engine 203

CoreWrapper.hpp
A file to allow for the app to view methods attached to the main functioanlity of the core 205

audio_output.hpp
Declares the AudioOutput class, responsible for interfacing with the ALSA audio subsystem for playback 207

cap1188_registers.hpp . . . 209
cap_sensor.hpp
Base driver class for the Adafruit CAP1188 sensor breakout board, specific to Raspberry Pi 5 over Linux kernel with I2C communication . . . 212
i2c_bus.hpp
Minimal Linux I2C bus wrapper . . . 215
Pin_Definitions.h . . . 217
pressure_sensor.hpp . . . 217
tof_sensor.hpp
Interface for VL53L1X Time-of-Flight sensor . . . 218
USBMidi.hpp
Declares the RtMidiSink class, responsible for sending MIDI messages to an external device via RtMidi . . . 220
envelope.hpp
A header file containing all the necessary declarations and references to create a functional envelope class using the ADSR standard . . . 222
eventHandler.hpp
Declares the EventHandler class, responsible for acquiring and normalising raw sensor inputs into a unified event stream . . . 224
main.hpp . . . 226
MidiCoordinator.hpp
Declaration of class for high-level musical coordination behaviour. Takes premapped sensor inputs and either internal or external synth . . . 227
MidiMessage.hpp . . . 229
MidiMessageBuilder.hpp . . . 229
PitchMapper.hpp . . . 230
synth.hpp
A header file containing all the necessary declarations and references to allow for the Tromboneless to synthesise the noise a trombone would make when playing the same notes . . . 231
tromboneSynth.hpp
A file to declare the synthesiser parameters and functions for the trombone specifically using the classes and methods declared in the `synth.hpp` file . . . 235
CoreWrapper.cpp
A file which encapsulations the main funcationality to allow for the app to act independently of it 238
cap_sensor.cpp
Implementation of CAP11888 driver for Tromboneless . . . 238
i2c_bus.cpp
Implementation of the I2CBus class for Linux I2C communication . . . 239
tof_sensor.cpp
Implementation of the ToFSensor class . . . 240
envelope.cpp
Contains the defintions of the functions and variables needed to create an envelope . . . 240
MidiCoordinator.cpp
Definition file for MIDICoordinator class . . . 241
synth.cpp
C file which contains the defintions for the functions declared in the `synth.hpp` file . . . 241
tromboneSynth.cpp
A file to define the synthesiser parameters and functions for the trombone specifically using the classes and methods declared in the `synth.hpp` file . . . 242
tromboneless_data.hpp . . . 243

Chapter 9

Class Documentation

9.1 AmplitudeMapper Class Reference

Public Member Functions

- bool **calculateBaseline** (float pressureSample)
- float **getBaseline** ()
- bool **noteEdge** (float pressureSample)
- float getGateFactor (void)

  A function to get the current pressure threshold which is set for the gate for the input blowing pressure by the user.
- void setGateFactor (float threshold)

  A function to set the pressure threshold which is set for the gate of the input blowing pressure by the user.

9.1.1 Member Function Documentation

9.1.1.1 getGateFactor()

```
float AmplitudeMapper::getGateFactor (
            void  )
```

A function to get the current pressure threshold which is set for the gate for the input blowing pressure by the user.

Return values

The	floating point representation of the current thhreshold value.

9.1.1.2 setGateFactor()

```
void AmplitudeMapper::setGateFactor (
            float threshold )
```

A function to set the pressure threshold which is set for the gate of the input blowing pressure by the user.

Parameters

threshold	The floating point representation of the new threshold which should be set.

The documentation for this class was generated from the following files:

- AmplitudeMapper.hpp
- AmplitudeMapper.cpp

9.2 AudioOutput Class Reference

Provides a low-level interface for audio playback via ALSA.

```
#include <audio_output.hpp>
```

Public Member Functions

- AudioOutput (bool isTest)

  Constructs the AudioOutput interface.
- ~AudioOutput ()

  Destructor.
- void writeSamples (const int16_t *buffer, int frames)

  Writes a buffer of audio samples to the output device.

9.2.1 Detailed Description

Provides a low-level interface for audio playback via ALSA.

This class encapsulates the configuration and use of an ALSA PCM playback device. It is responsible for sending rendered audio buffers to the system audio output in a format compatible with the configured device.

In test mode, hardware interaction is disabled, allowing the rest of the audio pipeline to be exercised without requiring a physical audio device.

Note

This class assumes audio data is provided as interleaved 16-bit samples.

Intended for real-time usage; buffer writes should be performed consistently to avoid underruns.

9.2.2 Constructor & Destructor Documentation

9.2.2.1 AudioOutput()

```
AudioOutput::AudioOutput (
            bool isTest )
```

Constructs the AudioOutput interface.

Parameters

isTest	Enables test mode (disables ALSA interaction).

9.2.2.2 ~AudioOutput()

```
AudioOutput::~AudioOutput ( )
```

Destructor.

Ensures that the ALSA device is properly closed and resources are released.

9.2.3 Member Function Documentation

9.2.3.1 writeSamples()

```
void AudioOutput::writeSamples (
            const int16_t * buffer,
            int frames )
```

Writes a buffer of audio samples to the output device.

Parameters

buffer	Pointer to the audio sample buffer.
frames	Number of frames to write.

The buffer is expected to contain 16-bit signed samples. The interpretation (mono/stereo, interleaving) must match the device configuration.

In test mode, this function performs no hardware write.

The documentation for this class was generated from the following files:

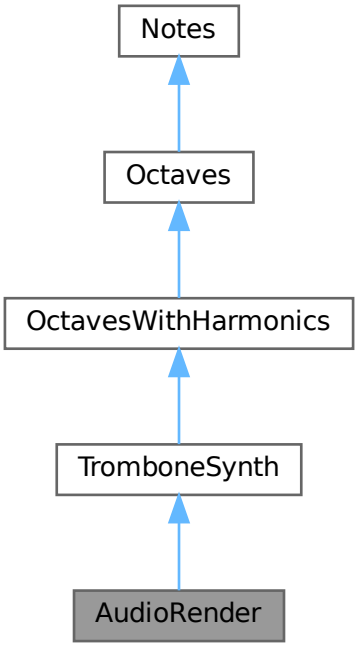
- audio_output.hpp
- audio_output.cpp

9.3 AudioRender Class Reference

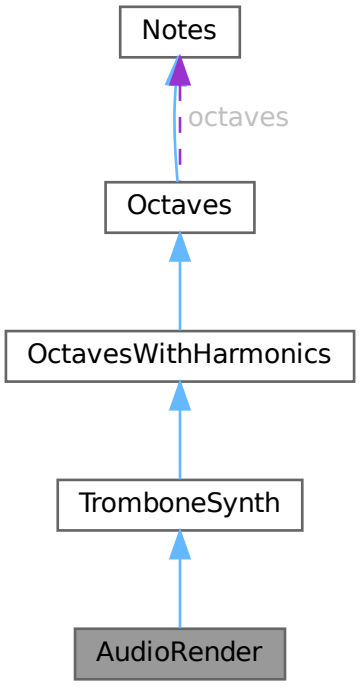
Handles real-time audio generation and output.

```
#include <audioRender.hpp>
```

Inheritance diagram for AudioRender:



Collaboration diagram for AudioRender:



Public Member Functions

- AudioRender (bool isTest)

  Constructs the AudioRender object.
- ~**AudioRender** ()

  Destructor. Ensures rendering thread is stopped safely.
- **AudioRender** (const AudioRender &)=delete
- AudioRender & **operator=** (const AudioRender &)=delete
- void **start** ()

  Starts the audio rendering thread.
- void **stop** ()

  Requests the rendering thread to stop and joins it.
- bool isRunning () const

  Checks whether the rendering thread is active.
- void noteOnFromMidi (int midiNote)

  Triggers a note-on event from a MIDI source.
- void noteOffFromMidi ()

  Triggers a note-off event from a MIDI source.
- void noteChangeFromMidi (int midiNote)

  Changes the currently playing note without retriggering.
- void pitchBendFromMidi (int bend)

  Applies pitch bend from a MIDI source.
- void setDebugTone (bool enabled)

  Enables or disables debug tone output.

Public Member Functions inherited from TromboneSynth

- TromboneSynth (int sampleRate=44100, float attack=10.0f, float decay=50.0f, float sustain=1.0f, float rest=10.0f)

  The constructor function for the `TromboneSynth` class.
- void StartTromboneNote (Notes::Notes_t note_in, int octave_in)

  A function which may be called to set the note and octave of a new sound by the user. If there is already a note playing, this will stop the previous not and play this one without phase shifting (initially).
- void ChangeTromboneNote (Notes::Notes_t note_in, int octave_in)

  A function that can change the current note without needed to change the stage that the envelope is in.
- float ReadTromboneAudio (void)

  A function which may be called to read the current amplitude of the audio signal being generated.
- void **StopTromboneNote** (void)

  A function which sends the synthesiser into its rest state.
- void NewTromboneNoteMIDI (int MIDINote)

  A function which can take in the MIDI equivalent note and map it to the Notes_t options to match the rest of the synth interface.
- void **HandleMIDINoteOn** ()

  A method to handle NoteOn messages as passed from the MIDI Coordinator.
- void **HandleMIDINoteOff** ()
- void **HandleMIDIPitchBend** (int midipitchbend)
- void **HandleMIDINoteChange** (int new_note)
- void setAttackMS (float attack_time)

  A function to set the amount of time in milliseconds that the attack stage takes.
- float getAttackMS (void)

  A function to get the current attack time in milliseconds.
- void setDecayMS (float decay_time)

  A function to set the amount of time in milliseconds that the decay stage takes.
- float getDecayMS (void)

  A function to get the current decay time in milliseconds.
- void setRestMS (float rest_time)

  A function to set the amount of time in milliseconds that the rest stage takes.
- float getRestMS (void)

  A function to get the current rest time in milliseconds.
- void setSustain (float sustain_in)

  A function to set the relative amplitude of the sustain stage.
- float getSustain (void)

  A function to get the current sustain value.
- void setNHarmonics (int n)

  A function to set the number of harmonics which are being generated for every note being played.
- int getNHarmonics (void)

  A function to get the number of harmonics which are being generated for every note being played.
- int getOctave (void)

  A function to get the current octave which the current note is in.
- Notes::Notes_t getNote (void)

  A function to get the current note which is being used.
- void setPitchBend (int bend)

  A function to set the pitch bend of the device.
- int getPitchBend (void)

  A function to get the pitch bend of the device.
- float getAdjustedFrequency (void)

  A function which determines the new frequency which should be being played depending upon the current note and octave as well as the pitch bend being input.
- Envelope **getEnvelope** (void)

Public Member Functions inherited from OctavesWithHarmonics

- **OctavesWithHarmonics** ()

  Creates an instance of the `OctavesWithHarmonics`.
- float PlayingNoteWithHarmonics (int n, int octave, Notes_t note, float phase)

  A function which may be called upon to stop playing a range of harmonic notes when a specific note is called upon to be played.
- float PlayingFrequencyWithHarmonics (int n, float phase)

  A function which may be called upon to stop playing a range of harmonic frequencies when a specific note is called upon to be played.
- float HarmonicDecay (int n)

  A function which calculates how much the amplitude of the harmonic should decay in comparison to the first harmonic.
- float getHarmonicDecayMax (int n)

  A function which returns the maximum amplitude possible for a set number of harmonics if they are all constructively interfering.
- void setDecayConstant (float decayC)

  A function that can change the value of the decay constant used to set the maxmimum amplitude of each successive harmonic.
- float getDecayConstant (void)

  A function which returns the decay constant being used to determine the relative ampltiude of each successive harmonic.

Public Member Functions inherited from Octaves

- float PlayingNote (int octave, Notes_t note, float phase)

  A function which may be called to determine the amplitude of a specified note at a specified phase.
- float PlayingFrequency (float phase)

  A function which may be called to determine the amplitude of a specified frequency at a specified time.

Public Member Functions inherited from Notes

- Notes (float C_in=16.35, float Db_in=17.32, float D_in=18.35, float Eb_in=19.45, float E_in=20.60, float F↩ _in=21.83, float Gb_in=23.12, float G_in=24.50, float Ab_in=25.96, float A_in=27.50, float Bb_in=29.14, float B_in=30.87)

  Constructor for the `Notes` class to create a new set of notes and assign specified frequencies to each. Each frequency is set to `Octave 0` by default.
- void setC (float frequency)

  A function that can change the value of the frequency that the C note plays.
- void setC_sharp (float frequency)

  A function that can change the value of the frequency that the C# note plays.
- void setDb (float frequency)

  A function that can change the value of the frequency that the Db note plays.
- void setD (float frequency)

  A function that can change the value of the frequency that the D note plays.
- void setD_sharp (float frequency)

  A function that can change the value of the frequency that the D# note plays.
- void setEb (float frequency)

  A function that can change the value of the frequency that the Eb note plays.
- void setE (float frequency)

  A function that can change the value of the frequency that the E note plays.
- void setF (float frequency)

A function that can change the value of the frequency that the F note plays.

- void setF_sharp (float frequency)

  A function that can change the value of the frequency that the F# note plays.
- void setGb (float frequency)

  A function that can change the value of the frequency that the Gb note plays.
- void setG (float frequency)

  A function that can change the value of the frequency that the G note plays.
- void setG_sharp (float frequency)

  A function that can change the value of the frequency that the G# note plays.
- void setAb (float frequency)

  A function that can change the value of the frequency that the Ab note plays.
- void setA (float frequency)

  A function that can change the value of the frequency that the A note plays.
- void setA_sharp (float frequency)

  A function that can change the value of the frequency that the A# note plays.
- void setBb (float frequency)

  A function that can change the value of the frequency that the Bb note plays.
- void setB (float frequency)

  A function that can change the value of the frequency that the B note plays.
- float setNote (Notes_t note, float freq)

  An alternative function which can allow for the user to alter the frequency of a specific note based upon the Notes_t enumerated values rather than specifying a function call.
- float getC (void)

  A function that returns the frequency of the C note for the specified object.
- float getC_sharp (void)

  A function that returns the frequency of the C# note for the specified object.
- float getDb (void)

  A function that returns the frequency of the Db note for the specified object.
- float getD (void)

  A function that returns the frequency of the D note for the specified object.
- float getD_sharp (void)

  A function that returns the frequency of the D# note for the specified object.
- float getEb (void)

  A function that returns the frequency of the Eb note for the specified object.
- float getE (void)

  A function that returns the frequency of the E note for the specified object.
- float getF (void)

  A function that returns the frequency of the F note for the specified object.
- float getF_sharp (void)

  A function that returns the frequency of the F# note for the specified object.
- float getGb (void)

  A function that returns the frequency of the Gb note for the specified object.
- float getG (void)

  A function that returns the frequency of the G note for the specified object.
- float getG_sharp (void)

  A function that returns the frequency of the G# note for the specified object.
- float getAb (void)

  A function that returns the frequency of the Ab note for the specified object.
- float getA (void)

  A function that returns the frequency of the A note for the specified object.
- float getA_sharp (void)

A function that returns the frequency of the A# note for the specified object.

- float getBb (void)

  A function that returns the frequency of the Bb note for the specified object.

- float getB (void)

  A function that returns the frequency of the B note for the specified object.

- float getNote (Notes_t note)

  An alternative function which can allow for the user to request a specific note based upon the Notes_t enumerated values rather than specifying a function call.

Static Public Attributes

- static constexpr int **frames** = 128

  Number of audio frames processed per buffer.

- static constexpr int **sample_rate** = 44100

  Audio sample rate in Hz.

Static Public Attributes inherited from Octaves

- static const char **nOctaves** = 9

  Stores the number of octaves being used.

Additional Inherited Members

Public Types inherited from Notes

- enum Notes_t {
  note_C , **note_Db** , **note_D** , **note_Eb** ,
  note_E , **note_F** , **note_Gb** , **note_G** ,
  note_Ab , **note_A** , **note_Bb** , **note_B** ,
  note_C_sharp = note_Db , **note_D_sharp** = note_Eb , **note_F_sharp** = note_Gb , **note_G_sharp** = note↩
  _Ab ,
  note_A_sharp = note_Bb }

  Creation of the enumerated list of possible notes.

Public Attributes inherited from Octaves

- Notes **octaves** [nOctaves]

  Initialising an array of the octaves and their notes.

Protected Member Functions inherited from OctavesWithHarmonics

- void **updateHarmonicCache** (int n)

Protected Attributes inherited from OctavesWithHarmonics

- float **decayConstant** = 0.1f
- std::array< float, maxHarmonics > **harmonicWeights** {}
- int **cachedNHarmonics** = 0
- bool **harmonicCacheDirty** = true

Static Protected Attributes inherited from OctavesWithHarmonics

- static constexpr int **maxHarmonics** = 64

9.3.1 Detailed Description

Handles real-time audio generation and output.

This class manages a dedicated audio rendering thread which continuously generates audio samples using the inherited TromboneSynth engine and sends them to the AudioOutput device.

It provides thread-safe control interfaces for note triggering and pitch manipulation, allowing interaction from external threads (e.g. MIDI input).

The class also supports a debug tone mode for validating the audio pipeline independently of the synthesis engine.

Note

Inherits from TromboneSynth to directly access synthesis functionality.

Thread-safe interaction is enforced via atomic flags and a mutex.

9.3.2 Constructor & Destructor Documentation

9.3.2.1 AudioRender()

```
AudioRender::AudioRender (
            bool isTest )
```

Constructs the AudioRender object.

Parameters

isTest	Enables test mode behaviour (e.g. simplified or debug operation).

9.3.3 Member Function Documentation

9.3.3.1 isRunning()

```
bool AudioRender::isRunning ( ) const  [inline]
```

Checks whether the rendering thread is active.

Returns

True if rendering is ongoing, false otherwise.

9.3.3.2 noteChangeFromMidi()

```
void AudioRender::noteChangeFromMidi (
            int midiNote )
```

Changes the currently playing note without retriggering.

Parameters

midiNote	New MIDI note number.

Intended for legato-style transitions.

9.3.3.3 noteOffFromMidi()

```
void AudioRender::noteOffFromMidi ( )
```

Triggers a note-off event from a MIDI source.

Thread-safe wrapper around synth note-off behaviour.

9.3.3.4 noteOnFromMidi()

```
void AudioRender::noteOnFromMidi (
            int midiNote )
```

Triggers a note-on event from a MIDI source.

Parameters

midiNote	MIDI note number.

Thread-safe wrapper around synth note-on behaviour.

9.3.3.5 pitchBendFromMidi()

```
void AudioRender::pitchBendFromMidi (
            int bend )
```

Applies pitch bend from a MIDI source.

Parameters

bend	Pitch bend value (typically 14-bit MIDI range).

Thread-safe wrapper for continuous pitch control.

9.3.3.6 setDebugTone()

```
void AudioRender::setDebugTone (
            bool enabled )
```

Enables or disables debug tone output.

Parameters

enabled	True to enable debug tone, false to disable.

When enabled, a simple tone is generated instead of the synth output, useful for validating the audio pipeline.

The documentation for this class was generated from the following files:

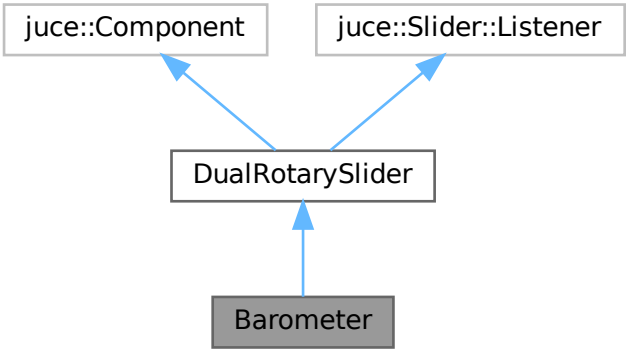
- audioRender.hpp
- audioRender.cpp

9.4 Barometer Class Reference

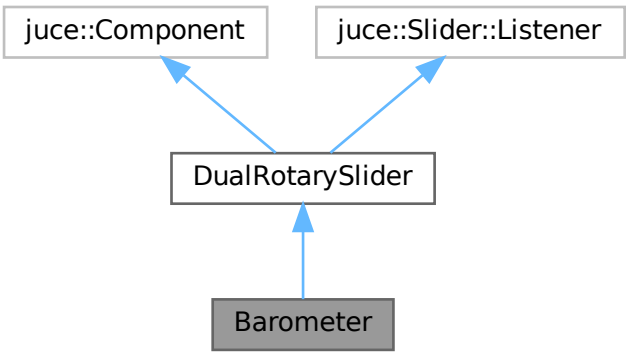
A class which can allow for the look and feel of a set of rotary sliders be modified to look like a barometer (pressure gauge).

```
#include <Widgets.hpp>
```

Inheritance diagram for Barometer:



Collaboration diagram for Barometer:



Public Member Functions

- **Barometer** (CoreWrapper &coreWrapper)

  Constructor for the Barometer class which sets the look and feel of the sliders, attaches a listener to them both and sets the labels text to be the corresponding text.
- ~**Barometer** ()

  Decontructor for the Barometer class which removes the listeners from the sliders as well as resetting their look and feel.
- void sliderValueChanged (juce::Slider *sliderChanged) override

  A callback function which is automatically called upon when either of the sliders knobs are adjusted.
- void paint (juce::Graphics &g) override

  A function which is used to change the background of the component to look like it is a barometer/ pressure gauge.
- void setBackgroundColour (juce::Colour colour)

  A function to set the colour of the background of the slider.
- juce::Colour getBackgroundColour (void)

  A function to get the current colour of the background of the slider.
- void setTextColour (juce::Colour colour)

  A function to set the colour of the text around the slider.
- juce::Colour getTextColour (void)

  A function to get the current colour of the text around the slider.
- void setBoarderColour (juce::Colour colour)

  A function to set the colour of the boarder of the slider.
- juce::Colour getBoarderColour (void)

  A function to get the current colour of the boarder of the slider.
- void setEdgeColour (juce::Colour colour)

  A function to set the colour of the edge around the slider.
- juce::Colour getEdgeColour (void)

  A function to get the current colour of the edge around the slider.
- void setOuterRadius (float radius)

  A function to set the relative radius of the outer rotary slider.
- float getOuterRadius (void)

  A function to get the relative radius of the outer rotary slider.

- void setInnerRadius (float radius)

  A function to set the relative radius of the inner rotary slider.
- float getInnerRadius (void)

  A function to get the relative radius of the inner rotary slider.
- void setLabelRadius (float radius)

  A function to set the relative radius of the label.
- float getLabelRadius (void)

  A function to get the relative radius of the label.

Public Member Functions inherited from DualRotarySlider

- **DualRotarySlider** ()

  The constructor function for the `DualRotarySlider` class which sets the range of the sliders, minimum distance and adds listeners to them both so that when the sliders are moved, the corresponding variables may be changed.
- ~**DualRotarySlider** ()

  The destructor function for the `DualRotarySlider` class which removes the listeners from the sliders.
- void sliderValueChanged (juce::Slider *sliderChanged) override

  A function that is called upon when either of the fingers are moved.
- void **resized** () override

  A callback function for when the window size has changed. This allows for the labels and sliders to be resized to be the best relative size. This can either be set to a predetermined size like 100px x 200px, but it is currrently set to be relative to the size of the screen.
- void setMinDifference (float difference)

  A function to set the minimum difference between the sliders 2 fingers.
- float getMinDifference (void)

  A function to get the current minimum difference between the 2 slider fingers.
- void setMinSliderRadius (float radius)

  A function to set the relative radius of the minimum finger slider. It should always be less than or equal to the maximum slider radius and be between 0 and 1.
- float getMinSliderRadius (void)

  A function to get the relative radius of the minimum finger slider for this object.
- void setMaxSliderRadius (float radius)

  A function to set the relative radius of the maximum finger slider. It should always be more than or equal to the minimum slider radius and be between 0 and 1.
- float getMaxSliderRadius (void)

  A function to get the relative radius of the maximum finger slider for this object.
- void setLabelHeight (float height)

  A function to set the maximum height of the label attached to the rotary slider.
- float getLabelHeight (void)

  A function to get the current height of the label which is attached to the rotary slider.
- void setMinLimit (float limit)

  A function to set the minimum limit of the rotary slider. This may be any floating point value that is less than the difference between the maximum value and the minimum difference between the slider fingers.
- float getMinLimit (void)

  A function to get the current minium limit of the rotary slider.
- void setMaxLimit (float limit)

  A function to set the maximum limit of the rotary slider. This may be any floating point value that is more than the sum of the minium value and the minimum difference between the slider fingers.
- float getMaxLimit (void)

  A function to get the current maximum limit of the rotary slider.
- void setInterval (float step)

A function to set the minimum step the slider may be moved by the user. This value is in the same scale as the maxLimit and minLimit.

- float getInterval (void)

  A function to get the current minimum step which the slider may be adjusted by the user. This value is in the same scale as the maxLimit and minLimit.

- void setMinAngle (float angle)

  A function to allow for the minimum angle (furtherest anticlockwise) that the slider may be rotated to be set. This is measured in radians.

- float getMinAngle (void)

  A function to return the current minimum angle (furtherest anticlockwise) that the slider may be rotated. This is measured in radians.

- void setMaxAngle (float angle)

  A function to allow for the maximum angle (furtherest clockwise) that the slider may be rotated to be set. This is measured in radians.

- float getMaxAngle (void)

  A function to return the current maximum angle (furtherest clockwise) that the slider may be rotated. This is measured in radians.

Additional Inherited Members

Public Attributes inherited from DualRotarySlider

- juce::Slider maxSlider
- juce::Slider minSlider
- juce::Label rotaryLabel

9.4.1 Detailed Description

A class which can allow for the look and feel of a set of rotary sliders be modified to look like a barometer (pressure gauge).

Note

This is used in the Tromboneless project to calibrate the maximum and minimum gains on the mouthpiece.

9.4.2 Member Function Documentation

9.4.2.1 getBackgroundColour()

```
juce::Colour Barometer::getBackgroundColour (
            void  )
```

A function to get the current colour of the background of the slider.

Return values

Returns	the colour as a juce::Colour type.

9.4.2.2 getBoarderColour()

```
juce::Colour Barometer::getBoarderColour (
            void  )
```

A function to get the current colour of the boarder of the slider.

Return values

Returns	the colour as a juce::Colour type.

9.4.2.3 getEdgeColour()

```
juce::Colour Barometer::getEdgeColour (
            void  )
```

A function to get the current colour of the edge around the slider.

Return values

Returns	the colour as a juce::Colour type.

9.4.2.4 getInnerRadius()

```
float Barometer::getInnerRadius (
            void  )
```

A function to get the relative radius of the inner rotary slider.

Return values

The	floating point representation of the relative radius with respect to the widgets (barometer instance) bounds.

9.4.2.5 getLabelRadius()

```
float Barometer::getLabelRadius (
            void  )
```

A function to get the relative radius of the label.

Return values

The	floating point representation of the relative radius with respect to the widgets (barometer instance) bounds.

9.4.2.6 getOuterRadius()

```
float Barometer::getOuterRadius (
            void  )
```

A function to get the relative radius of the outer rotary slider.

Return values

The	floating point representation of the relative radius with respect to the widgets (barometer instance) bounds.

9.4.2.7 getTextColour()

```
juce::Colour Barometer::getTextColour (
            void  )
```

A function to get the current colour of the text around the slider.

Return values

Returns	the colour as a juce::Colour type.

9.4.2.8 paint()

```
void Barometer::paint (
            juce::Graphics & g )  [override]
```

A function which is used to change the background of the component to look like it is a barometer/ pressure gauge.

Parameters

g	A pointer to the juce::Graphics instance which is being drawn to. This is likely the parent component or the main window of the system.

Note

This function may be altered to change the look of the device but it currently adds a white circle as a background, golden outer circle for the edge of it, 2 arcs around it corresponding to bounds, with lines connecting them representing the possible settings.

9.4.2.9 setBackgroundColour()

```
void Barometer::setBackgroundColour (
            juce::Colour colour )
```

A function to set the colour of the background of the slider.

Parameters

colour	The colour to which the background should be set to.

9.4.2.10 setBoarderColour()

```
void Barometer::setBoarderColour (
            juce::Colour colour )
```

A function to set the colour of the boarder of the slider.

Parameters

colour	The colour to which the boarder should be set to.

9.4.2.11 setEdgeColour()

```
void Barometer::setEdgeColour (
            juce::Colour colour )
```

A function to set the colour of the edge around the slider.

Parameters

colour	The colour to which the edge should be set to.

9.4.2.12 setInnerRadius()

```
void Barometer::setInnerRadius (
            float radius )
```

A function to set the relative radius of the inner rotary slider.

Parameters

radius	The floating point representation of the relative radius with respect to the widgets (barometer instance) bounds.

9.4.2.13 setLabelRadius()

```
void Barometer::setLabelRadius (
            float radius )
```

A function to set the relative radius of the label.

Parameters

radius	The floating point representation of the relative radius with respect to the widgets (barometer instance) bounds.

9.4.2.14 setOuterRadius()

```
void Barometer::setOuterRadius (
            float radius )
```

A function to set the relative radius of the outer rotary slider.

Parameters

radius	The floating point representation of the relative radius with respect to the widgets (barometer instance) bounds.

9.4.2.15 setTextColour()

```
void Barometer::setTextColour (
            juce::Colour colour )
```

A function to set the colour of the text around the slider.

Parameters

colour	The colour to which the text should be set to.

9.4.2.16 sliderValueChanged()

```
void Barometer::sliderValueChanged (
            juce::Slider * sliderChanged )  [override]
```

A callback function which is automatically called upon when either of the sliders knobs are adjusted.

Parameters

sliderChanged	A pointer to the slider which has been changed by the user.

The documentation for this class was generated from the following files:

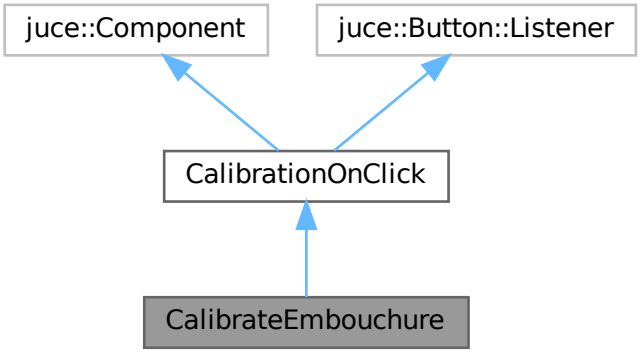
- Widgets.hpp
- Widgets.cpp

9.5 CalibrateEmbouchure Class Reference

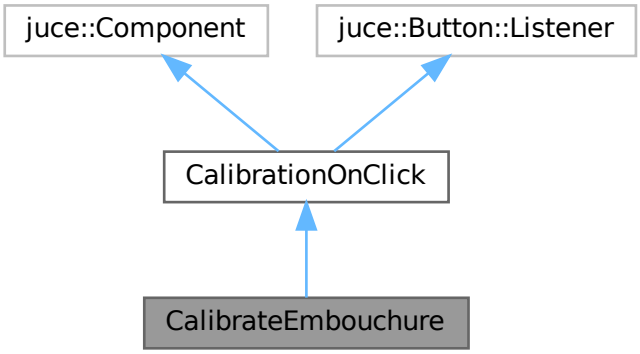
A class which is inherited from the CalibrationOnClick class that adds a dropdown menu next to the button for the user to select which option they wish to calibrate.

```
#include <Widgets.hpp>
```

Inheritance diagram for CalibrateEmbouchure:



Collaboration diagram for CalibrateEmbouchure:



Public Member Functions

- **CalibrateEmbouchure** ()

  Constructor function for the CalibrateEmbouchure class which sets the specific parameters of the label, dropdown menu and button which are being used.

- ~**CalibrateEmbouchure** ()=default

  Destructor function for the CalibrateEmbouchure class which is currently set to default as there are mo specific methods or parameters which need to be dealt with before destructing the object.
- void **resized** () override

  A function which is automatically called upon when the user window is adjusted in size. This function will allow for the change in relative width and height of the components dependent upon screen size.
- void buttonClicked (juce::Button *thisButtonClicked) override

  Callback function which is automatically triggered when the button within this class is clicked by the user. As this class only has one instance of a button, there is no need to check which button has been pressed. However, a pointer to the button is still passed as default from the JUCE framework.

Public Member Functions inherited from CalibrationOnClick

- **CalibrationOnClick** ()

  A constructor for the CalibrationOnClick class which inherits characteristics from juce:: Component and Button::↩ Listener. It is used to receive a calibration message when clicked.
- ~**CalibrationOnClick** ()

  Destructor for the CalibrationOnClick class which removes the listener from the attached button.
- void resized () override

  A function which is called upon whenever the user interface window has changed in size. This can change the size of the component so that it can either be kept at a constant size of scaled to fit a specified area of the screen.

Additional Inherited Members

Public Attributes inherited from CalibrationOnClick

- juce::TextButton button

9.5.1 Detailed Description

A class which is inherited from the CalibrationOnClick class that adds a dropdown menu next to the button for the user to select which option they wish to calibrate.

Note

This class was specifically designed for the embachure selction as no other widget needed to have a calibration button, hence the name of the class and lack of an intermediate step from CalibrationOnClick.

9.5.2 Member Function Documentation

9.5.2.1 buttonClicked()

```
void CalibrateEmbouchure::buttonClicked (
            juce::Button * thisButtonClicked )  [override]
```

Callback function which is automatically triggered when the button within this class is clicked by the user. As this class only has one instance of a button, there is no need to check which button has been pressed. However, a pointer to the button is still passed as default from the JUCE framework.

Note

Currently this function updates the trombonelessParameters struct instance to change the calibrate embouchure flag to be on. This will allow the Core to read the ADC and set the corresponding value.

The documentation for this class was generated from the following files:

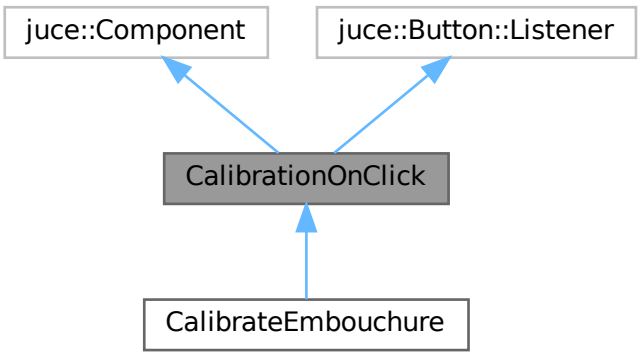
- Widgets.hpp
- Widgets.cpp

9.6 CalibrationOnClick Class Reference

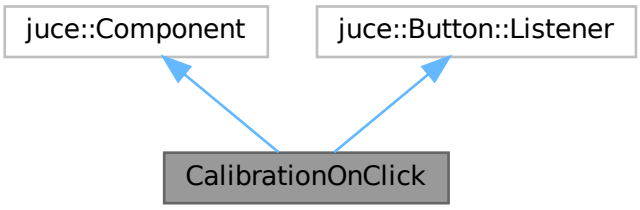
A class which allows for the additon of a button to the screen without inheriting the button directly. This allows for other widgets to be added to it later without compatibility issues.

```
#include <Widgets.hpp>
```

Inheritance diagram for CalibrationOnClick:



Collaboration diagram for CalibrationOnClick:



Public Member Functions

- **CalibrationOnClick** ()

  A constructor for the CalibrationOnClick class which inherits characteristics from juce:: Component and Button::↩ Listener. It is used to receive a calibration message when clicked.
- ~**CalibrationOnClick** ()

  Destructor for the CalibrationOnClick class which removes the listener from the attached button.
- void resized () override

  A function which is called upon whenever the user interface window has changed in size. This can change the size of the component so that it can either be kept at a constant size of scaled to fit a specified area of the screen.

Public Attributes

- juce::TextButton button

9.6.1 Detailed Description

A class which allows for the additon of a button to the screen without inheriting the button directly. This allows for other widgets to be added to it later without compatibility issues.

9.6.2 Member Function Documentation

9.6.2.1 resized()

```
void CalibrationOnClick::resized ( ) [override]
```

A function which is called upon whenever the user interface window has changed in size. This can change the size of the component so that it can either be kept at a constant size of scaled to fit a specified area of the screen.

Note

Currently, this adds a small margin around the local area and fills the rest of the area with the attached button.

9.6.3 Member Data Documentation

9.6.3.1 button

```
juce::TextButton CalibrationOnClick::button
```

Creating the TextButton instance for the CalibrationOnClick class to have a button within it.

The documentation for this class was generated from the following files:

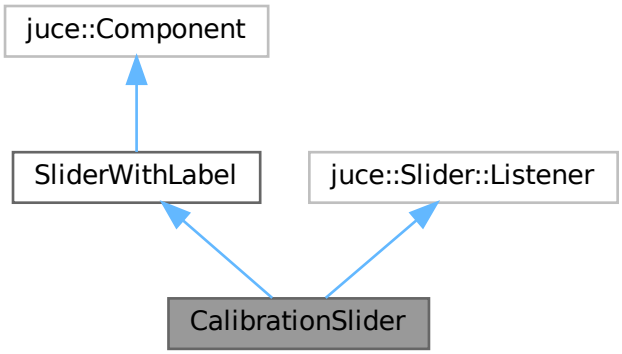
- Widgets.hpp
- Widgets.cpp

9.7 CalibrationSlider Class Reference

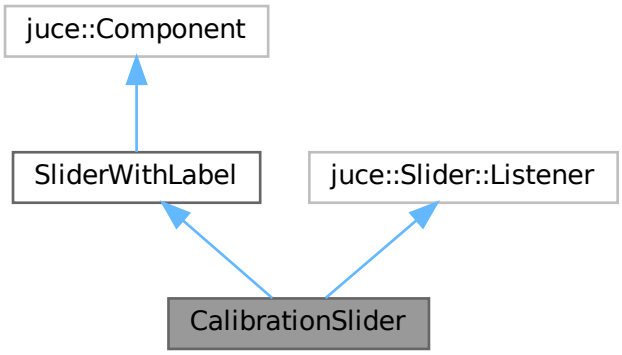
A class that adds more functionality to the slider class from Juce.

```
#include <Widgets.hpp>
```

Inheritance diagram for CalibrationSlider:



Collaboration diagram for CalibrationSlider:



Public Member Functions

- **CalibrationSlider** ()

  The contructor function for the `CalibrationSlider` class which requires the type of slider being used to be initiated. This is set to `TwoValueHorizontal` by default.
- ~**CalibrationSlider** ()

The destructor function for the `CalibrationSlider` class which removes the listener from the slider.

- void setMinDifference (float difference)

  A function which sets the minimum difference between the upper and lower fingers of the slider.
- float getMinDifference (void)

  A function which returns the current minimum distance that the slider fingers may be apart.
- void sliderValueChanged (juce::Slider *sliderChanged) override

  A function that is called upon when either of the fingers are moved.

Public Member Functions inherited from SliderWithLabel

- **SliderWithLabel** (juce::Slider::SliderStyle style)

  Constructor which sets the style of the slider being used and removes the text entry option.
- InitialiseLabelErrors_t CreateLabel (LabelPositions_t position, juce::String phrase, juce::NotificationType NotificationEnable=juce::dontSendNotification)

  A function which may be called to create or edit a specified label around the slider.
- void resized () override

  A function which is automatically called whenever the window size is modified. This automatically adjusts the bounds of each of the labels as well as the slider to ensure the size is correct with respect to the screen size for the component.
- void setTopLabelBounds (int bound)

  A function to set the bounds for which the top labels may be situated in.
- int getTopLabelBounds (void)

  A function to get the bounds for which the top labels are situated in.
- void setLeftLabelBounds (int bound)

  A function to set the bounds for which the labels on the left may be situated in.
- int getLeftLabelBounds (void)

  A function to get the bounds for which the labels on the left are situated in.
- void setRightLabelBounds (int bound)

  A function to set the bounds for which the labels on the right may be situated in.
- int getRightLabelBounds (void)

  A function to get the bounds for which the labels on the right are situated in.
- void setBottomLabelBounds (int bound)

  A function to set the bounds for which the bottom labels may be situated in.
- int getBottomLabelBounds (void)

  A function to get the bounds for which the bottom labels are situated in.

Additional Inherited Members

Public Types inherited from SliderWithLabel

- enum LabelPositions_t {
  UpperLeft , **UpperCentre** , **UpperRight** , **MiddleLeft** ,
  MiddleRight , **LowerLeft** , **LowerCentre** , **LowerRight** }

  Enumerated list of possible positions for the label to be attached relative to the slider.
- enum InitialiseLabelErrors_t { **LabelOK** , **PositonNotAccepted** }

  Enumerated list of possible errors when initialising the labels within this component.

Public Attributes inherited from SliderWithLabel

- juce::Slider **slider**

  The slider which is having the labels added to it.

9.7.1 Detailed Description

A class that adds more functionality to the slider class from Juce.

9.7.2 Member Function Documentation

9.7.2.1 getMinDifference()

```
float CalibrationSlider::getMinDifference (
            void  )
```

A function which returns the current minimum distance that the slider fingers may be apart.

Return values

The	double precision representation of the minimum distance between the slider fingers.

9.7.2.2 setMinDifference()

```
void CalibrationSlider::setMinDifference (
            float difference )
```

A function which sets the minimum difference between the upper and lower fingers of the slider.

Parameters

difference	The value of the minimum distance between the slider knobs.

9.7.2.3 sliderValueChanged()

```
void CalibrationSlider::sliderValueChanged (
            juce::Slider * sliderChanged )  [override]
```

A function that is called upon when either of the fingers are moved.

Note

This function automatically moves the fingers to ensure that they stay at least the minimum range apart from each other.

The documentation for this class was generated from the following files:

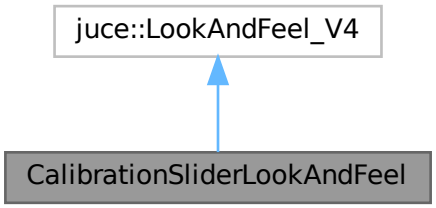
- Widgets.hpp
- Widgets.cpp

9.8 CalibrationSliderLookAndFeel Class Reference

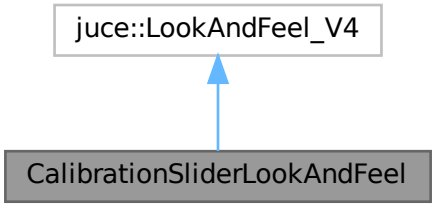
A class to set the look and feel of a two finger linear slider to look like a trombone where the left most finger is the main body and the right most is the end of the trombones slide.

```
#include <CustomStyles.hpp>
```

Inheritance diagram for CalibrationSliderLookAndFeel:



Collaboration diagram for CalibrationSliderLookAndFeel:



Public Member Functions

- virtual void drawLinearSlider (juce::Graphics &g, int x, int y, int width, int height, float sliderPos, float min↩ SliderPos, float maxSliderPos, const juce::Slider::SliderStyle style, juce::Slider &slider)

  When this look and feel is attached to an object which has a vertical slider in it, this function sets the look and feel of the slider being used. This makes it look like an average equalizer slider.
- void setEmptyTrackColour (juce::Colour colour)

  A function to allow for the empty track colour to be changed.
- juce::Colour getEmptyTrackColour (void)

  A function to get the current colour of the empty track of the slider.
- void setFullTrackColour (juce::Colour colour)

  A function to allow for the full track colour to be changed.
- juce::Colour getFullTrackColour (void)

A function to get the current colour of the full track of the slider.

- void setMinThumbColour (juce::Colour colour)

  A function to allow for the colour of the first thumb to be changed.
- juce::Colour getMinThumbColour (void)

  A function to get the current colour of the minimum finger of the slider.
- void setMaxThumbColour (juce::Colour colour)

  A function to allow for the colour of the second thumb to be changed.
- juce::Colour getMaxThumbColour (void)

  A function to get the current colour of the maximum finger of the slider.
- void setTrackWidth (float width)

  A function to set the width of the track to indicate what step the slider is positioned in.
- float getTrackWidth (void)

  A function which may be used to find the current track width of the object.

9.8.1 Detailed Description

A class to set the look and feel of a two finger linear slider to look like a trombone where the left most finger is the main body and the right most is the end of the trombones slide.

Note

This is used in the Tromboneless project to calibrate the maximum and minimum distances of the slider by the users preference.

9.8.2 Member Function Documentation

9.8.2.1 drawLinearSlider()

```
void CalibrationSliderLookAndFeel::drawLinearSlider (
            juce::Graphics & g,
            int x,
            int y,
            int width,
            int height,
            float sliderPos,
            float minSliderPos,
            float maxSliderPos,
            const juce::Slider::SliderStyle style,
            juce::Slider & slider )  [virtual]
```

When this look and feel is attached to an object which has a vertical slider in it, this function sets the look and feel of the slider being used. This makes it look like an average equalizer slider.

Parameters

g	A pointer to the `juce::Graphics` instance which is being drawn to. This is likely the main window or else the parent component.
x	The integer representation of the slider's left corner in the global window.
y	The integer representation of the sliders top corner in the global window.
width	The integer representation of the width of the slider. i.e. the difference between the left most and right most part of the slider.

Parameters

height	The integer representation of the width of the slider. i.e. the difference between the top most and bottom most parts of the slider's bounds.
sliderPos	The position of the sliders finger. The position represents x if the slider is horizontal and y if the slider is vertical.
minSliderPos	The minimum slider position.
maxSliderPos	The maximum slider position.
style	The style of the slider whose look and feel is getting modified. For example, LinearVertical or Rotary.
slider	A pointer to the slider which is being modified. This can allow for extraction of more data from the slider.

9.8.2.2 getEmptyTrackColour()

```
juce::Colour CalibrationSliderLookAndFeel::getEmptyTrackColour (
            void  )
```

A function to get the current colour of the empty track of the slider.

Return values

The	colour as a juce::Colour type.

9.8.2.3 getFullTrackColour()

```
juce::Colour CalibrationSliderLookAndFeel::getFullTrackColour (
            void  )
```

A function to get the current colour of the full track of the slider.

Return values

The	colour as a juce::Colour type.

Note

The full track refers to the part of the track between the two fingers/knobs

9.8.2.4 getMaxThumbColour()

```
juce::Colour CalibrationSliderLookAndFeel::getMaxThumbColour (
            void  )
```

A function to get the current colour of the maximum finger of the slider.

Return values

The	colour as a juce::Colour type.

9.8.2.5 getMinThumbColour()

```
juce::Colour CalibrationSliderLookAndFeel::getMinThumbColour (
            void  )
```

A function to get the current colour of the minimum finger of the slider.

Return values

The	colour as a juce::Colour type.

9.8.2.6 getTrackWidth()

```
float CalibrationSliderLookAndFeel::getTrackWidth (
            void  )
```

A function which may be used to find the current track width of the object.

Return values

Returns	the floating point representation of the track's width.

9.8.2.7 setEmptyTrackColour()

```
void CalibrationSliderLookAndFeel::setEmptyTrackColour (
            juce::Colour colour )
```

A function to allow for the empty track colour to be changed.

Parameters

colour	The colour which the emptry track should be set to.

9.8.2.8 setFullTrackColour()

```
void CalibrationSliderLookAndFeel::setFullTrackColour (
            juce::Colour colour )
```

A function to allow for the full track colour to be changed.

Parameters

colour	The colour which the full track should be set to.

Note

The full track refers to the part of the track between the two fingers/knobs

9.8.2.9 setMaxThumbColour()

```
void CalibrationSliderLookAndFeel::setMaxThumbColour (
            juce::Colour colour )
```

A function to allow for the colour of the second thumb to be changed.

Parameters

colour	The colour which the thumb should be set to.

9.8.2.10 setMinThumbColour()

```
void CalibrationSliderLookAndFeel::setMinThumbColour (
            juce::Colour colour )
```

A function to allow for the colour of the first thumb to be changed.

Parameters

colour	The colour which the thumb should be set to.

9.8.2.11 setTrackWidth()

```
void CalibrationSliderLookAndFeel::setTrackWidth (
            float width )
```

A function to set the width of the track to indicate what step the slider is positioned in.

Parameters

width	The width of the track in terms of the resolution.

The documentation for this class was generated from the following files:

- CustomStyles.hpp
- CustomStyles.cpp

9.9 CAP1188 Class Reference

Base driver class for the CAP1188.

```
#include <cap_sensor.hpp>
```

Public Types

- using **CAP1188CallbackInterface** = std::function< void(uint8_t)>

Public Member Functions

- CAP1188 (I2CBus &bus, uint8_t address=0x28, const std::string &gpioChipPath="/dev/gpiochip0", unsigned int gpioLine=27, uint8_t enabledChannels=0x01, uint8_t interruptEnabledChannels=0x01, bool isTest=false)

  Constructor for the CAP1188 base driver class.
- ~**CAP1188** ()

  Destructor for the CAP1188 base driver class. Calls "stop()" function.
- bool initialise ()

  Runs initialisation routine for driver.
- void **start** ()

  Configures the GPIO line for input and falling edge detection. Creates a thread with worker(). Can now have callback registered.
- void **stop** ()

  After thread has finished, releases it and all GPIO resources. This ensures data is released in a safe way.
- void **worker** ()

  Checks for data ready condition. When data is ready, call the handleDataReady() function.
- void **handleDataReady** ()

  If worker() determines data is ready this function is called. Gets the actual delta value on C1 channel and then clears the interrupt so that subsequent interrupts cause an edge event as opposed to maintaining a level. Publish delta value to the callback interface.
- void registerCallback (CAP1188CallbackInterface ci)

  Register the callback of pin 1 delta values to be published when pin 1 is touched.
- uint8_t getSensitivity ()

  Get the sensitivity value from the sensitivity control register.
- void setSensitivity (uint8_t sensitivityVal)

  Set the sensitivity value by writing to the sensitivity control register.
- uint8_t getAveraging ()

  Get the number of samples per average by reading from the Averaging and Sampling Config register.
- void setAveraging (uint8_t averagingVal)

  Set the sample number per channel average by writing to the Averaging and Sampling Config register.
- std::string getSampleTime ()

  Get the sampling time by reading from the Averaging and Sampling Config register.
- void setSampleTime (std::string newSampleTime)

  Set the sampling time by writing to the Averaging and Sampling Config register.
- std::string getCycleTime ()

  Get the cycle time by reading from the Averaging and Sampling Config register.
- void setCycleTime (std::string newCycleTime)

  Set the cycle time by writing to the Averaging and Sampling Config register.
- int touched ()

  Read the Sensor Input Status Register to find which pins are being touched.

- int8_t deltaCount (int pin)

  Set threshold for touched condition on one pin.
- void recalibratePins (uint8_t pins)

  Force-initiate the baseline re-calibration routine for given channels.
- std::array< uint8_t, 8 > getThresholds ()

  Get the touch thresholds for every pin.
- void setThresholds (std::array< uint8_t, 8 > newThresholds)

  Set new touch thresholds for every pin.
- void linkLEDs (uint8_t pinmask)

  Enable onboard LED functionality for given pin.
- void setInterruptRepeatRate (uint8_t rate)

  Sets the interrupt repeat according to a 4-bit word.
- void **enableChannels** ()

  Enable channels according to the enabledChannels setting bitmask.
- void **enableInterruptChannels** ()

  Enable interrupt channels according to the interruptEnabledChannels settings bitmask.
- void **disableStandby** ()

  Disable standby mode.
- void **clearInterrupt** ()

  Clear the interrupt bit. This is essential to do after data is read, to get edge events when the pin is touched, and so that the interrupt bit doesn't remain stuck after interrupt.

9.9.1 Detailed Description

Base driver class for the CAP1188.

Note

This driver has been created with reference to the CAP1188 datasheet and the CircuitPython driver for the Adafruit CAP1188 breakout board. This driver is specific to the Raspberry Pi, I2C protocol, and for the express purpose of using a single channel's delta measurement to determine a data ready condition, in which case the measurement is passed.

9.9.2 Constructor & Destructor Documentation

9.9.2.1 CAP1188()

```
CAP1188::CAP1188 (
            I2CBus & bus,
            uint8_t address = 0x28,
            const std::string & gpioChipPath = "/dev/gpiochip0",
            unsigned int gpioLine = 27,
            uint8_t enabledChannels = 0x01,
            uint8_t interruptEnabledChannels = 0x01,
            bool isTest = false )
```

Constructor for the CAP1188 base driver class.

Parameters

bus	I2Cbus object for the CAP1188 using i2c_bus

Parameters

address	I2C address for the CAP1188 sensor. With the Adafruit breakout board this defaults to 0x29. Wiring the '3Vo' pin to the 'AD' pin alters the address to 0x28. This is important as both the Time-of-Flight sensor and CAP1188 have the same default address. Default: 0x28.
gpioChipPath	Specifies the device file used by the linux kernel. So long as all I2C devices use the same chip path this can be any gpiochip. Default: "/dev/gpiochip0"
gpioLine	Specifies the GPIO pin on the Raspberry Pi used as the "data ready" pin.
enabledChannels	Specifies the enabled input channels. Only these channels will be sampled for capacitance values. Disabled pins will not be included in the sample cycle. This is formatted as a bit-mask of enabled pins from msb:C8 to lsb:C1. The default value is 0x01 = 0b0000'0001 i.e. Only C1.
interruptEnabledChannels	Specifies the interrupt enabled channels. Data ready will only be called when one of the specified pins have a touched status. This is formatted as a bit-mask of interrupt enabled pins from msb:C8 to lsb:C1. The default value is 0x01 = 0b0000'0001 i.e. Only C1.

9.9.3 Member Function Documentation

9.9.3.1 deltaCount()

```
int8_t CAP1188::deltaCount (
            int pin )
```

Set threshold for touched condition on one pin.

Parameters

pin	Pin number as integer from 1 to 8. This is used to source the relevant pin threshold register.
threshold	New threshold value for the threshold register to update to.

9.9.3.2 getAveraging()

```
uint8_t CAP1188::getAveraging ( )
```

Get the number of samples per average by reading from the Averaging and Sampling Config register.

Returns

uint8_t From the CAP1188 datasheet: "Bits 6 - 4 - AVG[2:0] - Determines the number of samples that are taken for all active channels during the sensor cycle... All samples are taken consecutively on the same channel before the next channel is sampled and the result is averaged over the number of samples measured before updating the measured results."

9.9.3.3 getCycleTime()

```
std::string CAP1188::getCycleTime ( )
```

Get the cycle time by reading from the Averaging and Sampling Config register.

Returns

std::string The cycle time determined by the CYCLE_TIME[B1:B0] buts. If the sensor input cycle ends befor the cycle time, then the system will wait for the full cycle time before initiating the next sensor input cycle. However, the cycle time is ignored if the sensor input cycle exceeds it.

9.9.3.4 getSampleTime()

```
std::string CAP1188::getSampleTime ( )
```

Get the sampling time by reading from the Averaging and Sampling Config register.

Returns

std::string The sampling time determined by the SAMP_TIME[1:0] bits. The sampling time is the period of time between each sample.

9.9.3.5 getSensitivity()

```
uint8_t CAP1188::getSensitivity ( )
```

Get the sensitivity value from the sensitivity control register.

Returns

uint8_t From the CAP1188 datasheet: "Bits 6-4 DELTA_SENSE[2:0] - Controls the sensitivity of a touch detection. The sensitivity settings act to scale the relative delta count value higher or lower based on the system parameters. A setting of 000b is the most sensitive while a setting of 111b is the least sensitive. At the more sensitive settings, touches are detected for a smaller delta capacitance corresponding to a “lighter” touch. These settings are more sensitive to noise, however, and a noisy environment may flag more false touches with higher sensitivity levels"

9.9.3.6 getThresholds()

```
std::array< uint8_t, 8 > CAP1188::getThresholds ( )
```

Get the touch thresholds for every pin.

Returns

std::array<uint8_t, 8> An array of size 8 (from C1 to C8) of every delta value (8-bit signed integer) at which the pin is considered touched

9.9.3.7 initialise()

```
bool CAP1188::initialise ( )
```

Runs initialisation routine for driver.

Returns

true Initialisation successful

false Initialisation failed

Note

This routine will open the I2C bus for the driver and check that a CAP1188 is found at the specified I2C address. Set enabled channels, set interrupt-enabled channels, enable LED-linking on enabled channels, set sampling cycle settings, set threshold for data-ready condition, enable interrupt on touch release, set the sensitivity, disable standby mode and clear the interrupt. After successful initialisation the sensor should be sampling correctly.

9.9.3.8 linkLEDs()

```
void CAP1188::linkLEDs (
            uint8_t pinmask )
```

Enable onboard LED functionality for given pin.

Parameters

pinmask	8-bit bitmask of channels to be linked to LED from C8(msb) to C1(lsb).

Note

LED will switch on if pin is touched

9.9.3.9 recalibratePins()

```
void CAP1188::recalibratePins (
            uint8_t pins )
```

Force-initiate the baseline re-calibration routine for given channels.

Parameters

pins	8-bit bitmask of channels to be re-calibrated from C8(msb) to C1(lsb).

9.9.3.10 registerCallback()

```
void CAP1188::registerCallback (
            CAP1188CallbackInterface ci )
```

Register the callback of pin 1 delta values to be published when pin 1 is touched.

Parameters

ci	Delta values as 8-bit unsigned integer word. This MUST be converted to an 8-bit signed integer (int8_t) later, as this is the true form of delta values.

9.9.3.11 setAveraging()

```
void CAP1188::setAveraging (
            uint8_t averagingVal )
```

Set the sample number per channel average by writing to the Averaging and Sampling Config register.

Parameters

averagingVal	The number of samples taken from each enabled channel during the sensor input cycle From the CAP1188 datasheet: "Bits 6 - 4 - AVG[2:0] - Determines the number of samples that are taken for all active channels during the sensor cycle... All samples are taken consecutively on the same channel before the next channel is sampled and the result is averaged over the number of samples measured before updating the measured results." Must be

9.9.3.12 setCycleTime()

```
void CAP1188::setCycleTime (
            std::string newCycleTime )
```

Set the cycle time by writing to the Averaging and Sampling Config register.

Parameters

newCycleTime	The cycle time determined by the CYCLE_TIME[B1:B0] buts. If the sensor input cycle ends befor the cycle time, then the system will wait for the full cycle time before initiating the next sensor input cycle. However, the cycle time is ignored if the sensor input cycle exceeds it. Options: {"35ms", "70ms", "105ms", "140ms"}

9.9.3.13 setInterruptRepeatRate()

```
void CAP1188::setInterruptRepeatRate (
            uint8_t rate )
```

Sets the interrupt repeat according to a 4-bit word.

Parameters

rate	New interrupt repeat rate. Starts at 35ms = 0b0000 and increments by 35ms for every added +1.

9.9.3.14 setSampleTime()

```
void CAP1188::setSampleTime (
            std::string newSampleTime )
```

Set the sampling time by writing to the Averaging and Sampling Config register.

Parameters

newSampleTime	The sampling time determined by the SAMP_TIME[B3:B2] bits. The sampling time is the period of time between each sample. Options: {"320us", "640us", "1.28ms", "2.56ms"}

9.9.3.15 setSensitivity()

```
void CAP1188::setSensitivity (
            uint8_t sensitivityVal )
```

Set the sensitivity value by writing to the sensitivity control register.

Parameters

sensitivityVal	From the CAP1188 datasheet: "Bits 6-4 DELTA_SENSE[2:0] - Controls the sensitivity of a touch detection. The sensitivity settings act to scale the relative delta count value higher or lower based on the system parameters. A setting of 000b is the most sensitive while a setting of 111b is the least sensitive. At the more sensitive settings, touches are detected for a smaller delta capacitance corresponding to a “lighter” touch. These settings are more sensitive to noise, however, and a noisy environment may flag more false touches with higher sensitivity levels"

9.9.3.16 setThresholds()

```
void CAP1188::setThresholds (
            std::array< uint8_t, 8 > newThresholds )
```

Set new touch thresholds for every pin.

Parameters

newThresholds	Size 8 array of signed 8-bit integer. From C1 to C8, every 8-bit word contains the delta value after which the delta value is considered touching.

9.9.3.17 touched()

```
int CAP1188::touched ( )
```

Read the Sensor Input Status Register to find which pins are being touched.

Returns

int An 8-bit pinmask of touched pins, from pin CS8:CS1. For example: 0110'0001 means pins 7, 6, and 1 are touched. The touched status is determined internally by the chip by comparing the current delta values with a pre-determined threshold. Only enabled channels can be registered as touched.

The documentation for this class was generated from the following files:

- cap_sensor.hpp
- cap_sensor.cpp

9.10 CAP1188Channel Class Reference

Helper class unique to one sensor channel.

```
#include <cap_sensor.hpp>
```

Public Member Functions

- bool isPinTouched ()

  Function to find whether this channel's pin is touched.
- int8_t pinDelta ()

  Function to find the delta value for this channel.
- int getThreshold ()

  Getter function for pin's threshold value.
- void setThreshold (int threshold)

  Setter function for pin's threshold value.
- void pinRecalibrate ()

  Function to initiate recalibration routine for this channel.

Friends

- class **CAP1188**

9.10.1 Detailed Description

Helper class unique to one sensor channel.

Note

This class is used to call base driver functions specific to only one input channel. When the base driver is initialised, a unique channel class is created for each of the 8 pins.

9.10.2 Member Function Documentation

9.10.2.1 getThreshold()

```
int CAP1188Channel::getThreshold ( )
```

Getter function for pin's threshold value.

Returns

int This channel's threshold value

9.10.2.2 isPinTouched()

```
bool CAP1188Channel::isPinTouched ( )
```

Function to find whether this channel's pin is touched.

Returns

true The pin is being touched

false The pin is not being touched

Note

This function points to the base driver's touched() function, passing only this pin to it. The state of being 'touched' is defined as having a delta greater than the threshold.

9.10.2.3 pinDelta()

```
int8_t CAP1188Channel::pinDelta ( )
```

Function to find the delta value for this channel.

Returns

int8_t This channel's delta value

Note

This function points to the base driver's deltaCount() function, passing only this pin to it. The delta value is defined as the difference from the current capacitance to the baseline capacitance.

9.10.2.4 pinRecalibrate()

```
void CAP1188Channel::pinRecalibrate ( )
```

Function to initiate recalibration routine for this channel.

Note

This function points to the base driver's recalibratePins function, passing only this pin to it.

9.10.2.5 setThreshold()

```
void CAP1188Channel::setThreshold (
            int threshold )
```

Setter function for pin's threshold value.

Parameters

threshold	This channel's new threshold value.

The documentation for this class was generated from the following files:

- cap_sensor.hpp
- cap_sensor.cpp

9.11 CAP1188Printer Class Reference

Public Member Functions

- void **hasCAPSample** (int8_t delta)

The documentation for this class was generated from the following file:

- cap1188_test.cpp

9.12 CoreWrapper Class Reference

Public Member Functions

- **CoreWrapper** (bool isTest)
- void **start** ()
- void **stop** ()
- EventHandler ∗ **getEventHandler** (void)
- RtMidiSink ∗ **getRtMidiSink** (void)
- AudioRender ∗ **getAudioRender** (void)
- AmplitudeMapper ∗ **getAmplitudeMapper** (void)
- PitchMapper ∗ **getPitchMapper** (void)
- MidiCoordinator ∗ **getMidiCoordinator** (void)

The documentation for this class was generated from the following files:

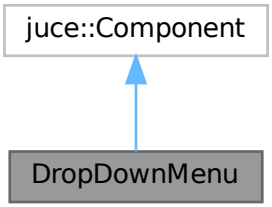
- CoreWrapper.hpp
- CoreWrapper.cpp

9.13 DropDownMenu Class Reference

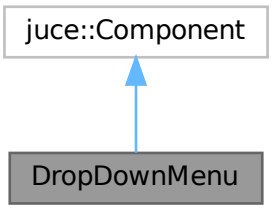
A class that creates a drop down menu or 'comboBox' with a label attached to it. This takes up 95% of the available local area. The label is set to have a maxmimum width of 200, but scales as the window size is reduced.

```
#include <Widgets.hpp>
```

Inheritance diagram for DropDownMenu:



Collaboration diagram for DropDownMenu:



Public Member Functions

- int **getSelectedId** () const
- template<typename Fn >
  void **SetOnChange** (Fn &&callback)
- DropDownMenu (juce::String labelPhrase="ComboBox")

  Constructor for the DropDownMenu class that creates an instance of the class. This makes a juce component that has a combobox or drop-down menu within it with a label attached to its left by default.
- ~**DropDownMenu** ()=default

  Destructor for the DropDownMenu which is set to default as there are no components within this class that need to be deinitialised before the deconstruction of the object.
- void **resized** () override

  Adding a function that will allow for the `DropDownMenu` to be resized when the window size is adjusted to fit best.
- void AddItem (juce::String itemName, int itemValue)

A function which allows for the addition of more items within the drop down menu.

- template<typename T >
  void OnChange (T *updateVariable)

  This function sets what will happen when the option wihtin the drop down menu has been changed by the user.
- void ChangeLabelText (juce::String text, juce::NotificationType notification=juce::dontSendNotification)

  This function allows for the editting of the main label displayed beside the drop down menu.
- juce::String getLabelText (void)

  A function which returns the current text which the label is displaying.

9.13.1 Detailed Description

A class that creates a drop down menu or 'comboBox' with a label attached to it. This takes up 95% of the available local area. The label is set to have a maxmimum width of 200, but scales as the window size is reduced.

9.13.2 Constructor & Destructor Documentation

9.13.2.1 DropDownMenu()

```
DropDownMenu::DropDownMenu (
            juce::String labelPhrase = "ComboBox" )
```

Constructor for the DropDownMenu class that creates an instance of the class. This makes a juce component that has a combobox or drop-down menu within it with a label attached to its left by default.

Parameters

labelPhrase	This parameter should be of type juce::String and contain the phrase which should be displayed by the label. By default this is set to "ComboBox" so it may be viewed upon creation.

9.13.3 Member Function Documentation

9.13.3.1 AddItem()

```
void DropDownMenu::AddItem (
            juce::String itemName,
            int itemValue )
```

A function which allows for the addition of more items within the drop down menu.

Parameters

itemName	This parameter of type `juce::String` should contain the phrase which will be displayed to the end user on the display to select this option.
itemValue	This is the unique number assigned to each of the items. It must be more than 0 for it to be displayed. Each item must have a different value for them to be distinguishable from each other.

9.13.3.2 ChangeLabelText()

```
void DropDownMenu::ChangeLabelText (
            juce::String text,
            juce::NotificationType notification = juce::dontSendNotification )
```

This function allows for the editting of the main label displayed beside the drop down menu.

Parameters

text	This parameter should contain the text to which the label should be updated to display.
notification	This is the type of notification which should be attached to the drop down menu. By default this value is set to `juce::dontSendNotification`.

9.13.3.3 getLabelText()

```
juce::String DropDownMenu::getLabelText (
            void )
```

A function which returns the current text which the label is displaying.

Return values

Returns	the text being distplayed as a type `juce::String`.

9.13.3.4 OnChange()

```
template<typename T >
void DropDownMenu::OnChange (
            T * updateVariable )  [inline]
```

This function sets what will happen when the option wihtin the drop down menu has been changed by the user.

Parameters

updateVariable	This should be a pointer to the variable which is being set to the item index which has been chosen.

Note

This function uses a lambda function to update the specified variable to ensure that the specified variable may be changed for different instances of the class.

The documentation for this class was generated from the following files:

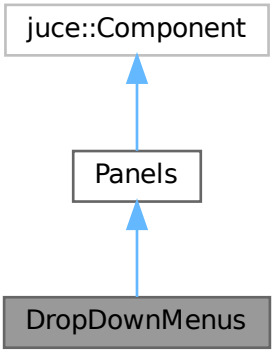
- Widgets.hpp
- Widgets.cpp

9.14 DropDownMenus Class Reference

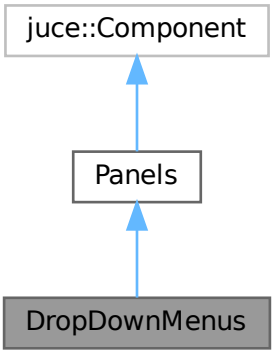
A class which is created to display the drop down menus required by the Tromboneless UI to choose the chosen option for calibration settings.

```
#include <Panels.hpp>
```

Inheritance diagram for DropDownMenus:



Collaboration diagram for DropDownMenus:



Public Member Functions

- **DropDownMenus** (CoreWrapper &coreWrapper)

  The constructor function for the `DropDownMenus` class which creates an instance of the class and adds the chosen options to the shift keying drop down specifically. This also makes the components within it visible to the user.

- ~**DropDownMenus** ()=default

  The destructor function for the `DropDownMenus` class which is set to default as no child object within the instance need to be altered within the class.
- void **resized** () override

  Adding a function that will allow for the `DropDownMenus` to be resized when the window size is adjusted to fit best.
- void **shiftKeyChoiceChanged** ()

Public Member Functions inherited from Panels

- **Panels** ()=default

  The constructor function for the `Panels` class which currently provides no other function than creating an instance of the `Panels` class. I.e. no initialisation functions or settings.
- ~**Panels** ()=default

  The destructor function for the `Panels` class which currently does nothing extra as there is no initialisation within the class.
- void paint (juce::Graphics &g) override

  This function draws onto the parent function the default shape and colour of the panels designed.
- void setBackgroundColour (juce::Colour colour)

  A function to set the background colour of the panels for the main window.
- juce::Colour getBackgroundColour (void)

  A function to get the current background colour being used for the panels within the main window.

9.14.1 Detailed Description

A class which is created to display the drop down menus required by the Tromboneless UI to choose the chosen option for calibration settings.

The documentation for this class was generated from the following files:

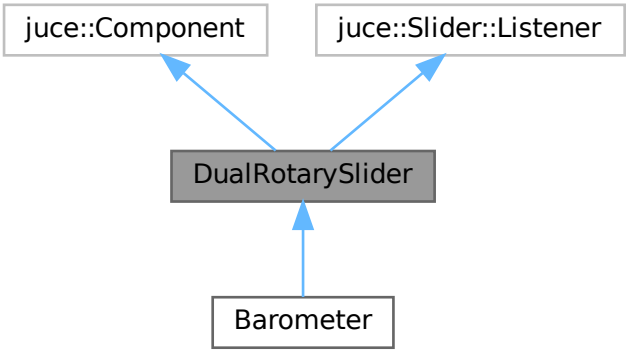
- Panels.hpp
- Panels.cpp

9.15 DualRotarySlider Class Reference

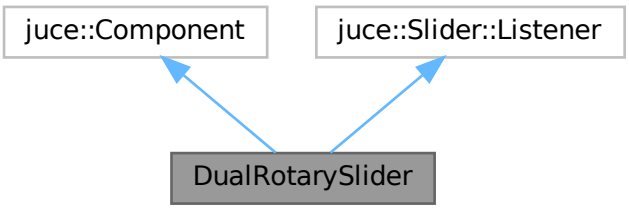
A class that adds more functionality to the slider class from Juce.

```
#include <Widgets.hpp>
```

Inheritance diagram for DualRotarySlider:



Collaboration diagram for DualRotarySlider:



Public Member Functions

- **DualRotarySlider** ()

  The constructor function for the `DualRotarySlider` class which sets the range of the sliders, minimum distance and adds listeners to them both so that when the sliders are moved, the corresponding variables may be changed.
- ~**DualRotarySlider** ()

  The destructor function for the `DualRotarySlider` class which removes the listeners from the sliders.
- void sliderValueChanged (juce::Slider *sliderChanged) override

  A function that is called upon when either of the fingers are moved.
- void **resized** () override

  A callback function for when the window size has changed. This allows for the labels and sliders to be resized to be the best relative size. This can either be set to a predetermined size like 100px x 200px, but it is currrently set to be relative to the size of the screen.
- void setMinDifference (float difference)

  A function to set the minimum difference between the sliders 2 fingers.
- float getMinDifference (void)

A function to get the current minimum difference between the 2 slider fingers.

- void setMinSliderRadius (float radius)

  A function to set the relative radius of the minimum finger slider. It should always be less than or equal to the maximum slider radius and be between 0 and 1.
- float getMinSliderRadius (void)

  A function to get the relative radius of the minimum finger slider for this object.
- void setMaxSliderRadius (float radius)

  A function to set the relative radius of the maximum finger slider. It should always be more than or equal to the minimum slider radius and be between 0 and 1.
- float getMaxSliderRadius (void)

  A function to get the relative radius of the maximum finger slider for this object.
- void setLabelHeight (float height)

  A function to set the maximum height of the label attached to the rotary slider.
- float getLabelHeight (void)

  A function to get the current height of the label which is attached to the rotary slider.
- void setMinLimit (float limit)

  A function to set the minimum limit of the rotary slider. This may be any floating point value that is less than the difference between the maximum value and the minimum difference between the slider fingers.
- float getMinLimit (void)

  A function to get the current minium limit of the rotary slider.
- void setMaxLimit (float limit)

  A function to set the maximum limit of the rotary slider. This may be any floating point value that is more than the sum of the minium value and the minimum difference between the slider fingers.
- float getMaxLimit (void)

  A function to get the current maximum limit of the rotary slider.
- void setInterval (float step)

  A function to set the minimum step the slider may be moved by the user. This value is in the same scale as the maxLimit and minLimit.
- float getInterval (void)

  A function to get the current minimum step which the slider may be adjusted by the user. This value is in the same scale as the maxLimit and minLimit.
- void setMinAngle (float angle)

  A function to allow for the minimum angle (furtherest anticlockwise) that the slider may be rotated to be set. This is measured in radians.
- float getMinAngle (void)

  A function to return the current minimum angle (furtherest anticlockwise) that the slider may be rotated. This is measured in radians.
- void setMaxAngle (float angle)

  A function to allow for the maximum angle (furtherest clockwise) that the slider may be rotated to be set. This is measured in radians.
- float getMaxAngle (void)

  A function to return the current maximum angle (furtherest clockwise) that the slider may be rotated. This is measured in radians.

Public Attributes

- juce::Slider maxSlider
- juce::Slider minSlider
- juce::Label rotaryLabel

9.15.1 Detailed Description

A class that adds more functionality to the slider class from Juce.

9.15.2 Member Function Documentation

9.15.2.1 getInterval()

```
float DualRotarySlider::getInterval (
            void  )
```

A function to get the current minimum step which the slider may be adjusted by the user. This value is in the same scale as the maxLimit and minLimit.

Return values

The	floating point representation of the current minimum step.

9.15.2.2 getLabelHeight()

```
float DualRotarySlider::getLabelHeight (
            void  )
```

A function to get the current height of the label which is attached to the rotary slider.

Return values

The	height of the label as a floating point value.

9.15.2.3 getMaxAngle()

```
float DualRotarySlider::getMaxAngle (
            void  )
```

A function to return the current maximum angle (furtherest clockwise) that the slider may be rotated. This is measured in radians.

Return values

The	angle in radians as a floating point value.

9.15.2.4 getMaxLimit()

```
float DualRotarySlider::getMaxLimit (
            void  )
```

A function to get the current maximum limit of the rotary slider.

Return values

The	floating point representation of the minimum limit.

9.15.2.5 getMaxSliderRadius()

```
float DualRotarySlider::getMaxSliderRadius (
            void  )
```

A function to get the relative radius of the maximum finger slider for this object.

Return values

Returns	the floating point representation of the relative radius.

9.15.2.6 getMinAngle()

```
float DualRotarySlider::getMinAngle (
            void  )
```

A function to return the current minimum angle (furtherest anticlockwise) that the slider may be rotated. This is measured in radians.

Return values

The	angle in radians as a floating point value.

9.15.2.7 getMinDifference()

```
float DualRotarySlider::getMinDifference (
            void  )
```

A function to get the current minimum difference between the 2 slider fingers.

Return values

Returns	the floating point representation of the minimum difference between the 2 sliders in terms of the chosen maximum and minimum values.

9.15.2.8 getMinLimit()

```
float DualRotarySlider::getMinLimit (
            void  )
```

A function to get the current minium limit of the rotary slider.

Return values

The	floating point representation of the minimum limit.

9.15.2.9 getMinSliderRadius()

```
float DualRotarySlider::getMinSliderRadius (
            void  )
```

A function to get the relative radius of the minimum finger slider for this object.

Return values

Returns	the floating point representation of the relative radius.

9.15.2.10 setInterval()

```
void DualRotarySlider::setInterval (
            float step )
```

A function to set the minimum step the slider may be moved by the user. This value is in the same scale as the maxLimit and minLimit.

Parameters

step	The minimum step as a floating point value.

9.15.2.11 setLabelHeight()

```
void DualRotarySlider::setLabelHeight (
            float height )
```

A function to set the maximum height of the label attached to the rotary slider.

Parameters

height	The height of the label in terms of the resolution as a floating point value.

9.15.2.12 setMaxAngle()

```
void DualRotarySlider::setMaxAngle (
            float angle )
```

A function to allow for the maximum angle (furtherest clockwise) that the slider may be rotated to be set. This is measured in radians.

Parameters

angle	The angle in radians as a floating point value.

9.15.2.13 setMaxLimit()

```
void DualRotarySlider::setMaxLimit (
            float limit )
```

A function to set the maximum limit of the rotary slider. This may be any floating point value that is more than the sum of the minium value and the minimum difference between the slider fingers.

Parameters

limit	The floating point representation of the limit.

9.15.2.14 setMaxSliderRadius()

```
void DualRotarySlider::setMaxSliderRadius (
            float radius )
```

A function to set the relative radius of the maximum finger slider. It should always be more than or equal to the minimum slider radius and be between 0 and 1.

Parameters

radius	The relative radius that the maximum slider should have.

9.15.2.15 setMinAngle()

```
void DualRotarySlider::setMinAngle (
            float angle )
```

A function to allow for the minimum angle (furtherest anticlockwise) that the slider may be rotated to be set. This is measured in radians.

Parameters

angle	The angle in radians as a floating point value.

9.15.2.16 setMinDifference()

```
void DualRotarySlider::setMinDifference (
            float difference )
```

A function to set the minimum difference between the sliders 2 fingers.

Parameters

difference	The difference in terms of the chosen maximum and minimum values as a floating point value.

9.15.2.17 setMinLimit()

```
void DualRotarySlider::setMinLimit (
            float limit )
```

A function to set the minimum limit of the rotary slider. This may be any floating point value that is less than the difference between the maximum value and the minimum difference between the slider fingers.

Parameters

limit	The floating point representation of the limit.

9.15.2.18 setMinSliderRadius()

```
void DualRotarySlider::setMinSliderRadius (
            float radius )
```

A function to set the relative radius of the minimum finger slider. It should always be less than or equal to the maximum slider radius and be between 0 and 1.

Parameters

radius	The relative radius that the minimum slider should have.

9.15.2.19 sliderValueChanged()

```
void DualRotarySlider::sliderValueChanged (
            juce::Slider * sliderChanged )  [override]
```

A function that is called upon when either of the fingers are moved.

Note

This function automatically moves the fingers to ensure that they stay at least the minimum range apart from each other.

9.15.3 Member Data Documentation

9.15.3.1 maxSlider

```
juce::Slider DualRotarySlider::maxSlider
```

The slider which will represent the high end of the calibration.

9.15.3.2 minSlider

```
juce::Slider DualRotarySlider::minSlider
```

The slider which will represent the low end of the calibration.

9.15.3.3 rotaryLabel

```
juce::Label DualRotarySlider::rotaryLabel
```

The label of the component.

The documentation for this class was generated from the following files:

- Widgets.hpp
- Widgets.cpp

9.16 Envelope Class Reference

A class which can allow for the create of an envelope function which outputs a value between 0 and 1 depending on what stage the envelope is in. This uses the ADSR standard in music synthesis where A is the Attack, D is the decay, S is the saturation and R is the rest stage.

```
#include <envelope.hpp>
```

Public Types

- enum envelope_stages_t {
  no_stage , **attack_stage** , **decay_stage** , **sustain_stage** ,
  rest_stage }

  Enumerated type to describe which part of the ADSR the object is in.

Public Member Functions

- Envelope (int attack_in=10, int decay_in=20, float sustain_in=0.98, int rest_in=10)

  The constructor function for the `Envelope` class which sets the values of the ADSR of the envelope and initialises the beginning of it.
- ~**Envelope** ()=default

  The destructor function for the `Envelope` class which is currently just set to default.
- float getAmplitude (void)

  A function that will return the current maximum amplitude of the envelope function. Depending one how many times this has been called sets at which stage the envelope is in.
- void **startEnvelope** (void)

  A function that enters the device into the attack stage.
- void **endEnvelope** (void)

  A function which sets the envelope to enter the rest stage and begin to approach 0.
- void setAttack (int attack_in)

  A function which can allow for the attack value to be changed externally from the class.
- int getAttack (void)

  A function which returns the number of times the `getAmplitude()` function needs to be called to go from 0 to 1 for the envelope gain value.
- void setDecay (int decay_in)

  A function which can allow for the decay value to be changed externally from the class.
- int getDecay (void)

  A function which returns the number of times the `getAmplitude()` function needs to be called to go from 1 to the sustain level for the envelope gain value.

- void setSustain (float sustain_in)

  A function which can allow for the sustain value to be changed externally from the class.
- float getSustain (void)

  A function which returns the value of the sustain level currently set for the object.
- void setRest (int rest_in)

  A function which can allow for the rest value to be changed externally from the class.
- int getRest (void)

  A function which can allow for the rest value to be used outside of the object without affecting the objects instance of the value from changing.
- float attackFunction (void)

  This function determines the shape of the attack curve. It needs to start from 0 and end at 1.
- float decayFunction (void)

  This function determines the shape of the decay curve. It needs to start from 1 and finish at the sustain value which has been set by the object.
- float restFunction (void)

  This function determines the shape of the rest curve. It needs to start from the sustain value and end at 0.
- envelope_stages_t getStage (void)

  A function which gets the current stage which the envelope is in.
- void setStage (envelope_stages_t stage_in)

  A function which sets the current stage which the envelope is in.

9.16.1 Detailed Description

A class which can allow for the create of an envelope function which outputs a value between 0 and 1 depending on what stage the envelope is in. This uses the ADSR standard in music synthesis where A is the Attack, D is the decay, S is the saturation and R is the rest stage.

Note

The public functions to set up the envelope take the time in milliseconds for each parameter as well as the sample rate to determine the number of samples each stage should last for.

9.16.2 Constructor & Destructor Documentation

9.16.2.1 Envelope()

```
Envelope::Envelope (
            int attack_in = 10,
            int decay_in = 20,
            float sustain_in = 0.98,
            int rest_in = 10 )
```

The constructor function for the `Envelope` class which sets the values of the ADSR of the envelope and initialises the beginning of it.

Parameters

attack_in	The number of times the `getAmplitude()` function should be called before the envelope reaches its peak of 1.
decay_in	The number of times the `getAmplitude()` function should be called after the attack stage to reach the sustain level.
sustain↩	The relative amplitude of the signal after the decay stage.
_in	**Generated by Doxygen**
rest_in	The number of times the `getAmplitde()` function should be called once the endNote flag has been triggered before the noise reaches an amplitude of 0.

Note

This class does not depend upon the real time or clock of the device, but instead the number of times the `getAmplitude()` function is called upon.

9.16.3 Member Function Documentation

9.16.3.1 attackFunction()

```
float Envelope::attackFunction (
            void  )
```

This function determines the shape of the attack curve. It needs to start from 0 and end at 1.

Return values

Returns	the amplitude at the time of calling.

Note

This is currently a linear ramp function between 0 and 1.

9.16.3.2 decayFunction()

```
float Envelope::decayFunction (
            void  )
```

This function determines the shape of the decay curve. It needs to start from 1 and finish at the sustain value which has been set by the object.

Return values

Returns	the floating point representation of the ampltiude at the time of request.

Note

This is currently a linear ramp function between 1 and the sustain value which has been set.

9.16.3.3 getAmplitude()

```
float Envelope::getAmplitude (
            void  )
```

A function that will return the current maximum amplitude of the envelope function. Depending one how many times this has been called sets at which stage the envelope is in.

Return values

A	floating point value between 0 and 1 representing the gain which would be applied to a signal it is attached to.

9.16.3.4 getAttack()

```
int Envelope::getAttack (
            void  )
```

A function which returns the number of times the `getAmplitude()` function needs to be called to go from 0 to 1 for the envelope gain value.

Return values

The	integer representation of how many times the the `getAmplitude()` function needs to be called to exit the attack stage.

9.16.3.5 getDecay()

```
int Envelope::getDecay (
            void  )
```

A function which returns the number of times the `getAmplitude()` function needs to be called to go from 1 to the sustain level for the envelope gain value.

Return values

The	integer representation of how many times the the `getAmplitude()` function needs to be called to exit the decay stage and enter sustain.

9.16.3.6 getRest()

```
int Envelope::getRest (
            void  )
```

A function which can allow for the rest value to be used outside of the object without affecting the objects instance of the value from changing.

Return values

Returns	the number of times the `getAmplitude()` function needs to be called before the ampltude reaches 0 from the sustain value.

9.16.3.7 getStage()

```
Envelope::envelope_stages_t Envelope::getStage (
```

```
            void  )
```

A function which gets the current stage which the envelope is in.

Return values

The	stage at which the envelope is in as type `envelope_stages_t`.

9.16.3.8 getSustain()

```
float Envelope::getSustain (
            void  )
```

A function which returns the value of the sustain level currently set for the object.

Return values

A	floating point representation of the sustian amplitude.

9.16.3.9 restFunction()

```
float Envelope::restFunction (
            void  )
```

This function determines the shape of the rest curve. It needs to start from the sustain value and end at 0.

Return values

Returns	the floating point representation of the amplitude at the time of request.

Note

This is currently a linear ramp function between the sustain
value set and 0. This may be modified to match the shape of the instrument better.

9.16.3.10 setAttack()

```
void Envelope::setAttack (
            int attack_in )
```

A function which can allow for the attack value to be changed externally from the class.

Parameters

attack↩ _in	The number of times the `getAmplitude()` function should be called before the envelope reaches its peak of 1.

9.16.3.11 setDecay()

```
void Envelope::setDecay (
            int decay_in )
```

A function which can allow for the decay value to be changed externally from the class.

Parameters

decay↩ _in	The number of times the `getAmplitude()` function should be called before the output goes from 1 to the chosen sustain value.

9.16.3.12 setRest()

```
void Envelope::setRest (
            int rest_in )
```

A function which can allow for the rest value to be changed externally from the class.

Parameters

rest↩ _in	The number of times the `getAmplitude()` function should be called before the envelope reaches its peak of 0 from the sustain value.

9.16.3.13 setStage()

```
void Envelope::setStage (
            envelope_stages_t stage_in )
```

A function which sets the current stage which the envelope is in.

Parameters

stage↩ _in	The stage at which the envelope should enter or already be in.

9.16.3.14 setSustain()

```
void Envelope::setSustain (
            float sustain_in )
```

A function which can allow for the sustain value to be changed externally from the class.

Parameters

sustain↩ _in	The relative amplitude that the envelope should hold after the decay stage. This should be a floating point value between 0 and 1.

The documentation for this class was generated from the following files:

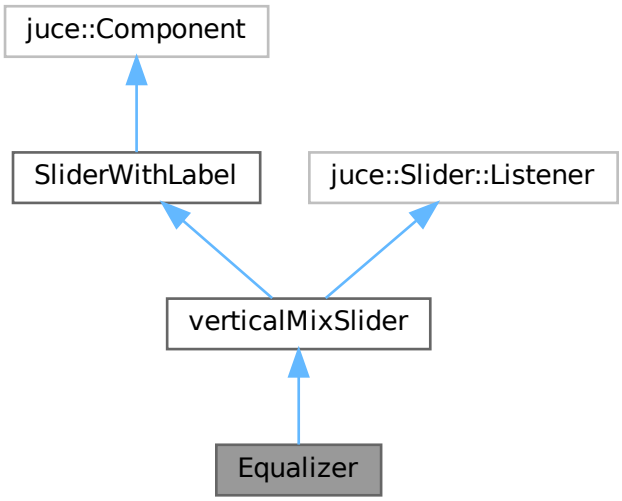
- envelope.hpp
- envelope.cpp

9.17 Equalizer Class Reference

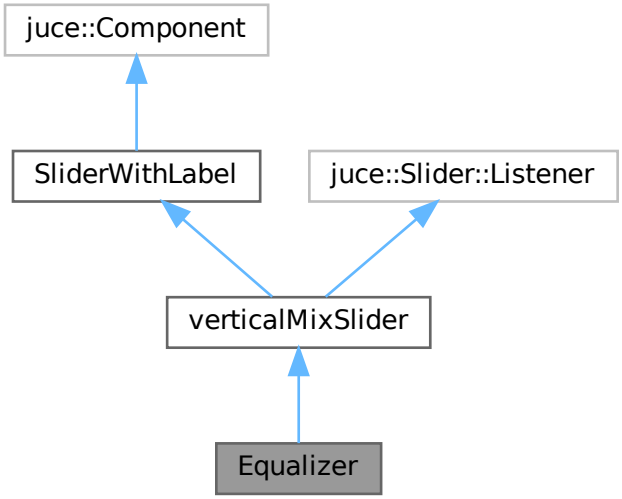
A class which inherits its sliders from `verticalMixSlider` made to act as an equalizer.

```
#include <Widgets.hpp>
```

Inheritance diagram for Equalizer:



Collaboration diagram for Equalizer:



Public Member Functions

- Equalizer (char numberOfSliders=10)

  Contructor for the Equalizer class which creates a set of n vertical sliders in the style of an equalizer.
- ~**Equalizer** ()

  Destructor for the Equalizer which removes the slider listeners from each of the sliders as well as resetting the look and feel of each of them.
- void **resized** () override

  A function to set the size of the sliders with respect to the equalizer size. Whenever the parent component changes size, this function is called upon.
- void sliderValueChanged (juce::Slider *sliderChanged) override

  A callback function for when one of the sliders within the equalizer has been detected to have changed in value. This is where the corresponding variables the sliders affect may be changed.

Public Member Functions inherited from verticalMixSlider

- **verticalMixSlider** ()

  Constructor function for the vericalMixSlider which creates an instance of the class and sets the parameters to match it best such as setting the range to be between -9 dB and 9 dB with a step size of 3 dB as well as label size.
- ~**verticalMixSlider** ()

  Destructor function for the verticalMixSlider class which removes the listener from the slider before killing the object.
- void sliderValueChanged (juce::Slider *sliderChanged) override

  A callback function for when the slider used has been adjusted and its value may have been changed.

Public Member Functions inherited from SliderWithLabel

- **SliderWithLabel** (juce::Slider::SliderStyle style)

  Constructor which sets the style of the slider being used and removes the text entry option.
- InitialiseLabelErrors_t CreateLabel (LabelPositions_t position, juce::String phrase, juce::NotificationType NotificationEnable=juce::dontSendNotification)

  A function which may be called to create or edit a specified label around the slider.
- void resized () override

  A function which is automatically called whenever the window size is modified. This automatically adjusts the bounds of each of the labels as well as the slider to ensure the size is correct with respect to the screen size for the component.
- void setTopLabelBounds (int bound)

  A function to set the bounds for which the top labels may be situated in.
- int getTopLabelBounds (void)

  A function to get the bounds for which the top labels are situated in.
- void setLeftLabelBounds (int bound)

  A function to set the bounds for which the labels on the left may be situated in.
- int getLeftLabelBounds (void)

  A function to get the bounds for which the labels on the left are situated in.
- void setRightLabelBounds (int bound)

  A function to set the bounds for which the labels on the right may be situated in.
- int getRightLabelBounds (void)

  A function to get the bounds for which the labels on the right are situated in.
- void setBottomLabelBounds (int bound)

  A function to set the bounds for which the bottom labels may be situated in.
- int getBottomLabelBounds (void)

  A function to get the bounds for which the bottom labels are situated in.

Public Attributes

- juce::OwnedArray< verticalMixSlider > **eqSliders**

  Creating an array of sliders for the equalizer.

Public Attributes inherited from SliderWithLabel

- juce::Slider **slider**

  The slider which is having the labels added to it.

Additional Inherited Members

Public Types inherited from SliderWithLabel

- enum LabelPositions_t {
  UpperLeft , **UpperCentre** , **UpperRight** , **MiddleLeft** ,
  MiddleRight , **LowerLeft** , **LowerCentre** , **LowerRight** }

  Enumerated list of possible positions for the label to be attached relative to the slider.
- enum InitialiseLabelErrors_t { **LabelOK** , **PositonNotAccepted** }

  Enumerated list of possible errors when initialising the labels within this component.

9.17.1 Detailed Description

A class which inherits its sliders from `verticalMixSlider` made to act as an equalizer.

Parameters

nSliders	The number of sliders which should be added to the object. 10 by default.

9.17.2 Constructor & Destructor Documentation

9.17.2.1 Equalizer()

```
Equalizer::Equalizer (
            char numberOfSliders = 10 )
```

Contructor for the Equalizer class which creates a set of n vertical sliders in the style of an equalizer.

Parameters

numberOfSliders	The number of sliders which are present within the equalizer. This is set to 10 by default by may be altered.

Note

Caution: If the nuimberOfSliders is set to be higher than the number of elements in the `gains` variable in `SynthesiserParameters_t`, then the extra sliders will not affect any variable.

9.17.3 Member Function Documentation

9.17.3.1 sliderValueChanged()

```
void Equalizer::sliderValueChanged (
            juce::Slider * sliderChanged )  [override]
```

A callback function for when one of the sliders within the equalizer has been detected to have changed in value. This is where the corresponding variables the sliders affect may be changed.

Parameters

sliderChanged	A pointer to the slider which has been changed.

Note

As the equalizer has multiple sliders within it, it is important to check which of the sliders has been changed before altering any values. This function uses predefined variables which are affected by each of the sliders. If you wish to change this, you must change it within the function defintion as there are no functions to call to change the variable.

The documentation for this class was generated from the following files:

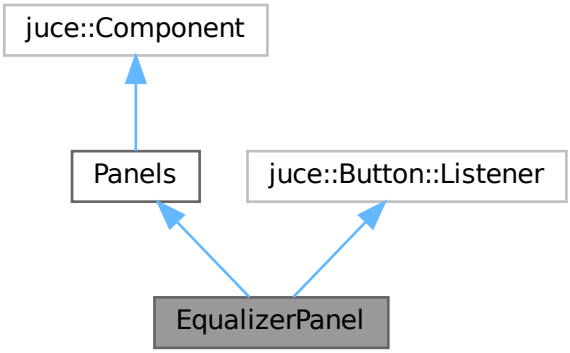
- Widgets.hpp
- Widgets.cpp

9.18 EqualizerPanel Class Reference

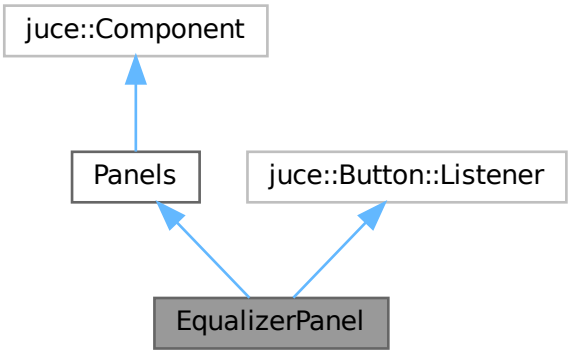
A class which may be called to add a panel to the window contains a series of sliders for the equalizer allowing for live editting of the synthesisers sound to ensure it sounds how the Trombonist wishes it to.

```
#include <Panels.hpp>
```

Inheritance diagram for EqualizerPanel:



Collaboration diagram for EqualizerPanel:



Public Member Functions

- **EqualizerPanel** (CoreWrapper &coreWrapper)

  The constructor function for the `EqualizerPanel` class which adds the child components to the parent window and attaches a listener to the equalizer to detect when the slider values have been adjusted by the user.

- ~**EqualizerPanel** ()

  The destructor function which for the `EqualizerPanel` class which removes the listener from the equalizer before deinitialising.
- void resized () override

  A function which changes the size of the child components when the parent window has changed in size.
- void buttonClicked (juce::Button *buttonClicked) override

  A callback function which is called upon automatically when the user has pressed a button within this class.

Public Member Functions inherited from Panels

- **Panels** ()=default

  The constructor function for the `Panels` class which currently provides no other function than creating an instance of the `Panels` class. I.e. no initialisation functions or settings.
- ~**Panels** ()=default

  The destructor function for the `Panels` class which currently does nothing extra as there is no initialisation within the class.
- void paint (juce::Graphics &g) override

  This function draws onto the parent function the default shape and colour of the panels designed.
- void setBackgroundColour (juce::Colour colour)

  A function to set the background colour of the panels for the main window.
- juce::Colour getBackgroundColour (void)

  A function to get the current background colour being used for the panels within the main window.

9.18.1 Detailed Description

A class which may be called to add a panel to the window contains a series of sliders for the equalizer allowing for live editting of the synthesisers sound to ensure it sounds how the Trombonist wishes it to.

9.18.2 Member Function Documentation

9.18.2.1 buttonClicked()

```
void EqualizerPanel::buttonClicked (
            juce::Button * buttonClicked )  [override]
```

A callback function which is called upon automatically when the user has pressed a button within this class.

Parameters

buttonClicked	A pointer to the button which has been clicked.

Note

As this class only has one button, there is no need to check which button has been clicked but it is predefined to pass a parameter in the juce library. This button is used to show or hide the equalizer from the user in the parent window.

9.18.2.2 resized()

```
void EqualizerPanel::resized ( ) [override]
```

A function which changes the size of the child components when the parent window has changed in size.

Note

This function currently adds a slight margin around the edges of the bounds to ensure that the child components do not overlap with any external components.

The documentation for this class was generated from the following files:

- Panels.hpp
- Panels.cpp

9.19 EventHandler Class Reference

Handles acquisition and dispatch of raw sensor input events.

```
#include <eventHandler.hpp>
```

Public Types

- using **EventQueue** = std::queue< RawInputEvent >

  Alias for the internal event queue type.

Public Member Functions

- EventHandler (bool isTest)

  Constructs the EventHandler.
- bool initialise ()

  Initialises all input devices.
- void start ()

  Starts event acquisition.
- RawInputEvent waitForEvent ()

  Blocks until an event is available and returns it.

9.19.1 Detailed Description

Handles acquisition and dispatch of raw sensor input events.

This class interfaces with all hardware input devices (ToF sensor, pressure sensor, capacitive touch sensor) and converts their outputs into a unified event stream. Events are pushed into an internal queue and consumed by other system components.

A thread-safe producer-consumer model is used:

- Sensor callbacks or polling logic push events into a queue.
- Consumers block on waitForEvent() until data is available.

The class also supports a test mode, where hardware interaction is bypassed, allowing controlled input injection without physical devices.

9.19.2 Constructor & Destructor Documentation

9.19.2.1 EventHandler()

```
EventHandler::EventHandler (
            bool isTest )
```

Constructs the EventHandler.

Parameters

isTest	Enables test mode (disables hardware-dependent behaviour).

9.19.3 Member Function Documentation

9.19.3.1 initialise()

```
bool EventHandler::initialise ( )
```

Initialises all input devices.

Returns

True if all devices were successfully initialised, false otherwise.

Must be called before start().

9.19.3.2 start()

```
void EventHandler::start ( )
```

Starts event acquisition.

Begins polling or interrupt-driven handling of input devices and populates the internal event queue.

9.19.3.3 waitForEvent()

```
RawInputEvent EventHandler::waitForEvent ( )
```

Blocks until an event is available and returns it.

Returns

The next RawInputEvent in the queue.

Thread-safe. Intended for use by consumer threads.

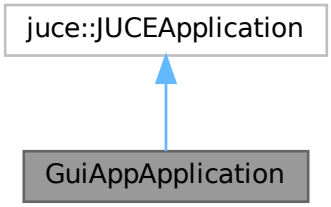
The documentation for this class was generated from the following files:

- eventHandler.hpp
- eventHandler.cpp

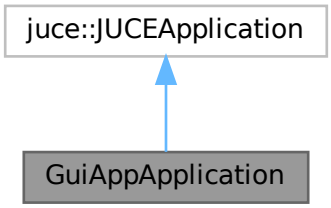
9.20 GuiAppApplication Class Reference

Creation of main application class.

Inheritance diagram for GuiAppApplication:



Collaboration diagram for GuiAppApplication:



Classes

- class MainWindow

  This class implements the desktop window that contains an instance of the MainComponent class.

Public Member Functions

- **GuiAppApplication** ()

  Defining the GuiAppApplication function.
- const juce::String getApplicationName () override

  A function which returns the name of the applicaiton.
- const juce::String getApplicationVersion () override

  A function which gets the current juce version and returns the name of the one being used.
- bool **moreThanOneInstanceAllowed** () override

A function which determines whether multiple instances of the window may be opened simultaneously. This is currently set to be false so that there may on be one window in use at a time.

- void initialise (const juce::String &commandLine) override

  The function which initialises the main window when the code is executed.
- void shutdown () override

  Function used to control what happens when the user wants to close the application.
- void **systemRequestedQuit** () override

  A function used to close the window. This is requested when the user presses the 'X' button.
- void anotherInstanceStarted (const juce::String &commandLine) override

  Handler for when the user tries to open mutliple instances of the application.

9.20.1 Detailed Description

Creation of main application class.

Note

This is called upon when the code is run for the first time.

9.20.2 Member Function Documentation

9.20.2.1 anotherInstanceStarted()

```
void GuiAppApplication::anotherInstanceStarted (
            const juce::String & commandLine )  [inline], [override]
```

Handler for when the user tries to open mutliple instances of the application.

Note

This function will return the command line parameters passed for the new instance were and may be used within the current window.

9.20.2.2 getApplicationName()

```
const juce::String GuiAppApplication::getApplicationName ( )  [inline], [override]
```

A function which returns the name of the applicaiton.

Note

We inject the name as compile definitions from the CMakeLists.txt

9.20.2.3 getApplicationVersion()

```
const juce::String GuiAppApplication::getApplicationVersion ( )  [inline], [override]
```

A function which gets the current juce version and returns the name of the one being used.

Note

We inject the name as compile definitions from the CMakeLists.txt

9.20.2.4 initialise()

```
void GuiAppApplication::initialise (
            const juce::String & commandLine )  [inline], [override]
```

The function which initialises the main window when the code is executed.

Parameters

commandLine	The arguments which the user has input on the terminal when executing the code. This is currently not used.

9.20.2.5 shutdown()

```
void GuiAppApplication::shutdown ( ) [inline], [override]
```

Function used to control what happens when the user wants to close the application.

Note

Currently nothing extra is happening here other than closing the window

The documentation for this class was generated from the following file:

- App/src/main.cpp

9.21 I2CBus Class Reference

Minimal Linux I2C bus wrapper for register-based communication.

```
#include <i2c_bus.hpp>
```

Public Member Functions

- I2CBus (const char ∗devicePath="/dev/i2c-1", bool isTest=false)

  Construct an I2CBus object.
- ∼**I2CBus** ()

  Destructor ensures the I2C device file is closed.
- bool openBus ()

  Open the I2C device file.
- void **closeBus** ()

  Close the I2C device file.
- int writeBlock16 (uint8_t address, uint16_t reg, const uint8_t ∗data, uint8_t length)

  Write a block of data to a 16-bit register address.
- int readBlock16 (uint8_t address, uint16_t reg, uint8_t ∗data, uint8_t length)

  Read a block of data from a 16-bit register address.
- int writeBlock8 (uint8_t address, uint8_t reg, const uint8_t ∗data, uint8_t length)

  Write a block of data to an 8-bit register address.
- int readBlock8 (uint8_t address, uint8_t reg, uint8_t ∗data, uint8_t length)

  Read a block of data from an 8-bit register address.

9.21.1 Detailed Description

Minimal Linux I2C bus wrapper for register-based communication.

This class:

- Opens and closes the I2C device file (e.g. "/dev/i2c-1").
- Provides methods to read/write blocks of data to/from 8-bit or 16-bit registers on I2C devices.

This class does not:

- Handle any sensor-specific logic or protocols.

9.21.2 Constructor & Destructor Documentation

9.21.2.1 I2CBus()

```
I2CBus::I2CBus (
            const char * devicePath = "/dev/i2c-1",
            bool isTest = false )  [explicit]
```

Construct an I2CBus object.

Parameters

devicePath	Path to the Linux I2C device file (e.g. "/dev/i2c-1").

9.21.3 Member Function Documentation

9.21.3.1 openBus()

```
bool I2CBus::openBus ( )
```

Open the I2C device file.

If the bus is already open, this function returns success.

Returns

true if successful, false otherwise.

9.21.3.2 readBlock16()

```
int I2CBus::readBlock16 (
            uint8_t address,
            uint16_t reg,
            uint8_t * data,
            uint8_t length )
```

Read a block of data from a 16-bit register address.

Parameters

address	7-bit I2C device address.
reg	16-bit register address.
data	Pointer to the buffer where read data will be stored.
length	Number of bytes to read.

Returns

0 on success, -1 on failure.

9.21.3.3 readBlock8()

```
int I2CBus::readBlock8 (
            uint8_t address,
            uint8_t reg,
            uint8_t * data,
            uint8_t length )
```

Read a block of data from an 8-bit register address.

Parameters

address	7-bit I2C device address.
reg	8-bit register address.
data	Pointer to the buffer where read data will be stored.
length	Number of bytes to read.

Returns

0 on success, -1 on failure.

9.21.3.4 writeBlock16()

```
int I2CBus::writeBlock16 (
            uint8_t address,
            uint16_t reg,
            const uint8_t * data,
            uint8_t length )
```

Write a block of data to a 16-bit register address.

Parameters

address	7-bit I2C device address.
reg	16-bit register address.
data	Pointer to the data buffer.
length	Number of bytes to write.

Returns

0 on success, -1 on failure.

9.21.3.5 writeBlock8()

```
int I2CBus::writeBlock8 (
            uint8_t address,
            uint8_t reg,
            const uint8_t * data,
            uint8_t length )
```

Write a block of data to an 8-bit register address.

Parameters

address	7-bit I2C device address.
reg	8-bit register address.
data	Pointer to the data buffer.
length	Number of bytes to write.

Returns

0 on success, -1 on failure.

The documentation for this class was generated from the following files:

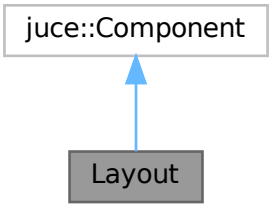
- i2c_bus.hpp
- i2c_bus.cpp

9.22 Layout Class Reference

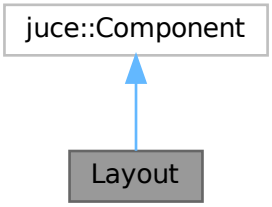
Main class that creates the window.

```
#include <Layout.hpp>
```

Inheritance diagram for Layout:



Collaboration diagram for Layout:



Public Member Functions

- **Layout** (CoreWrapper &coreWrapper)
- void paint (juce::Graphics &g) override

  Function that sets the background colour of the window.
- void resized () override

  Function that changes the size of the profiler.

9.22.1 Detailed Description

Main class that creates the window.

9.22.2 Member Function Documentation

9.22.2.1 paint()

```
void Layout::paint (
            juce::Graphics & g )  [override]
```

Function that sets the background colour of the window.

Parameters

g	The panel which should be modified.

Note

Always sets the background to the user preference for the background. This function also adds a title to be within the window.

9.22.2.2 resized()

```
void Layout::resized ( )  [override]
```

Function that changes the size of the profiler.

Note

Resizes the contents of the window including dropdown menus, labels and buttons

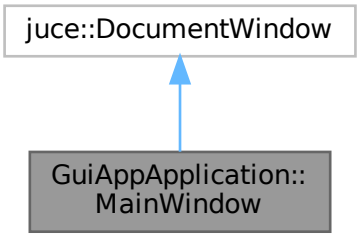
The documentation for this class was generated from the following files:

- Layout.hpp
- Layout.cpp

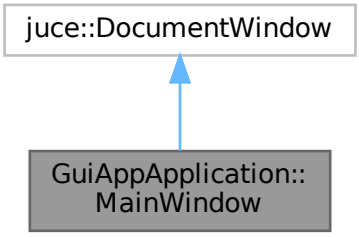
9.23 GuiAppApplication::MainWindow Class Reference

This class implements the desktop window that contains an instance of the MainComponent class.

Inheritance diagram for GuiAppApplication::MainWindow:



Collaboration diagram for GuiAppApplication::MainWindow:



Public Member Functions

- MainWindow (CoreWrapper &coreWrapper, juce::String name)
- void closeButtonPressed () override

  function handler for when the user tries to close the program

9.23.1 Detailed Description

This class implements the desktop window that contains an instance of the MainComponent class.

Note

Inherits the juce::DocumentWindow class and functions.

9.23.2 Constructor & Destructor Documentation

9.23.2.1 MainWindow()

```
GuiAppApplication::MainWindow::MainWindow (
            CoreWrapper & coreWrapper,
            juce::String name )  [inline], [explicit]
```

Generation of what the main window will look like.

9.23.3 Member Function Documentation

9.23.3.1 closeButtonPressed()

```
void GuiAppApplication::MainWindow::closeButtonPressed ( )  [inline], [override]
```

function handler for when the user tries to close the program

Note

Currently does not do anything other than request the system to close the application.

The documentation for this class was generated from the following file:

- App/src/main.cpp

9.24 MidiCoordinator Class Reference

Class to coordinate Midi message construction based on output from sensor mappers. This class is stateful and responsible for translating high-level sensor data into Midi messages. Utilises MidiMessageBuilder for external synths and directly controls internal synth.

```
#include <MidiCoordinator.hpp>
```

Public Member Functions

- **MidiCoordinator** (AudioRender &synth, bool isTest)

  Constructor takes reference to internal synth and optionally triggers tests.
- void **RegisterCallback** (CallbackInterface cb)
- void PressureEdge (bool on)

  Method to update gate on/off.
- void ChangeNote (int note)

  Method to update note from mouthpiece.
- void setBend (int bend)

  Method to update pitch bend from slide.
- void setExpr (int expr)
- void setState (State newstate)

  Method to set state of instrument.
- void setDevice (bool external_device_present)

  Set state of whether to use internal or external synth.
- AudioRender & **getSynth** (void)

9.24.1 Detailed Description

Class to coordinate Midi message construction based on output from sensor mappers. This class is stateful and responsible for translating high-level sensor data into Midi messages. Utilises MidiMessageBuilder for external synths and directly controls internal synth.

9.24.2 Member Function Documentation

9.24.2.1 ChangeNote()

```
void MidiCoordinator::ChangeNote (
            int note )
```

Method to update note from mouthpiece.

Parameters

note	MIDI standard 0-127 notes.

9.24.2.2 PressureEdge()

```
void MidiCoordinator::PressureEdge (
            bool on )
```

Method to update gate on/off.

Parameters

on	true = 'Note on' message, false = 'Note off'

9.24.2.3 setBend()

```
void MidiCoordinator::setBend (
            int bend )
```

Method to update pitch bend from slide.

Parameters

bend	Values from 8192 to 0 (so only downward bends possible)

9.24.2.4 setDevice()

```
void MidiCoordinator::setDevice (
            bool external_device_present )
```

Set state of whether to use internal or external synth.

Parameters

external_device_present	true = Use soundcard/external synth, false = use internal synth

9.24.2.5 setExpr()

```
void MidiCoordinator::setExpr (
            int expr )
```

Parameters

Method	to update MIDI expression.
expr	Expression value, 0-127.

9.24.2.6 setState()

```
void MidiCoordinator::setState (
            State newstate )
```

Method to set state of instrument.

Parameters

newstate	Set state to either IDLE, PLAYING or ERROR.

The documentation for this class was generated from the following files:

- MidiCoordinator.hpp
- MidiCoordinator.cpp

9.25 MidiMessage Class Reference

Represents a raw MIDI message as a sequence of bytes.

```
#include <MidiMessage.hpp>
```

Public Types

- using **Byte** = unsigned char

Public Member Functions

- MidiMessage ()=default

  Default constructor.
- MidiMessage (std::vector< Byte > bytes)

  Construct a MIDI message from an existing byte vector.
- MidiMessage (std::initializer_list< Byte > bytes)

  Construct a MIDI message from a list of bytes.
- const std::vector< Byte > & bytes () const noexcept

  Access the raw byte buffer.
- std::size_t size () const noexcept

  Get number of bytes in the message.
- bool empty () const noexcept

  Check whether the message contains any bytes.

9.25.1 Detailed Description

Represents a raw MIDI message as a sequence of bytes.

This class is a lightweight value type used to store MIDI messages before transmission. It does not perform validation beyond basic structural construction.

A MIDI message consists of:

- A status byte
- Zero, one, or two data bytes (for standard channel messages)
- Or an arbitrary number of bytes (e.g. SysEx)

The underlying storage uses std::vector<unsigned char> to allow variable-length messages for future extensibility.

9.25.2 Constructor & Destructor Documentation

9.25.2.1 MidiMessage() [1/3]

```
MidiMessage::MidiMessage ( ) [default]
```

Default constructor.

Creates an empty MIDI message.

9.25.2.2 MidiMessage() [2/3]

```
MidiMessage::MidiMessage (
            std::vector< Byte > bytes ) [explicit]
```

Construct a MIDI message from an existing byte vector.

Parameters

bytes	Vector containing raw MIDI bytes.

9.25.2.3 MidiMessage() [3/3]

```
MidiMessage::MidiMessage (
            std::initializer_list< Byte > bytes )
```

Construct a MIDI message from a list of bytes.

Example:
```
MidiMessage msg{0x90, 0x3C, 0x64};
```

Parameters

bytes	Initialiser list of raw MIDI bytes.

9.25.3 Member Function Documentation

9.25.3.1 bytes()

```
const std::vector< MidiMessage::Byte > & MidiMessage::bytes ( ) const  [noexcept]
```

Access the raw byte buffer.

Returns

Const reference to internal byte vector.

9.25.3.2 empty()

```
bool MidiMessage::empty ( ) const  [noexcept]
```

Check whether the message contains any bytes.

Returns

True if empty, false otherwise.

9.25.3.3 size()

```
std::size_t MidiMessage::size ( ) const  [noexcept]
```

Get number of bytes in the message.

Returns

Size of the message in bytes.

The documentation for this class was generated from the following files:

- MidiMessage.hpp
- MidiMessage.cpp

9.26 MidiMessageBuilder Class Reference

Factory class for constructing common MIDI channel messages.

```
#include <MidiMessageBuilder.hpp>
```

Public Types

- using **Byte** = MidiMessage::Byte

Public Member Functions

- MidiMessage noteOn (int channel, Byte note, Byte velocity) const

  Create a MIDI Note On message.
- MidiMessage noteOff (int channel, Byte note, Byte velocity=0) const

  Create a MIDI Note Off message.
- MidiMessage pitchBend (int channel, int value) const

  Pitch bend message constructor.
- MidiMessage **expr** (int channel, int value) const

9.26.1 Detailed Description

Factory class for constructing common MIDI channel messages.

This class encapsulates the encoding logic required to construct correctly formatted MIDI channel voice messages.

It separates message construction from message transmission, supporting a clean single-responsibility design.

Channels are specified using human-friendly numbering (1–16) and internally converted to zero-based encoding (0–15).

9.26.2 Member Function Documentation

9.26.2.1 noteOff()

```
MidiMessage MidiMessageBuilder::noteOff (
            int channel,
            Byte note,
            Byte velocity = 0 ) const
```

Create a MIDI Note Off message.

This implementation uses the dedicated Note Off status byte (0x8n) rather than Note On with velocity 0.

Parameters

channel	MIDI channel number (1–16).
note	MIDI note number (0–127).
velocity	Release velocity (default 0).

Returns

Constructed MidiMessage.

9.26.2.2 noteOn()

```
MidiMessage MidiMessageBuilder::noteOn (
            int channel,
            Byte note,
            Byte velocity ) const
```

Create a MIDI Note On message.

Parameters

channel	MIDI channel number (1–16).
note	MIDI note number (0–127).
velocity	Note velocity (0–127).

Returns

Constructed MidiMessage.

9.26.2.3 pitchBend()

```
MidiMessage MidiMessageBuilder::pitchBend (
            int channel,
            int value ) const
```

Pitch bend message constructor.

Parameters

channel	MIDI channel number (1–16).
value	Pitch bend value (0–8192).

Returns

Constructed MidiMessage.

The documentation for this class was generated from the following files:

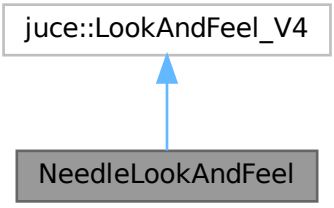
- MidiMessageBuilder.hpp
- MidiMessageBuilder.cpp

9.27 NeedleLookAndFeel Class Reference

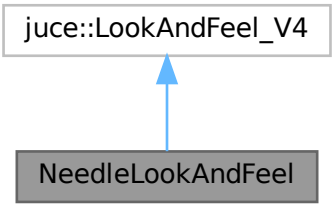
A class which may be attached to a rotary slider from the juce framework to change its look and feel. This changes the look of the slider to have an arrow rotate about the centre of the slider pointing to the different values rather than a knob moving in an arc as the basic rotary slider.

```
#include <CustomStyles.hpp>
```

Inheritance diagram for NeedleLookAndFeel:



Collaboration diagram for NeedleLookAndFeel:



Public Member Functions

- **NeedleLookAndFeel** ()=default

  The constructor function for the `NeedleLookAndFeel` class which is set to default.
- virtual ~**NeedleLookAndFeel** ()=default

  The destructor function for the `NeedleLookAndFeel` class which is set to default.
- virtual void drawRotarySlider (juce::Graphics &g, int x, int y, int width, int height, float sliderPos, const float rotaryStartAngle, const float rotaryEndAngle, juce::Slider &slider) override

  A function which is used to redraw the rotary slider whose look and feel is being changed. This will full describe how it should appear to the user.
- void setNeedleColour (juce::Colour colour)

  A function to set the colour of the needle which represents the position of the slider.
- juce::Colour getNeedleColour (void)

  A function to get the current colour of the needle which represents the position of the slider.

9.27.1 Detailed Description

A class which may be attached to a rotary slider from the juce framework to change its look and feel. This changes the look of the slider to have an arrow rotate about the centre of the slider pointing to the different values rather than a knob moving in an arc as the basic rotary slider.

9.27.2 Member Function Documentation

9.27.2.1 drawRotarySlider()

```
void NeedleLookAndFeel::drawRotarySlider (
            juce::Graphics & g,
            int x,
            int y,
            int width,
            int height,
            float sliderPos,
            const float rotaryStartAngle,
            const float rotaryEndAngle,
            juce::Slider & slider )  [override], [virtual]
```

A function which is used to redraw the rotary slider whose look and feel is being changed. This will full describe how it should appear to the user.

Parameters

g	A pointer to the `juce::Graphics` instance which is being updated. This is the parent component or the main window.
x	The x position of the slider.
y	The y position of the slider.
width	The width of the slider.
height	The maximum height of the slider.
sliderPos	The position of the slider finger.
rotaryStartAngle	The minimum angle which the slider starts at. This represents the minimum value, typically bottom left.
rotaryEndAngle	The maximum angle which the slider ends at. This represents the maximum value, typically bottom right.
slider	This is a pointer to the slider which is being drawn. Extra data may be extracted from it if necessary.

Note

This function is a call back which is automatically called to redraw the slider to look how described. It should not need to be called by any high level function.

9.27.2.2 getNeedleColour()

```
juce::Colour NeedleLookAndFeel::getNeedleColour (
            void  )
```

A function to get the current colour of the needle which represents the position of the slider.

Return values

The	colour of the needle as a juce::Colour type.

9.27.2.3 setNeedleColour()

```
void NneedleLookAndFeel::setNeedleColour (
            juce::Colour colour )
```

A function to set the colour of the needle which represents the position of the slider.

Parameters

colour	The colour which the needle should be set to.

The documentation for this class was generated from the following files:

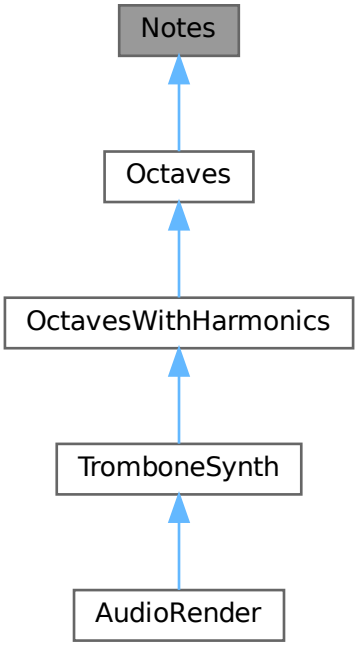
- CustomStyles.hpp
- CustomStyles.cpp

9.28 Notes Class Reference

A class to store the frequency of the notes which may be referenced. The notes are stored in an enum `Notes_t` which allow for the higher level functions to call upon the specific note as opposed to frequency.

```
#include <synth.hpp>
```

Inheritance diagram for Notes:



Public Types

- enum Notes_t {
  note_C , **note_Db** , **note_D** , **note_Eb** ,
  note_E , **note_F** , **note_Gb** , **note_G** ,
  note_Ab , **note_A** , **note_Bb** , **note_B** ,
  note_C_sharp = note_Db , **note_D_sharp** = note_Eb , **note_F_sharp** = note_Gb , **note_G_sharp** = note↩_Ab ,
  note_A_sharp = note_Bb }

  Creation of the enumerated list of possible notes.

Public Member Functions

- Notes (float C_in=16.35, float Db_in=17.32, float D_in=18.35, float Eb_in=19.45, float E_in=20.60, float F↩_in=21.83, float Gb_in=23.12, float G_in=24.50, float Ab_in=25.96, float A_in=27.50, float Bb_in=29.14, float B_in=30.87)

  Constructor for the `Notes` class to create a new set of notes and assign specified frequencies to each. Each frequency is set to `Octave 0` by default.
- void setC (float frequency)

  A function that can change the value of the frequency that the C note plays.
- void setC_sharp (float frequency)

  A function that can change the value of the frequency that the C# note plays.
- void setDb (float frequency)

  A function that can change the value of the frequency that the Db note plays.

- void setD (float frequency)

  A function that can change the value of the frequency that the D note plays.
- void setD_sharp (float frequency)

  A function that can change the value of the frequency that the D# note plays.
- void setEb (float frequency)

  A function that can change the value of the frequency that the Eb note plays.
- void setE (float frequency)

  A function that can change the value of the frequency that the E note plays.
- void setF (float frequency)

  A function that can change the value of the frequency that the F note plays.
- void setF_sharp (float frequency)

  A function that can change the value of the frequency that the F# note plays.
- void setGb (float frequency)

  A function that can change the value of the frequency that the Gb note plays.
- void setG (float frequency)

  A function that can change the value of the frequency that the G note plays.
- void setG_sharp (float frequency)

  A function that can change the value of the frequency that the G# note plays.
- void setAb (float frequency)

  A function that can change the value of the frequency that the Ab note plays.
- void setA (float frequency)

  A function that can change the value of the frequency that the A note plays.
- void setA_sharp (float frequency)

  A function that can change the value of the frequency that the A# note plays.
- void setBb (float frequency)

  A function that can change the value of the frequency that the Bb note plays.
- void setB (float frequency)

  A function that can change the value of the frequency that the B note plays.
- float setNote (Notes_t note, float freq)

  An alternative function which can allow for the user to alter the frequency of a specific note based upon the Notes_t enumerated values rather than specifying a function call.
- float getC (void)

  A function that returns the frequency of the C note for the specified object.
- float getC_sharp (void)

  A function that returns the frequency of the C# note for the specified object.
- float getDb (void)

  A function that returns the frequency of the Db note for the specified object.
- float getD (void)

  A function that returns the frequency of the D note for the specified object.
- float getD_sharp (void)

  A function that returns the frequency of the D# note for the specified object.
- float getEb (void)

  A function that returns the frequency of the Eb note for the specified object.
- float getE (void)

  A function that returns the frequency of the E note for the specified object.
- float getF (void)

  A function that returns the frequency of the F note for the specified object.
- float getF_sharp (void)

  A function that returns the frequency of the F# note for the specified object.
- float getGb (void)

  A function that returns the frequency of the Gb note for the specified object.

- float getG (void)

  A function that returns the frequency of the G note for the specified object.
- float getG_sharp (void)

  A function that returns the frequency of the G# note for the specified object.
- float getAb (void)

  A function that returns the frequency of the Ab note for the specified object.
- float getA (void)

  A function that returns the frequency of the A note for the specified object.
- float getA_sharp (void)

  A function that returns the frequency of the A# note for the specified object.
- float getBb (void)

  A function that returns the frequency of the Bb note for the specified object.
- float getB (void)

  A function that returns the frequency of the B note for the specified object.
- float getNote (Notes_t note)

  An alternative function which can allow for the user to request a specific note based upon the Notes_t enumerated values rather than specifying a function call.

9.28.1 Detailed Description

A class to store the frequency of the notes which may be referenced. The notes are stored in an enum `Notes_t` which allow for the higher level functions to call upon the specific note as opposed to frequency.

9.28.2 Constructor & Destructor Documentation

9.28.2.1 Notes()

```
Notes::Notes (
            float C_in = 16.35,
            float Db_in = 17.32,
            float D_in = 18.35,
            float Eb_in = 19.45,
            float E_in = 20.60,
            float F_in = 21.83,
            float Gb_in = 23.12,
            float G_in = 24.50,
            float Ab_in = 25.96,
            float A_in = 27.50,
            float Bb_in = 29.14,
            float B_in = 30.87 )
```

Constructor for the `Notes` class to create a new set of notes and assign specified frequencies to each. Each frequency is set to `Octave 0` by default.

Parameters

C_in	The frequency of the C key.
Db↩ _in	The frequency of the Db key.
D_in	The frequency of the D key.

Parameters

Eb↩ _in	The frequency of the Eb key.
E_in	The frequency of the E key.
F_in	The frequency of the F key.
Gb↩ _in	The frequency of the Gb key.
G_in	The frequency of the G key.
Ab↩ _in	The frequency of the Ab key.
A_in	The frequency of the A key.
Bb↩ _in	The frequency of the Bb key.
B_in	The frequency of the B key.

Note

This function simply sets the local frequenies to be equal to the passed in frequencies.

9.28.3 Member Function Documentation

9.28.3.1 getA()

```
float Notes::getA (
            void  )
```

A function that returns the frequency of the A note for the specified object.

Return values

The	frequency of the A note as a floating point value.

9.28.3.2 getA_sharp()

```
float Notes::getA_sharp (
            void  )
```

A function that returns the frequency of the A# note for the specified object.

Return values

The	frequency of the A# note as a floating point value.

9.28.3.3 getAb()

```
float Notes::getAb (
```

```
            void  )
```

A function that returns the frequency of the Ab note for the specified object.

Return values

The	frequency of the Ab note as a floating point value.

9.28.3.4 getB()

```
float Notes::getB (
            void  )
```

A function that returns the frequency of the B note for the specified object.

Return values

The	frequency of the C note as a floating point value.

9.28.3.5 getBb()

```
float Notes::getBb (
            void  )
```

A function that returns the frequency of the Bb note for the specified object.

Return values

The	frequency of the Bb note as a floating point value.

9.28.3.6 getC()

```
float Notes::getC (
            void  )
```

A function that returns the frequency of the C note for the specified object.

Return values

The	frequency of the C note as a floating point value.

9.28.3.7 getC_sharp()

```
float Notes::getC_sharp (
            void  )
```

A function that returns the frequency of the C# note for the specified object.

Return values

The	frequency of the C# note as a floating point value.

9.28.3.8 getD()

```
float Notes::getD (
            void  )
```

A function that returns the frequency of the D note for the specified object.

Return values

The	frequency of the D note as a floating point value.

9.28.3.9 getD_sharp()

```
float Notes::getD_sharp (
            void  )
```

A function that returns the frequency of the D# note for the specified object.

Return values

The	frequency of the D# note as a floating point value.

9.28.3.10 getDb()

```
float Notes::getDb (
            void  )
```

A function that returns the frequency of the Db note for the specified object.

Return values

The	frequency of the Db note as a floating point value.

9.28.3.11 getE()

```
float Notes::getE (
            void  )
```

A function that returns the frequency of the E note for the specified object.

Return values

The	frequency of the E note as a floating point value.

9.28.3.12 getEb()

```
float Notes::getEb (
            void  )
```

A function that returns the frequency of the Eb note for the specified object.

Return values

The	frequency of the Eb note as a floating point value.

9.28.3.13 getF()

```
float Notes::getF (
            void  )
```

A function that returns the frequency of the F note for the specified object.

Return values

The	frequency of the F note as a floating point value.

9.28.3.14 getF_sharp()

```
float Notes::getF_sharp (
            void  )
```

A function that returns the frequency of the F# note for the specified object.

Return values

The	frequency of the F# note as a floating point value.

9.28.3.15 getG()

```
float Notes::getG (
            void  )
```

A function that returns the frequency of the G note for the specified object.

Return values

The	frequency of the G note as a floating point value.

9.28.3.16 getG_sharp()

```
float Notes::getG_sharp (
            void  )
```

A function that returns the frequency of the G# note for the specified object.

Return values

The	frequency of the G# note as a floating point value.

9.28.3.17 getGb()

```
float Notes::getGb (
            void  )
```

A function that returns the frequency of the Gb note for the specified object.

Return values

The	frequency of the Gb note as a floating point value.

9.28.3.18 getNote()

```
float Notes::getNote (
            Notes_t note )
```

An alternative function which can allow for the user to request a specific note based upon the Notes_t enumerated values rather than specifying a function call.

Parameters

note	The note of which the user wants to receive the frequency of.

Return values

Returns	a floating point value representing the frequency in Hz of the note specified.

9.28.3.19 setA()

```
void Notes::setA (
```

```
            float frequency )
```

A function that can change the value of the frequency that the A note plays.

Parameters

frequency	The frequency which the A not should play.

Note

There is no return value from this function.

9.28.3.20 setA_sharp()

```
void Notes::setA_sharp (
            float frequency )
```

A function that can change the value of the frequency that the A# note plays.

Parameters

frequency	The frequency which the A# not should play.

Note

There is no return value from this function.

9.28.3.21 setAb()

```
void Notes::setAb (
            float frequency )
```

A function that can change the value of the frequency that the Ab note plays.

Parameters

frequency	The frequency which the Ab not should play.

Note

There is no return value from this function.

9.28.3.22 setB()

```
void Notes::setB (
            float frequency )
```

A function that can change the value of the frequency that the B note plays.

Parameters

frequency	The frequency which the B not should play.

Note

There is no return value from this function.

9.28.3.23 setBb()

```
void Notes::setBb (
            float frequency )
```

A function that can change the value of the frequency that the Bb note plays.

Parameters

frequency	The frequency which the Bb not should play.

Note

There is no return value from this function.

9.28.3.24 setC()

```
void Notes::setC (
            float frequency )
```

A function that can change the value of the frequency that the C note plays.

Parameters

frequency	The frequency which the C not should play.

Note

There is no return value from this function.

9.28.3.25 setC_sharp()

```
void Notes::setC_sharp (
            float frequency )
```

A function that can change the value of the frequency that the C# note plays.

Parameters

frequency	The frequency which the C# not should play.

Note

There is no return value from this function.

9.28.3.26 setD()

```
void Notes::setD (
            float frequency )
```

A function that can change the value of the frequency that the D note plays.

Parameters

frequency	The frequency which the D not should play.

Note

There is no return value from this function.

9.28.3.27 setD_sharp()

```
void Notes::setD_sharp (
            float frequency )
```

A function that can change the value of the frequency that the D# note plays.

Parameters

frequency	The frequency which the D# not should play.

Note

There is no return value from this function.

9.28.3.28 setDb()

```
void Notes::setDb (
            float frequency )
```

A function that can change the value of the frequency that the Db note plays.

Parameters

frequency	The frequency which the Db not should play.

Note

There is no return value from this function.

9.28.3.29 setE()

```
void Notes::setE (
            float frequency )
```

A function that can change the value of the frequency that the E note plays.

Parameters

frequency	The frequency which the E not should play.

Note

There is no return value from this function.

9.28.3.30 setEb()

```
void Notes::setEb (
            float frequency )
```

A function that can change the value of the frequency that the Eb note plays.

Parameters

frequency	The frequency which the Eb not should play.

Note

There is no return value from this function.

9.28.3.31 setF()

```
void Notes::setF (
            float frequency )
```

A function that can change the value of the frequency that the F note plays.

Parameters

frequency	The frequency which the F not should play.

Note

There is no return value from this function.

9.28.3.32 setF_sharp()

```
void Notes::setF_sharp (
            float frequency )
```

A function that can change the value of the frequency that the F# note plays.

Parameters

frequency	The frequency which the F# not should play.

Note

There is no return value from this function.

9.28.3.33 setG()

```
void Notes::setG (
            float frequency )
```

A function that can change the value of the frequency that the G note plays.

Parameters

frequency	The frequency which the G not should play.

Note

There is no return value from this function.

9.28.3.34 setG_sharp()

```
void Notes::setG_sharp (
            float frequency )
```

A function that can change the value of the frequency that the G# note plays.

Parameters

frequency	The frequency which the G# not should play.

Note

There is no return value from this function.

9.28.3.35 setGb()

```
void Notes::setGb (
            float frequency )
```

A function that can change the value of the frequency that the Gb note plays.

Parameters

frequency	The frequency which the Gb not should play.

Note

There is no return value from this function.

9.28.3.36 setNote()

```
float Notes::setNote (
            Notes_t note,
            float freq )
```

An alternative function which can allow for the user to alter the frequency of a specific note based upon the Notes_t enumerated values rather than specifying a function call.

Parameters

note	The note of which the user wants to change the frequency of.
freq	The frequency which the specified note should be changed to.

The documentation for this class was generated from the following files:

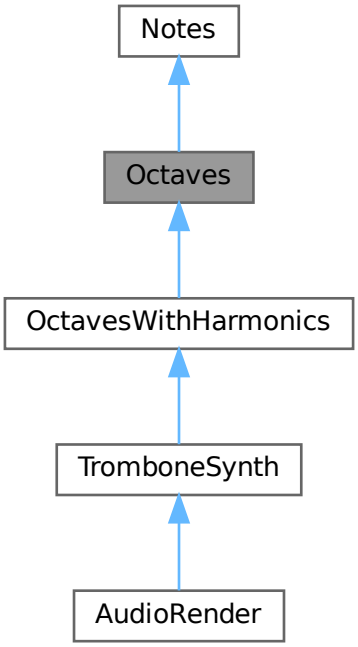
- synth.hpp
- synth.cpp

9.29 Octaves Class Reference

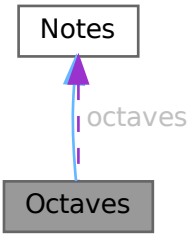
Creation of a class to store the frequencies of notes at different octaves. Inherits from `Notes` class.

```
#include <synth.hpp>
```

Inheritance diagram for Octaves:



Collaboration diagram for Octaves:



Public Member Functions

- float PlayingNote (int octave, Notes_t note, float phase)

  A function which may be called to determine the amplitude of a specified note at a specified phase.
- float PlayingFrequency (float phase)

  A function which may be called to determine the amplitude of a specified frequency at a specified time.

Public Member Functions inherited from Notes

- Notes (float C_in=16.35, float Db_in=17.32, float D_in=18.35, float Eb_in=19.45, float E_in=20.60, float F↩ _in=21.83, float Gb_in=23.12, float G_in=24.50, float Ab_in=25.96, float A_in=27.50, float Bb_in=29.14, float B_in=30.87)

  Constructor for the `Notes` class to create a new set of notes and assign specified frequencies to each. Each frequency is set to `Octave 0` by default.
- void setC (float frequency)

  A function that can change the value of the frequency that the C note plays.
- void setC_sharp (float frequency)

  A function that can change the value of the frequency that the C# note plays.
- void setDb (float frequency)

  A function that can change the value of the frequency that the Db note plays.
- void setD (float frequency)

  A function that can change the value of the frequency that the D note plays.
- void setD_sharp (float frequency)

  A function that can change the value of the frequency that the D# note plays.
- void setEb (float frequency)

  A function that can change the value of the frequency that the Eb note plays.
- void setE (float frequency)

  A function that can change the value of the frequency that the E note plays.
- void setF (float frequency)

  A function that can change the value of the frequency that the F note plays.
- void setF_sharp (float frequency)

  A function that can change the value of the frequency that the F# note plays.
- void setGb (float frequency)

  A function that can change the value of the frequency that the Gb note plays.
- void setG (float frequency)

  A function that can change the value of the frequency that the G note plays.
- void setG_sharp (float frequency)

  A function that can change the value of the frequency that the G# note plays.
- void setAb (float frequency)

  A function that can change the value of the frequency that the Ab note plays.
- void setA (float frequency)

  A function that can change the value of the frequency that the A note plays.
- void setA_sharp (float frequency)

  A function that can change the value of the frequency that the A# note plays.
- void setBb (float frequency)

  A function that can change the value of the frequency that the Bb note plays.
- void setB (float frequency)

  A function that can change the value of the frequency that the B note plays.
- float setNote (Notes_t note, float freq)

  An alternative function which can allow for the user to alter the frequency of a specific note based upon the Notes_t enumerated values rather than specifying a function call.
- float getC (void)

  A function that returns the frequency of the C note for the specified object.
- float getC_sharp (void)

  A function that returns the frequency of the C# note for the specified object.
- float getDb (void)

  A function that returns the frequency of the Db note for the specified object.
- float getD (void)

  A function that returns the frequency of the D note for the specified object.

- float getD_sharp (void)

  A function that returns the frequency of the D# note for the specified object.
- float getEb (void)

  A function that returns the frequency of the Eb note for the specified object.
- float getE (void)

  A function that returns the frequency of the E note for the specified object.
- float getF (void)

  A function that returns the frequency of the F note for the specified object.
- float getF_sharp (void)

  A function that returns the frequency of the F# note for the specified object.
- float getGb (void)

  A function that returns the frequency of the Gb note for the specified object.
- float getG (void)

  A function that returns the frequency of the G note for the specified object.
- float getG_sharp (void)

  A function that returns the frequency of the G# note for the specified object.
- float getAb (void)

  A function that returns the frequency of the Ab note for the specified object.
- float getA (void)

  A function that returns the frequency of the A note for the specified object.
- float getA_sharp (void)

  A function that returns the frequency of the A# note for the specified object.
- float getBb (void)

  A function that returns the frequency of the Bb note for the specified object.
- float getB (void)

  A function that returns the frequency of the B note for the specified object.
- float getNote (Notes_t note)

  An alternative function which can allow for the user to request a specific note based upon the Notes_t enumerated values rather than specifying a function call.

Public Attributes

- Notes **octaves** [nOctaves]

  Initialising an array of the octaves and their notes.

Static Public Attributes

- static const char **nOctaves** = 9

  Stores the number of octaves being used.

Additional Inherited Members

Public Types inherited from Notes

- enum Notes_t {
  note_C , **note_Db** , **note_D** , **note_Eb** ,
  note_E , **note_F** , **note_Gb** , **note_G** ,
  note_Ab , **note_A** , **note_Bb** , **note_B** ,
  note_C_sharp = note_Db , **note_D_sharp** = note_Eb , **note_F_sharp** = note_Gb , **note_G_sharp** = note↩
  _Ab ,
  note_A_sharp = note_Bb }

  Creation of the enumerated list of possible notes.

9.29.1 Detailed Description

Creation of a class to store the frequencies of notes at different octaves. Inherits from `Notes` class.

9.29.2 Member Function Documentation

9.29.2.1 PlayingFrequency()

```
float Octaves::PlayingFrequency (
            float phase )
```

A function which may be called to determine the amplitude of a specified frequency at a specified time.

Parameters

phase	The phase of the frequency at the time of sampling.

Return values

Returns	a floating point value representing the ampliude of the note at the time it was sampled.

Note

This function only uses unit amplitudes and does not change depending on the volume. The maxmimum and minimum will always be 1 and -1 respectively.

9.29.2.2 PlayingNote()

```
float Octaves::PlayingNote (
            int octave,
            Notes_t note,
            float phase )
```

A function which may be called to determine the amplitude of a specified note at a specified phase.

Parameters

octave	The octave in which the note being played is in.
note	The note which is currently being played.
phase	The phase of the wave at the time of sampling.

Return values

Returns	a floating point value representing the ampliude of the note at the time it was sampled.

Note

This function only uses unit amplitudes and does not change depending on the volume. The maxmimum and minimum will always be 1 and -1 respectively.

The documentation for this class was generated from the following files:

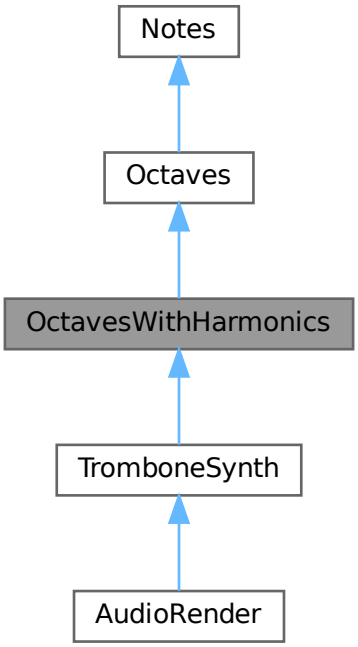
- synth.hpp
- synth.cpp

9.30 OctavesWithHarmonics Class Reference

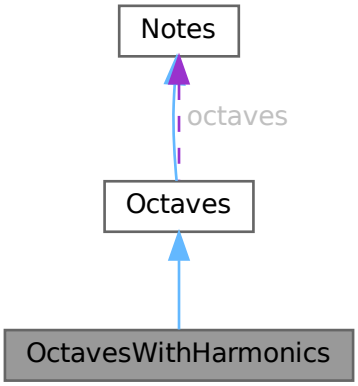
A class which inherits functions and variables from the `Octaves` class allowing for the notes being output to have a set of harmonic frequencies attached to them too.

```
#include <synth.hpp>
```

Inheritance diagram for OctavesWithHarmonics:



Collaboration diagram for OctavesWithHarmonics:



Public Member Functions

- **OctavesWithHarmonics** ()

  Creates an instance of the `OctavesWithHarmonics`.
- float PlayingNoteWithHarmonics (int n, int octave, Notes_t note, float phase)

  A function which may be called upon to stop playing a range of harmonic notes when a specific note is called upon to be played.
- float PlayingFrequencyWithHarmonics (int n, float phase)

  A function which may be called upon to stop playing a range of harmonic frequencies when a specific note is called upon to be played.
- float HarmonicDecay (int n)

  A function which calculates how much the amplitude of the harmonic should decay in comparison to the first harmonic.
- float getHarmonicDecayMax (int n)

  A function which returns the maximum amplitude possible for a set number of harmonics if they are all constructively interfering.
- void setDecayConstant (float decayC)

  A function that can change the value of the decay constant used to set the maxmimum amplitude of each successive harmonic.
- float getDecayConstant (void)

  A function which returns the decay constant being used to determine the relative ampltiude of each successive harmonic.

Public Member Functions inherited from Octaves

- float PlayingNote (int octave, Notes_t note, float phase)

  A function which may be called to determine the amplitude of a specified note at a specified phase.
- float PlayingFrequency (float phase)

  A function which may be called to determine the amplitude of a specified frequency at a specified time.

Public Member Functions inherited from Notes

- Notes (float C_in=16.35, float Db_in=17.32, float D_in=18.35, float Eb_in=19.45, float E_in=20.60, float F↩ _in=21.83, float Gb_in=23.12, float G_in=24.50, float Ab_in=25.96, float A_in=27.50, float Bb_in=29.14, float B_in=30.87)

  Constructor for the `Notes` class to create a new set of notes and assign specified frequencies to each. Each frequency is set to `Octave 0` by default.
- void setC (float frequency)

  A function that can change the value of the frequency that the C note plays.
- void setC_sharp (float frequency)

  A function that can change the value of the frequency that the C# note plays.
- void setDb (float frequency)

  A function that can change the value of the frequency that the Db note plays.
- void setD (float frequency)

  A function that can change the value of the frequency that the D note plays.
- void setD_sharp (float frequency)

  A function that can change the value of the frequency that the D# note plays.
- void setEb (float frequency)

  A function that can change the value of the frequency that the Eb note plays.
- void setE (float frequency)

  A function that can change the value of the frequency that the E note plays.
- void setF (float frequency)

  A function that can change the value of the frequency that the F note plays.
- void setF_sharp (float frequency)

  A function that can change the value of the frequency that the F# note plays.
- void setGb (float frequency)

  A function that can change the value of the frequency that the Gb note plays.
- void setG (float frequency)

  A function that can change the value of the frequency that the G note plays.
- void setG_sharp (float frequency)

  A function that can change the value of the frequency that the G# note plays.
- void setAb (float frequency)

  A function that can change the value of the frequency that the Ab note plays.
- void setA (float frequency)

  A function that can change the value of the frequency that the A note plays.
- void setA_sharp (float frequency)

  A function that can change the value of the frequency that the A# note plays.
- void setBb (float frequency)

  A function that can change the value of the frequency that the Bb note plays.
- void setB (float frequency)

  A function that can change the value of the frequency that the B note plays.
- float setNote (Notes_t note, float freq)

  An alternative function which can allow for the user to alter the frequency of a specific note based upon the Notes_t enumerated values rather than specifying a function call.
- float getC (void)

  A function that returns the frequency of the C note for the specified object.
- float getC_sharp (void)

  A function that returns the frequency of the C# note for the specified object.
- float getDb (void)

  A function that returns the frequency of the Db note for the specified object.
- float getD (void)

  A function that returns the frequency of the D note for the specified object.

- float getD_sharp (void)

  A function that returns the frequency of the D# note for the specified object.
- float getEb (void)

  A function that returns the frequency of the Eb note for the specified object.
- float getE (void)

  A function that returns the frequency of the E note for the specified object.
- float getF (void)

  A function that returns the frequency of the F note for the specified object.
- float getF_sharp (void)

  A function that returns the frequency of the F# note for the specified object.
- float getGb (void)

  A function that returns the frequency of the Gb note for the specified object.
- float getG (void)

  A function that returns the frequency of the G note for the specified object.
- float getG_sharp (void)

  A function that returns the frequency of the G# note for the specified object.
- float getAb (void)

  A function that returns the frequency of the Ab note for the specified object.
- float getA (void)

  A function that returns the frequency of the A note for the specified object.
- float getA_sharp (void)

  A function that returns the frequency of the A# note for the specified object.
- float getBb (void)

  A function that returns the frequency of the Bb note for the specified object.
- float getB (void)

  A function that returns the frequency of the B note for the specified object.
- float getNote (Notes_t note)

  An alternative function which can allow for the user to request a specific note based upon the Notes_t enumerated values rather than specifying a function call.

Protected Member Functions

- void **updateHarmonicCache** (int n)

Protected Attributes

- float **decayConstant** = 0.1f
- std::array< float, maxHarmonics > **harmonicWeights** {}
- int **cachedNHarmonics** = 0
- bool **harmonicCacheDirty** = true

Static Protected Attributes

- static constexpr int **maxHarmonics** = 64

Additional Inherited Members

Public Types inherited from Notes

- enum Notes_t {
  note_C , **note_Db** , **note_D** , **note_Eb** ,
  note_E , **note_F** , **note_Gb** , **note_G** ,
  note_Ab , **note_A** , **note_Bb** , **note_B** ,
  note_C_sharp = note_Db , **note_D_sharp** = note_Eb , **note_F_sharp** = note_Gb , **note_G_sharp** = note↩_Ab ,
  note_A_sharp = note_Bb }

  Creation of the enumerated list of possible notes.

Public Attributes inherited from Octaves

- Notes **octaves** [nOctaves]

  Initialising an array of the octaves and their notes.

Static Public Attributes inherited from Octaves

- static const char **nOctaves** = 9

  Stores the number of octaves being used.

9.30.1 Detailed Description

A class which inherits functions and variables from the `Octaves` class allowing for the notes being output to have a set of harmonic frequencies attached to them too.

9.30.2 Member Function Documentation

9.30.2.1 getDecayConstant()

```
float OctavesWithHarmonics::getDecayConstant (
            void  )
```

A function which returns the decay constant being used to determine the relative ampltiude of each successive harmonic.

Return values

The	floating point representation of the decay constant.

9.30.2.2 getHarmonicDecayMax()

```
float OctavesWithHarmonics::getHarmonicDecayMax (
            int n )
```

A function which returns the maximum amplitude possible for a set number of harmonics if they are all constructively interfering.

Parameters

n	The number of harmonics

Return values

The	maximum amplitude possible as a floating point value.

9.30.2.3 HarmonicDecay()

```
float OctavesWithHarmonics::HarmonicDecay (
            int n )
```

A function which calculates how much the amplitude of the harmonic should decay in comparison to the first harmonic.

Parameters

n	The harmonic which is being used.

Return values

Returns	a floating point value which represents the ratio of amplitudes.

Note

This function needs to be modified to better match the relationship between the frequency and the decay rate over the various harmonic frequencies.

9.30.2.4 PlayingFrequencyWithHarmonics()

```
float OctavesWithHarmonics::PlayingFrequencyWithHarmonics (
            int n,
            float phase )
```

A function which may be called upon to stop playing a range of harmonic frequencies when a specific note is called upon to be played.

Parameters

freq	The frequency in Hz that should be played.
time	The amount of time the signal has been playing. This corresponds to the phase that the waveform is at.

Return values

This	function will return a floating point value representing the normalised changed amplitude of the sigal at that point in time.

9.30.2.5 PlayingNoteWithHarmonics()

```
float OctavesWithHarmonics::PlayingNoteWithHarmonics (
            int n,
            int octave,
            Notes_t note,
            float phase )
```

A function which may be called upon to stop playing a range of harmonic notes when a specific note is called upon to be played.

Parameters

n	The number of harmonics which should be applied to the signal.
octave	The octave in which the first harmonic is in.
note	The note which is being played.
phase	The phase of the waveform at the time of sampling.

Return values

This	function will return a floating point value representing the normalised changed amplitude of the signal at that point in time.

Note

This function could be made to produce a much cleaner response by increasing the number of harmonics present and using non integer harmonics too.

9.30.2.6 setDecayConstant()

```
void OctavesWithHarmonics::setDecayConstant (
            float decayC )
```

A function that can change the value of the decay constant used to set the maxmimum amplitude of each successive harmonic.

Parameters

decayC	The floating point value which the decay constant should be set to.

The documentation for this class was generated from the following files:

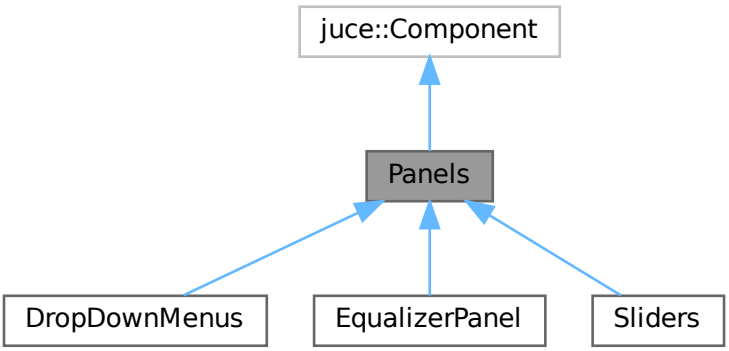
- synth.hpp
- synth.cpp

9.31 Panels Class Reference

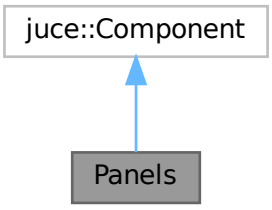
A class which is inherited from Juce:Component reponsible for setting the general style of a panel within the main window.

```
#include <Panels.hpp>
```

Inheritance diagram for Panels:



Collaboration diagram for Panels:



Public Member Functions

- **Panels** ()=default

  The constructor function for the `Panels` class which currently provides no other function than creating an instance of the `Panels` class. I.e. no initialisation functions or settings.
- ~**Panels** ()=default

  The destructor function for the `Panels` class which currently does nothing extra as there is no initialisation within the class.
- void paint (juce::Graphics &g) override

  This function draws onto the parent function the default shape and colour of the panels designed.

- void setBackgroundColour (juce::Colour colour)

  A function to set the background colour of the panels for the main window.
- juce::Colour getBackgroundColour (void)

  A function to get the current background colour being used for the panels within the main window.

9.31.1 Detailed Description

A class which is inherited from Juce:Component reponsible for setting the general style of a panel within the main window.

Note

A panel should represent at least one component such as an image, label, slider or another widget type. Similar types should be grouped together such as all calibration settings.

9.31.2 Member Function Documentation

9.31.2.1 getBackgroundColour()

```
juce::Colour Panels::getBackgroundColour (
            void  )
```

A function to get the current background colour being used for the panels within the main window.

Return values

The	colour being used as a juce::Colour type.

9.31.2.2 paint()

```
void Panels::paint (
            juce::Graphics & g )  [override]
```

This function draws onto the parent function the default shape and colour of the panels designed.

Parameters

g	The point to the `juce::Graphics` instance which is being modified.

Note

The current design uses a rounded rectangle with a dark colour and a slight transparency to have the panel have a slight contrast to the background.

9.31.2.3 setBackgroundColour()

```
void Panels::setBackgroundColour (
            juce::Colour colour )
```

A function to set the background colour of the panels for the main window.

Parameters

colour	The colour which the background should be set to. Must be of type juce::Colour.

The documentation for this class was generated from the following files:

- Panels.hpp
- Panels.cpp

9.32 PitchMapper::PartialDef Struct Reference

Public Attributes

- int **minDelta**
- int **hysteresisDelta**
- int **partial**
- int **midiNote**

The documentation for this struct was generated from the following file:

- PitchMapper.hpp

9.33 PitchMapper Class Reference

Takes raw input values from the LIDAR and mouthpiece/embouchure sensors and maps them to appropriate values for the musical coordinator to work with.

```
#include <PitchMapper.hpp>
```

Classes

- struct PartialDef

Public Member Functions

- **PitchMapper** ()

  Constructor.
- int tof_to_MIDI_bend (uint16_t tof_distance)

  Maps raw LiDAR time-of-flight values to MIDI pitch bend scaled and clamped to range of 0-8192.
- int mouthpiece_to_MIDI_note (int8_t delta)

  Maps capacitative mouthpiece readings to discrete MIDI notes.
- void **SetSlideMaxLimit** (int new_slide_max_limit_mm)

  Set maximum trombone slide limit in mm.
- int **GetSlideMaxLimit** ()

  Return current max trombone slide limit in mm.
- void **SetSlideMinLimit** (int new_slide_min_limit_mm)

  Set minimum trombone slide limit in mm.
- int **GetSlideMinLimit** ()

  Return current min trombone slide limit in mm.
- void **SetMouthMIDI** (int new_MIDI_note)

  Set current mouthpiece MIDI note, bypassing mapping function.
- int **GetMouthMIDI** ()

  Return current mouthpiece MIDI note.
- void **SetTromboneType** (ShiftKeyingOptions_t trombone_choice)

9.33.1 Detailed Description

Takes raw input values from the LIDAR and mouthpiece/embouchure sensors and maps them to appropriate values for the musical coordinator to work with.

9.33.2 Member Function Documentation

9.33.2.1 mouthpiece_to_MIDI_note()

```
int PitchMapper::mouthpiece_to_MIDI_note (
            int8_t delta )
```

Maps capacitative mouthpiece readings to discrete MIDI notes.

Parameters

delta	Raw value supplied by the cap1188

9.33.2.2 tof_to_MIDI_bend()

```
int PitchMapper::tof_to_MIDI_bend (
            uint16_t tof_distance )
```

Maps raw LiDAR time-of-flight values to MIDI pitch bend scaled and clamped to range of 0-8192.

Parameters

tof_distance	Raw value returned from time-of-flight sensor.

Returns

Integer 0-8192

The documentation for this class was generated from the following files:

- PitchMapper.hpp
- PitchMapper.cpp

9.34 PressureSensor Class Reference

Wrapper around ADS1115rpi library (https://github.com/berndporr/rpi_ads1115) by Bernd Porr that provides an interface for pressure sensing.

```
#include <pressure_sensor.hpp>
```

Public Types

- using **PressureCallback** = std::function< void(float)>

Public Member Functions

- **PressureSensor** (const PressureSensor &)=delete
- PressureSensor & **operator=** (const PressureSensor &)=delete
- void **registerCallback** (PressureCallback cb)
- void **start** (ADS1115settings settings=getDefaultSettings())
- void **stop** ()
- ADS1115rpi & **getADC** ()

Static Public Member Functions

- static ADS1115settings **getDefaultSettings** ()

9.34.1 Detailed Description

Wrapper around ADS1115rpi library (https://github.com/berndporr/rpi_ads1115) by Bernd Porr that provides an interface for pressure sensing.

The documentation for this class was generated from the following file:

- pressure_sensor.hpp

9.35 RawInputEvent Struct Reference

Represents a generic input event produced by hardware sensors.

```
#include <eventHandler.hpp>
```

Public Types

- enum class Type { ToFDistance , PressureReading , MouthpieceReading , Keycontrol }

  Enumeration of supported input event types.

Public Attributes

- Type type {}
- uint16_t tofDistance {}
- float pressureReading {}
- int8_t mouthpieceReading {}
- char keycontrol {}

9.35.1 Detailed Description

Represents a generic input event produced by hardware sensors.

This struct encapsulates all possible raw input types generated by the system. Only the field relevant to the event type should be considered valid.

9.35.2 Member Enumeration Documentation

9.35.2.1 Type

```
enum class RawInputEvent::Type  [strong]
```

Enumeration of supported input event types.

Enumerator

ToFDistance	Time-of-Flight distance measurement (mm).
PressureReading	Breath/pressure sensor reading.
MouthpieceReading	Capacitive embouchure input.
Keycontrol	Keyboard/control input for testing or overrides.

9.35.3 Member Data Documentation

9.35.3.1 keycontrol

```
char RawInputEvent::keycontrol {}
```

Key input (valid if Keycontrol).

9.35.3.2 mouthpieceReading

```
int8_t RawInputEvent::mouthpieceReading {}
```

Embouchure state (valid if MouthpieceReading).

9.35.3.3 pressureReading

```
float RawInputEvent::pressureReading {}
```

Pressure value (valid if PressureReading).

9.35.3.4 tofDistance

```
uint16_t RawInputEvent::tofDistance {}
```

Distance measurement (valid if ToFDistance).

9.35.3.5 type

```
Type RawInputEvent::type {}
```

Type of the event.

The documentation for this struct was generated from the following file:

- eventHandler.hpp

9.36 RtMidiSink Class Reference

Handles MIDI output to external devices using RtMidi.

```
#include <USBMidi.hpp>
```

Public Member Functions

- RtMidiSink (bool isTest)

  Constructs the MIDI sink.
- void send (const MidiMessage &message)

  Sends a MIDI message to the selected output device.
- void DetectUSBMidiOutput ()

  Detects and selects a suitable USB MIDI output device.
- bool GetDeviceStatus ()

  Returns the status of the external MIDI device.

9.36.1 Detailed Description

Handles MIDI output to external devices using RtMidi.

This class wraps the RtMidiOut interface to provide a simplified mechanism for sending MIDI messages to a connected USB MIDI device.

It is responsible for:

- Detecting available MIDI output ports
- Selecting and opening an external MIDI device
- Sending formatted MIDI messages

The class maintains state indicating whether a valid external device is available, allowing higher-level components to fall back to internal synthesis if required.

9.36.2 Constructor & Destructor Documentation

9.36.2.1 RtMidiSink()

```
RtMidiSink::RtMidiSink (
            bool isTest )
```

Constructs the MIDI sink.

Parameters

isTest	Enables test mode (may disable actual device interaction).

9.36.3 Member Function Documentation

9.36.3.1 DetectUSBMidiOutput()

```
void RtMidiSink::DetectUSBMidiOutput ( )
```

Detects and selects a suitable USB MIDI output device.

Scans available MIDI output ports and attempts to identify an external device. If successful, stores the port index and marks the device as present.

9.36.3.2 GetDeviceStatus()

```
bool RtMidiSink::GetDeviceStatus ( )
```

Returns the status of the external MIDI device.

Returns

True if a valid external device is available, false otherwise.

9.36.3.3 send()

```
void RtMidiSink::send (
            const MidiMessage & message )
```

Sends a MIDI message to the selected output device.

Parameters

message	MIDI message to transmit.

The message is forwarded to the currently opened RtMidi output port. Behaviour is undefined if no valid device is available.

The documentation for this class was generated from the following files:

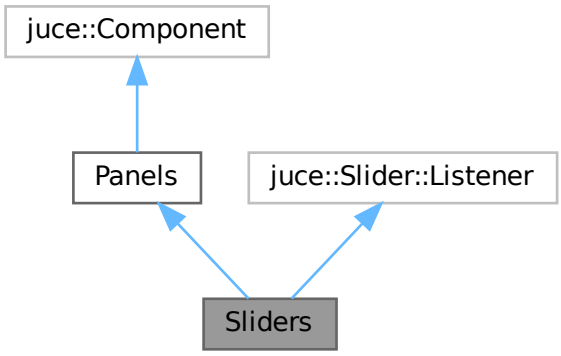
- USBMidi.hpp
- USBMidi.cpp

9.37 Sliders Class Reference

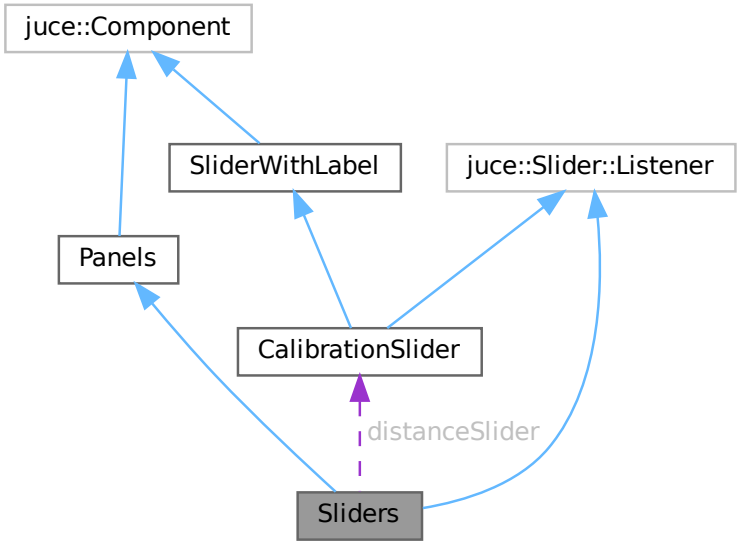
A class which adds all the linear sliders being used within the calibration section to be within the same panel.

```
#include <Panels.hpp>
```

Inheritance diagram for Sliders:



Collaboration diagram for Sliders:



Public Member Functions

- **Sliders** (CoreWrapper &coreWrapper)

  The constructor function for the `Sliders` class which sets the maximum and minimum distance of the slider as well as its knobs' initial positions and other such initialisation steps.
- ~**Sliders** ()

  The destructor function for the `Sliders` class which removes the look and feel of the sliders from the sliders as well as their listeners.

- void resized () override

  Adding a function that will allow for the `Sliders` class to be resized when the window size is adjusted to fit best.
- void sliderValueChanged (juce::Slider *sliderChanged) override

  A callback function which is automatically called upon when one of the sliders present has the value of one of it's knobs changed.
- void setMinimumDistance (float distance)

  A function to set the minimum distance to which the slider may be set to.
- float getMinimumDistance (void)

  A function to get the current minimum distance to which the slider may be set to.
- void setMaximumDistance (float distance)

  A function to set the maximum distance to which the slider may be set to.
- float getMaximumDistance (void)

  A function to get the current maximum distance to which the slider may be set to.
- void setStepDistance (float step)

  A function to set the minimum step to which the slider may be moved by the user.
- float getStepDistance (void)

  A function to get the current minimum distance to which the slider may be moved by the user (step size or interval).
- void setDistanceRange (float range)

  A function to set the minimum range between the maximum and minimum fingers on the slider.
- float getDistanceRange (void)

  A function to get the current minimum distance to which the sliders lower finger may be away from the higher finger.

Public Member Functions inherited from Panels

- **Panels** ()=default

  The constructor function for the `Panels` class which currently provides no other function than creating an instance of the `Panels` class. I.e. no initialisation functions or settings.
- ~**Panels** ()=default

  The destructor function for the `Panels` class which currently does nothing extra as there is no initialisation within the class.
- void paint (juce::Graphics &g) override

  This function draws onto the parent function the default shape and colour of the panels designed.
- void setBackgroundColour (juce::Colour colour)

  A function to set the background colour of the panels for the main window.
- juce::Colour getBackgroundColour (void)

  A function to get the current background colour being used for the panels within the main window.

Public Attributes

- CalibrationSlider distanceSlider

9.37.1 Detailed Description

A class which adds all the linear sliders being used within the calibration section to be within the same panel.

9.37.2 Member Function Documentation

9.37.2.1 getDistanceRange()

```
float Sliders::getDistanceRange (
            void  )
```

A function to get the current minimum distance to which the sliders lower finger may be away from the higher finger.

Return values

The	floating point representaion of the minimum distance the between the slider fingers.

9.37.2.2 getMaximumDistance()

```
float Sliders::getMaximumDistance (
            void  )
```

A function to get the current maximum distance to which the slider may be set to.

Return values

The	floating point representaion of the maximum distance the slider is currently set to.

9.37.2.3 getMinimumDistance()

```
float Sliders::getMinimumDistance (
            void  )
```

A function to get the current minimum distance to which the slider may be set to.

Return values

The	floating point representaion of the minimum distance the slider is currently set to.

9.37.2.4 getStepDistance()

```
float Sliders::getStepDistance (
            void  )
```

A function to get the current minimum distance to which the slider may be moved by the user (step size or interval).

Return values

The	floating point representaion of the minimum distance the slider can currently be moved by.

9.37.2.5 resized()

```
void Sliders::resized ( )  [override]
```

Adding a function that will allow for the `Sliders` class to be resized when the window size is adjusted to fit best.

Note

For this function, this sets the size of the labels to be a maximum size and scales them downwards if the relative size of the window is too small to fit the phrase. This scales the length of the trombone to be able to fit in a window with a width of any size.

9.37.2.6 setDistanceRange()

```
void Sliders::setDistanceRange (
            float range )
```

A function to set the minimum range between the maximum and minimum fingers on the slider.

Parameters

range	The floating point representation of the minimum range.

9.37.2.7 setMaximumDistance()

```
void Sliders::setMaximumDistance (
            float distance )
```

A function to set the maximum distance to which the slider may be set to.

Parameters

distance	The floating point representation of the maximum distance.

9.37.2.8 setMinimumDistance()

```
void Sliders::setMinimumDistance (
            float distance )
```

A function to set the minimum distance to which the slider may be set to.

Parameters

distance	The floating point representation of the minimum distance.

9.37.2.9 setStepDistance()

```
void Sliders::setStepDistance (
            float step )
```

A function to set the minimum step to which the slider may be moved by the user.

Parameters

step	The floating point representation of the minimum step.

9.37.2.10 sliderValueChanged()

```
void Sliders::sliderValueChanged (
            juce::Slider * sliderChanged )  [override]
```

A callback function which is automatically called upon when one of the sliders present has the value of one of it's knobs changed.

Parameters

sliderChanged	A pointer to the slider which has been modified. This allows for the comparison between the different sliders within the class to detect which one has been affected.

Note

This is where the interaction with the Core side of the Tromboneless takes place by referring to the specific variable which stores all the calibration data for the Tromboneless.

9.37.3 Member Data Documentation

9.37.3.1 distanceSlider

```
CalibrationSlider Sliders::distanceSlider
```

Creating an instance of the CalibrationSlider class to be used for calibrating the maximum and minimum distance which the trombonist wishes to have their hand for the pitch bend.

The documentation for this class was generated from the following files:

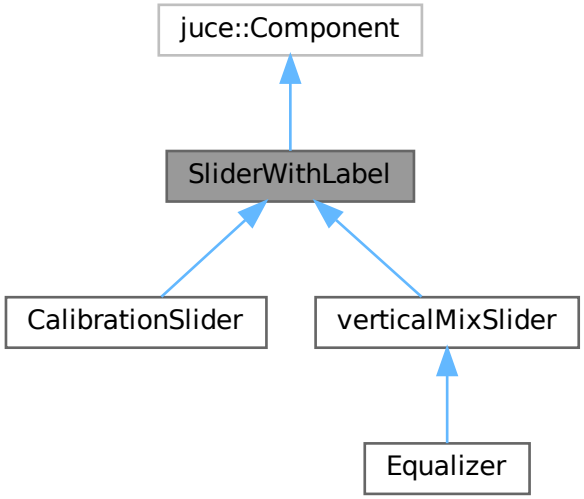
- Panels.hpp
- Panels.cpp

9.38 SliderWithLabel Class Reference

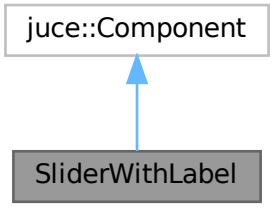
A class which inherits attributes from the juce::Component class to create a new component containing a 8 possible label locations around the slider.

```
#include <Widgets.hpp>
```

Inheritance diagram for SliderWithLabel:



Collaboration diagram for SliderWithLabel:



Public Types

- enum LabelPositions_t {
  UpperLeft , **UpperCentre** , **UpperRight** , **MiddleLeft** ,
  MiddleRight , **LowerLeft** , **LowerCentre** , **LowerRight** }

  Enumerated list of possible positions for the label to be attached relative to the slider.
- enum InitialiseLabelErrors_t { **LabelOK** , **PositonNotAccepted** }

  Enumerated list of possible errors when initialising the labels within this component.

Public Member Functions

- **SliderWithLabel** (juce::Slider::SliderStyle style)

  Constructor which sets the style of the slider being used and removes the text entry option.
- InitialiseLabelErrors_t CreateLabel (LabelPositions_t position, juce::String phrase, juce::NotificationType NotificationEnable=juce::dontSendNotification)

  A function which may be called to create or edit a specified label around the slider.
- void resized () override

  A function which is automatically called whenever the window size is modified. This automatically adjusts the bounds of each of the labels as well as the slider to ensure the size is correct with respect to the screen size for the component.
- void setTopLabelBounds (int bound)

  A function to set the bounds for which the top labels may be situated in.
- int getTopLabelBounds (void)

  A function to get the bounds for which the top labels are situated in.
- void setLeftLabelBounds (int bound)

  A function to set the bounds for which the labels on the left may be situated in.
- int getLeftLabelBounds (void)

  A function to get the bounds for which the labels on the left are situated in.
- void setRightLabelBounds (int bound)

  A function to set the bounds for which the labels on the right may be situated in.
- int getRightLabelBounds (void)

  A function to get the bounds for which the labels on the right are situated in.
- void setBottomLabelBounds (int bound)

  A function to set the bounds for which the bottom labels may be situated in.
- int getBottomLabelBounds (void)

  A function to get the bounds for which the bottom labels are situated in.

Public Attributes

- juce::Slider **slider**

  The slider which is having the labels added to it.

9.38.1 Detailed Description

A class which inherits attributes from the juce::Component class to create a new component containing a 8 possible label locations around the slider.

9.38.2 Member Function Documentation

9.38.2.1 CreateLabel()

```
SliderWithLabel::InitialiseLabelErrors_t SliderWithLabel::CreateLabel (
            LabelPositions_t position,
            juce::String phrase,
            juce::NotificationType NotificationEnable = juce::dontSendNotification )
```

A function which may be called to create or edit a specified label around the slider.

Parameters

position	The position index of the label which should be modified. These are saved as an enum in `Widgets.h`.
phrase	An array of type juce::String which contains the phrase which should be displayed where the label is.
NotificationEnable	The type of notification which should be attached to the label. This is set to `dontSendNotification` by default but may be modified.

Note

This function may only address one label at a time, but may be called upon for each label needed. There may only be one label in each position.

9.38.2.2 getBottomLabelBounds()

```
int SliderWithLabel::getBottomLabelBounds (
            void  )
```

A function to get the bounds for which the bottom labels are situated in.

Return values

The	integer representation of the bottom label bounds.

9.38.2.3 getLeftLabelBounds()

```
int SliderWithLabel::getLeftLabelBounds (
            void  )
```

A function to get the bounds for which the labels on the left are situated in.

Return values

The	integer representation of the left label bounds.

9.38.2.4 getRightLabelBounds()

```
int SliderWithLabel::getRightLabelBounds (
            void  )
```

A function to get the bounds for which the labels on the right are situated in.

Return values

The	integer representation of the right label bounds.

9.38.2.5 getTopLabelBounds()

```
int SliderWithLabel::getTopLabelBounds (
            void  )
```

A function to get the bounds for which the top labels are situated in.

Return values

The	integer representation of the top label bounds.

9.38.2.6 resized()

```
void SliderWithLabel::resized ( )  [override]
```

A function which is automatically called whenever the window size is modified. This automatically adjusts the bounds of each of the labels as well as the slider to ensure the size is correct with respect to the screen size for the component.

Note

The bounds are preset to be 20 for the top and bottom labels and 40 for the left and right labels. These values may be overwritten by writting to the ____`LabelBounds` where ___ may be `top`, `right`, `left` or `bottom`.

9.38.2.7 setBottomLabelBounds()

```
void SliderWithLabel::setBottomLabelBounds (
            int bound )
```

A function to set the bounds for which the bottom labels may be situated in.

Parameters

bound	The integer representation of the bounds which are to be set.

9.38.2.8 setLeftLabelBounds()

```
void SliderWithLabel::setLeftLabelBounds (
            int bound )
```

A function to set the bounds for which the labels on the left may be situated in.

Parameters

bound	The integer representation of the bounds which are to be set.

9.38.2.9 setRightLabelBounds()

```
void SliderWithLabel::setRightLabelBounds (
            int bound )
```

A function to set the bounds for which the labels on the right may be situated in.

Parameters

bound	The integer representation of the bounds which are to be set.

9.38.2.10 setTopLabelBounds()

```
void SliderWithLabel::setTopLabelBounds (
            int bound )
```

A function to set the bounds for which the top labels may be situated in.

Parameters

bound	The integer representation of the bounds which are to be set.

The documentation for this class was generated from the following files:

- Widgets.hpp
- Widgets.cpp

9.39 SynthesiserParameters_t Struct Reference

A struct to store any of the synthesier parameters that may be changed within the app. A data changed flag variable should be added to this struct to tell the core to read the new values.

```
#include <tromboneless_data.hpp>
```

Public Attributes

- bool **synthEnable**

  The variable used to determine when the internal synth and equalizer is used.
- float **gains** [10] = {1, 1, 1, 1, 1, 1, 1, 1, 1, 1}

  The chosen gains for the various frequency bands in the equalizer.

9.39.1 Detailed Description

A struct to store any of the synthesier parameters that may be changed within the app. A data changed flag variable should be added to this struct to tell the core to read the new values.

The documentation for this struct was generated from the following file:

- tromboneless_data.hpp

9.40 ToFSensor Class Reference

Time-of-Flight sensor interface using I2C and GPIO interrupt handling.

```
#include <tof_sensor.hpp>
```

Public Types

- using DistanceCallback = std::function< void(uint16_t)>

  Callback type used to publish distance measurements.

Public Member Functions

- ToFSensor (I2CBus &bus, uint8_t i2cAddress7Bit=0x29, const std::string &gpioChipPath="/dev/gpiochip0", unsigned int gpioLine=4, bool isTest=false)

  Construct a ToFSensor object.
- ~**ToFSensor** ()

  Destructor ensures sensor is stopped and resources are released.
- bool initialise ()

  Initialise the sensor and prepare it for operation.
- void registerCallback (DistanceCallback cb)

  Register a callback for new distance measurements.
- void **start** ()

  Start sensor ranging and launch the worker thread.
- void **stop** ()

  Stop the worker thread, stop ranging, and release GPIO resources.

9.40.1 Detailed Description

Time-of-Flight sensor interface using I2C and GPIO interrupt handling.

This class manages a VL53L1X sensor. It communicates with the sensor over I2C, waits for a GPIO interrupt to indicate that a new measurement is ready, and publishes the measured distance via a callback.

9.40.2 Member Typedef Documentation

9.40.2.1 DistanceCallback

```
using ToFSensor::DistanceCallback = std::function<void(uint16_t)>
```

Callback type used to publish distance measurements.

The callback receives one argument:

- measured distance in millimetres

9.40.3 Constructor & Destructor Documentation

9.40.3.1 ToFSensor()

```
ToFSensor::ToFSensor (
            I2CBus & bus,
            uint8_t i2cAddress7Bit = 0x29,
            const std::string & gpioChipPath = "/dev/gpiochip0",
            unsigned int gpioLine = 4,
            bool isTest = false )
```

Construct a ToFSensor object.

Parameters

bus	Reference to an I2CBus object for communication.
i2cAddress7Bit	7-bit I2C address of the sensor in 7-bit Linux form (0x29 by default).
gpioChipPath	Path to the GPIO chip (e.g. "/dev/gpiochip0").
gpioLine	GPIO line connected to the sensor interrupt output (e.g. 4).

9.40.4 Member Function Documentation

9.40.4.1 initialise()

```
bool ToFSensor::initialise ( )
```

Initialise the sensor and prepare it for operation.

Opens the I2C bus, initialises the ST platform layer, resets the sensor, waits for boot, verifies sensor ID, and applies the configured ranging and interrupt settings.

Returns

true if initialisation succeeded.

false if initialisation failed (e.g. I2C communication error).

9.40.4.2 registerCallback()

```
void ToFSensor::registerCallback (
            DistanceCallback cb )
```

Register a callback for new distance measurements.

Parameters

cb	The callback function to call when a new measurement is available.

The documentation for this class was generated from the following files:

- tof_sensor.hpp
- tof_sensor.cpp

9.41 TrombonelessParameters_t Struct Reference

A struct which stores all the necessary parameters that may be changed on the app side effecting the core functionality. This struct should add a flag which is triggered whenever a parameter has been changed.

```
#include <tromboneless_data.hpp>
```

Public Attributes

- ShiftKeyingOptions_t **shiftKeyingOption**

  The chosen shiftkey option by the user corresponding to the type of trombone.
- double **nearDistance**

  The nearest distance the user wishes to have their hand from the sensor for pitch bend.
- double **farDistance**

  The furthest distance the user wishes to have their hand from the sensor for pitch bend.
- double **lowPressure**

  The lower pressure gain which the user has chosen for the mouthpiece corresponding to the peak input.
- double **highPressure**

  The upper pressure gain which the user has chosen for the mouthpiece corresponding to the peak input.
- EmbachureOptions_t **embouchureOption**

  The chosen embouchure option which the user wishes to calibrate.
- bool **triggerEmbouchureCalibrate**

  High/true when the user has pressed the button to calibrate the Embouchure.

9.41.1 Detailed Description

A struct which stores all the necessary parameters that may be changed on the app side effecting the core functionality. This struct should add a flag which is triggered whenever a parameter has been changed.

The documentation for this struct was generated from the following file:

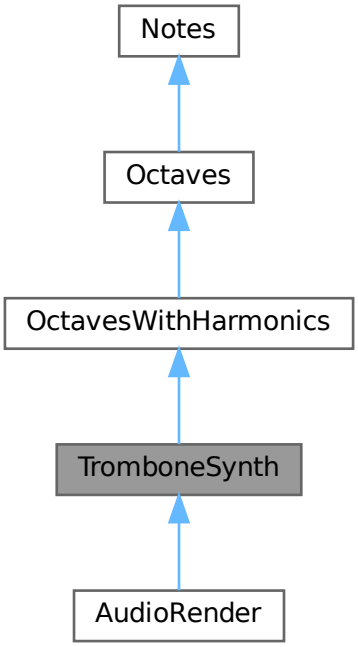
- tromboneless_data.hpp

9.42 TromboneSynth Class Reference

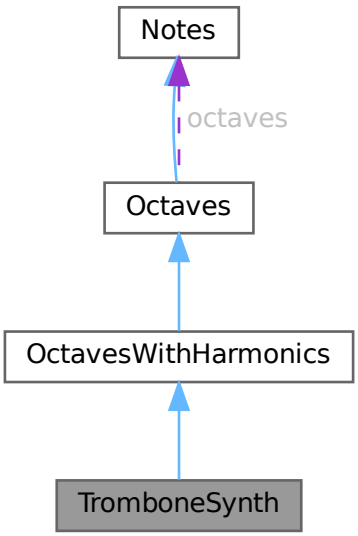
A class which may be used to create a custom synth with an envelope attached to it.

```
#include <tromboneSynth.hpp>
```

Inheritance diagram for TromboneSynth:



Collaboration diagram for TromboneSynth:



Public Member Functions

- TromboneSynth (int sampleRate=44100, float attack=10.0f, float decay=50.0f, float sustain=1.0f, float rest=10.0f)

  The constructor function for the `TromboneSynth` class.
- void StartTromboneNote (Notes::Notes_t note_in, int octave_in)

  A function which may be called to set the note and octave of a new sound by the user. If there is already a note playing, this will stop the previous not and play this one without phase shifting (initially).
- void ChangeTromboneNote (Notes::Notes_t note_in, int octave_in)

  A function that can change the current note without needed to change the stage that the envelope is in.
- float ReadTromboneAudio (void)

  A function which may be called to read the current amplitude of the audio signal being generated.
- void **StopTromboneNote** (void)

  A function which sends the synthesiser into its rest state.
- void NewTromboneNoteMIDI (int MIDINote)

  A function which can take in the MIDI equivalent note and map it to the Notes_t options to match the rest of the synth interface.
- void **HandleMIDINoteOn** ()

  A method to handle NoteOn messages as passed from the MIDI Coordinator.
- void **HandleMIDINoteOff** ()
- void **HandleMIDIPitchBend** (int midipitchbend)
- void **HandleMIDINoteChange** (int new_note)
- void setAttackMS (float attack_time)

  A function to set the amount of time in milliseconds that the attack stage takes.
- float getAttackMS (void)

  A function to get the current attack time in milliseconds.
- void setDecayMS (float decay_time)

  A function to set the amount of time in milliseconds that the decay stage takes.
- float getDecayMS (void)

  A function to get the current decay time in milliseconds.
- void setRestMS (float rest_time)

  A function to set the amount of time in milliseconds that the rest stage takes.
- float getRestMS (void)

  A function to get the current rest time in milliseconds.
- void setSustain (float sustain_in)

  A function to set the relative amplitude of the sustain stage.
- float getSustain (void)

  A function to get the current sustain value.
- void setNHarmonics (int n)

  A function to set the number of harmonics which are being generated for every note being played.
- int getNHarmonics (void)

  A function to get the number of harmonics which are being generated for every note being played.
- int getOctave (void)

  A function to get the current octave which the current note is in.
- Notes::Notes_t getNote (void)

  A function to get the current note which is being used.
- void setPitchBend (int bend)

  A function to set the pitch bend of the device.
- int getPitchBend (void)

  A function to get the pitch bend of the device.
- float getAdjustedFrequency (void)

  A function which determines the new frequency which should be being played depending upon the current note and octave as well as the pitch bend being input.
- Envelope **getEnvelope** (void)

Public Member Functions inherited from OctavesWithHarmonics

- **OctavesWithHarmonics** ()

  Creates an instance of the `OctavesWithHarmonics`.
- float PlayingNoteWithHarmonics (int n, int octave, Notes_t note, float phase)

  A function which may be called upon to stop playing a range of harmonic notes when a specific note is called upon to be played.
- float PlayingFrequencyWithHarmonics (int n, float phase)

  A function which may be called upon to stop playing a range of harmonic frequencies when a specific note is called upon to be played.
- float HarmonicDecay (int n)

  A function which calculates how much the amplitude of the harmonic should decay in comparison to the first harmonic.
- float getHarmonicDecayMax (int n)

  A function which returns the maximum amplitude possible for a set number of harmonics if they are all constructively interfering.
- void setDecayConstant (float decayC)

  A function that can change the value of the decay constant used to set the maxmimum amplitude of each successive harmonic.
- float getDecayConstant (void)

  A function which returns the decay constant being used to determine the relative ampltiude of each successive harmonic.

Public Member Functions inherited from Octaves

- float PlayingNote (int octave, Notes_t note, float phase)

  A function which may be called to determine the amplitude of a specified note at a specified phase.
- float PlayingFrequency (float phase)

  A function which may be called to determine the amplitude of a specified frequency at a specified time.

Public Member Functions inherited from Notes

- Notes (float C_in=16.35, float Db_in=17.32, float D_in=18.35, float Eb_in=19.45, float E_in=20.60, float F↩ _in=21.83, float Gb_in=23.12, float G_in=24.50, float Ab_in=25.96, float A_in=27.50, float Bb_in=29.14, float B_in=30.87)

  Constructor for the `Notes` class to create a new set of notes and assign specified frequencies to each. Each frequency is set to `Octave 0` by default.
- void setC (float frequency)

  A function that can change the value of the frequency that the C note plays.
- void setC_sharp (float frequency)

  A function that can change the value of the frequency that the C# note plays.
- void setDb (float frequency)

  A function that can change the value of the frequency that the Db note plays.
- void setD (float frequency)

  A function that can change the value of the frequency that the D note plays.
- void setD_sharp (float frequency)

  A function that can change the value of the frequency that the D# note plays.
- void setEb (float frequency)

  A function that can change the value of the frequency that the Eb note plays.
- void setE (float frequency)

  A function that can change the value of the frequency that the E note plays.
- void setF (float frequency)

A function that can change the value of the frequency that the F note plays.

- void setF_sharp (float frequency)

  A function that can change the value of the frequency that the F# note plays.
- void setGb (float frequency)

  A function that can change the value of the frequency that the Gb note plays.
- void setG (float frequency)

  A function that can change the value of the frequency that the G note plays.
- void setG_sharp (float frequency)

  A function that can change the value of the frequency that the G# note plays.
- void setAb (float frequency)

  A function that can change the value of the frequency that the Ab note plays.
- void setA (float frequency)

  A function that can change the value of the frequency that the A note plays.
- void setA_sharp (float frequency)

  A function that can change the value of the frequency that the A# note plays.
- void setBb (float frequency)

  A function that can change the value of the frequency that the Bb note plays.
- void setB (float frequency)

  A function that can change the value of the frequency that the B note plays.
- float setNote (Notes_t note, float freq)

  An alternative function which can allow for the user to alter the frequency of a specific note based upon the Notes_t enumerated values rather than specifying a function call.
- float getC (void)

  A function that returns the frequency of the C note for the specified object.
- float getC_sharp (void)

  A function that returns the frequency of the C# note for the specified object.
- float getDb (void)

  A function that returns the frequency of the Db note for the specified object.
- float getD (void)

  A function that returns the frequency of the D note for the specified object.
- float getD_sharp (void)

  A function that returns the frequency of the D# note for the specified object.
- float getEb (void)

  A function that returns the frequency of the Eb note for the specified object.
- float getE (void)

  A function that returns the frequency of the E note for the specified object.
- float getF (void)

  A function that returns the frequency of the F note for the specified object.
- float getF_sharp (void)

  A function that returns the frequency of the F# note for the specified object.
- float getGb (void)

  A function that returns the frequency of the Gb note for the specified object.
- float getG (void)

  A function that returns the frequency of the G note for the specified object.
- float getG_sharp (void)

  A function that returns the frequency of the G# note for the specified object.
- float getAb (void)

  A function that returns the frequency of the Ab note for the specified object.
- float getA (void)

  A function that returns the frequency of the A note for the specified object.
- float getA_sharp (void)

A function that returns the frequency of the A# note for the specified object.

- float getBb (void)

  A function that returns the frequency of the Bb note for the specified object.

- float getB (void)

  A function that returns the frequency of the B note for the specified object.

- float getNote (Notes_t note)

  An alternative function which can allow for the user to request a specific note based upon the Notes_t enumerated values rather than specifying a function call.

Additional Inherited Members

Public Types inherited from Notes

- enum Notes_t {
  note_C , **note_Db** , **note_D** , **note_Eb** ,
  note_E , **note_F** , **note_Gb** , **note_G** ,
  note_Ab , **note_A** , **note_Bb** , **note_B** ,
  note_C_sharp = note_Db , **note_D_sharp** = note_Eb , **note_F_sharp** = note_Gb , **note_G_sharp** = note↩
  _Ab ,
  note_A_sharp = note_Bb }

  Creation of the enumerated list of possible notes.

Public Attributes inherited from Octaves

- Notes **octaves** [nOctaves]

  Initialising an array of the octaves and their notes.

Static Public Attributes inherited from Octaves

- static const char **nOctaves** = 9

  Stores the number of octaves being used.

Protected Member Functions inherited from OctavesWithHarmonics

- void **updateHarmonicCache** (int n)

Protected Attributes inherited from OctavesWithHarmonics

- float **decayConstant** = 0.1f
- std::array< float, maxHarmonics > **harmonicWeights** {}
- int **cachedNHarmonics** = 0
- bool **harmonicCacheDirty** = true

Static Protected Attributes inherited from OctavesWithHarmonics

- static constexpr int **maxHarmonics** = 64

9.42.1 Detailed Description

A class which may be used to create a custom synth with an envelope attached to it.

9.42.2 Constructor & Destructor Documentation

9.42.2.1 TromboneSynth()

```
TromboneSynth::TromboneSynth (
            int sampleRate = 44100,
            float attack = 10.0f,
            float decay = 50.0f,
            float sustain = 1.0f,
            float rest = 10.0f )
```

The constructor function for the `TromboneSynth` class.

Parameters

sampleRate	The sample rate which will be used to measure the amplitude of the sound wave being synthesised. This should be passed through in Hz.
attack	The amount of time in ms that the attack should last for.
decay	The amount of time in ms that the decay should last for.
sustain	The relative amplitude that is being output at the steady state.
rest	The amount of time in milliseconds that the rest stage should last for.

9.42.3 Member Function Documentation

9.42.3.1 ChangeTromboneNote()

```
void TromboneSynth::ChangeTromboneNote (
            Notes::Notes_t note_in,
            int octave_in )
```

A function that can change the current note without needed to change the stage that the envelope is in.

Parameters

note_in	The note which the synthesiser should start to play.
octave↩ _in	The octave in which the note is in.

9.42.3.2 getAdjustedFrequency()

```
float TromboneSynth::getAdjustedFrequency (
            void )
```

A function which determines the new frequency which should be being played depending upon the current note and octave as well as the pitch bend being input.

Return values

The	floating point representation of the frequency.

9.42.3.3 getAttackMS()

```
float TromboneSynth::getAttackMS (
            void  )
```

A function to get the current attack time in milliseconds.

Return values

The	time in milliseconds as a floating point value.

9.42.3.4 getDecayMS()

```
float TromboneSynth::getDecayMS (
            void  )
```

A function to get the current decay time in milliseconds.

Return values

The	time in milliseconds as a floating point value.

9.42.3.5 getNHarmonics()

```
int TromboneSynth::getNHarmonics (
            void  )
```

A function to get the number of harmonics which are being generated for every note being played.

Return values

The	integer representation of the number of harmonics.

9.42.3.6 getNote()

```
Notes::Notes_t TromboneSynth::getNote (
            void  )
```

A function to get the current note which is being used.

Return values

The	note of type Notes::Notes_t being played.

9.42.3.7 getOctave()

```
int TromboneSynth::getOctave (
            void  )
```

A function to get the current octave which the current note is in.

Return values

The	octave as an integer representation.

9.42.3.8 getPitchBend()

```
int TromboneSynth::getPitchBend (
            void  )
```

A function to get the pitch bend of the device.

Return values

The	integer representation between 8192 and 0 of how much of the pitch has been bent.

9.42.3.9 getRestMS()

```
float TromboneSynth::getRestMS (
            void  )
```

A function to get the current rest time in milliseconds.

Return values

The	time in milliseconds as a floating point value.

9.42.3.10 getSustain()

```
float TromboneSynth::getSustain (
            void  )
```

A function to get the current sustain value.

Return values

The	floating point relative amplitude

9.42.3.11 NewTromboneNoteMIDI()

```
void TromboneSynth::NewTromboneNoteMIDI (
            int MIDINote )
```

A function which can take in the MIDI equivalent note and map it to the Notes_t options to match the rest of the synth interface.

Parameters

MIDINote	The integer value between 0 and 127 representing the note which is being played.

9.42.3.12 ReadTromboneAudio()

```
float TromboneSynth::ReadTromboneAudio (
            void  )
```

A function which may be called to read the current amplitude of the audio signal being generated.

Return values

Returns	a floating point representation of the amplitude normalised between -1 and 1.

9.42.3.13 setAttackMS()

```
void TromboneSynth::setAttackMS (
            float attack_time )
```

A function to set the amount of time in milliseconds that the attack stage takes.

Parameters

attack_time	The time in milliseconds.

9.42.3.14 setDecayMS()

```
void TromboneSynth::setDecayMS (
            float decay_time )
```

A function to set the amount of time in milliseconds that the decay stage takes.

Parameters

decay_time	The time in milliseconds.

9.42.3.15 setNHarmonics()

```
void TromboneSynth::setNHarmonics (
            int n )
```

A function to set the number of harmonics which are being generated for every note being played.

Parameters

n	The number of harmonics which should be added to each note.

9.42.3.16 setPitchBend()

```
void TromboneSynth::setPitchBend (
            int bend )
```

A function to set the pitch bend of the device.

Parameters

bend	The integer representation between 8192 and 0 of how much of the pitch has been bent.

9.42.3.17 setRestMS()

```
void TromboneSynth::setRestMS (
            float rest_time )
```

A function to set the amount of time in milliseconds that the rest stage takes.

Parameters

rest_time	The time in milliseconds.

9.42.3.18 setSustain()

```
void TromboneSynth::setSustain (
            float sustain_in )
```

A function to set the relative amplitude of the sustain stage.

Parameters

sustain↩ _in	The floating point relative amplitde between 0 and 1.

9.42.3.19 StartTromboneNote()

```
void TromboneSynth::StartTromboneNote (
            Notes::Notes_t note_in,
            int octave_in )
```

A function which may be called to set the note and octave of a new sound by the user. If there is already a note playing, this will stop the previous not and play this one without phase shifting (initially).

Parameters

note_in	The note which should start to play.
octave↩ _in	The octave which the note is in.

The documentation for this class was generated from the following files:

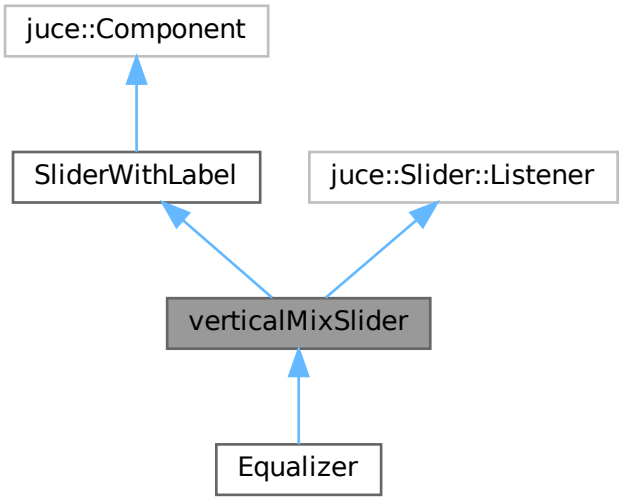
- tromboneSynth.hpp
- tromboneSynth.cpp

9.43 verticalMixSlider Class Reference

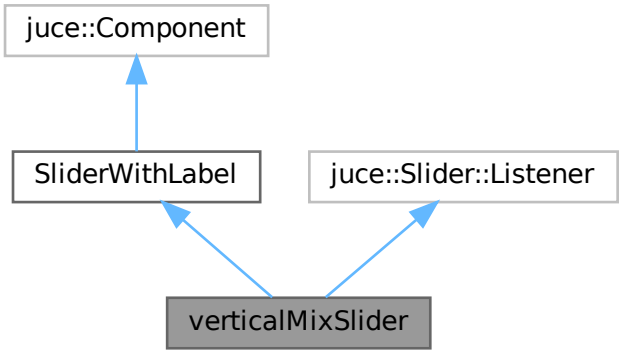
A class that adds more functionality to the slider class from Juce.

```
#include <Widgets.hpp>
```

Inheritance diagram for verticalMixSlider:



Collaboration diagram for verticalMixSlider:



Public Member Functions

- **verticalMixSlider** ()

  Constructor function for the vericalMixSlider which creates an instance of the class and sets the parameters to match it best such as setting the range to be between -9 dB and 9 dB with a step size of 3 dB as well as label size.
- ~**verticalMixSlider** ()

  Destructor function for the verticalMixSlider class which removes the listener from the slider before killing the object.
- void sliderValueChanged (juce::Slider *sliderChanged) override

  A callback function for when the slider used has been adjusted and its value may have been changed.

Public Member Functions inherited from SliderWithLabel

- **SliderWithLabel** (juce::Slider::SliderStyle style)

  Constructor which sets the style of the slider being used and removes the text entry option.
- InitialiseLabelErrors_t CreateLabel (LabelPositions_t position, juce::String phrase, juce::NotificationType NotificationEnable=juce::dontSendNotification)

  A function which may be called to create or edit a specified label around the slider.
- void resized () override

  A function which is automatically called whenever the window size is modified. This automatically adjusts the bounds of each of the labels as well as the slider to ensure the size is correct with respect to the screen size for the component.
- void setTopLabelBounds (int bound)

  A function to set the bounds for which the top labels may be situated in.
- int getTopLabelBounds (void)

  A function to get the bounds for which the top labels are situated in.
- void setLeftLabelBounds (int bound)

  A function to set the bounds for which the labels on the left may be situated in.
- int getLeftLabelBounds (void)

  A function to get the bounds for which the labels on the left are situated in.
- void setRightLabelBounds (int bound)

  A function to set the bounds for which the labels on the right may be situated in.
- int getRightLabelBounds (void)

  A function to get the bounds for which the labels on the right are situated in.
- void setBottomLabelBounds (int bound)

  A function to set the bounds for which the bottom labels may be situated in.
- int getBottomLabelBounds (void)

  A function to get the bounds for which the bottom labels are situated in.

Additional Inherited Members

Public Types inherited from SliderWithLabel

- enum LabelPositions_t {
  UpperLeft , **UpperCentre** , **UpperRight** , **MiddleLeft** ,
  MiddleRight , **LowerLeft** , **LowerCentre** , **LowerRight** }

  Enumerated list of possible positions for the label to be attached relative to the slider.
- enum InitialiseLabelErrors_t { **LabelOK** , **PositonNotAccepted** }

  Enumerated list of possible errors when initialising the labels within this component.

Public Attributes inherited from SliderWithLabel

- juce::Slider **slider**

  The slider which is having the labels added to it.

9.43.1 Detailed Description

A class that adds more functionality to the slider class from Juce.

9.43.2 Member Function Documentation

9.43.2.1 sliderValueChanged()

```
void verticalMixSlider::sliderValueChanged (
            juce::Slider * sliderChanged )  [override]
```

A callback function for when the slider used has been adjusted and its value may have been changed.

Parameters

sliderChanged	This is a pointer to the slider which has been changed.

Note

As this class only has one slider that is in it, there is no need to use it but the JUCE function passes it anyway.

The documentation for this class was generated from the following files:

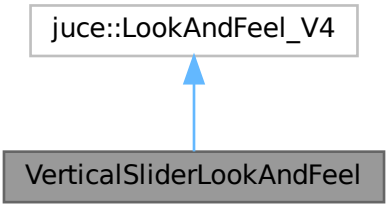
- Widgets.hpp
- Widgets.cpp

9.44 VerticalSliderLookAndFeel Class Reference

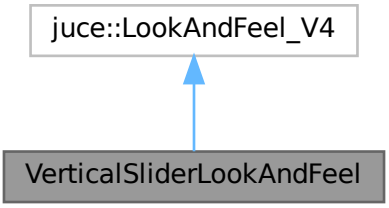
Creation of a class which can be used to change the look of the sliders used within the equalizer for the Trombone-less project.

```
#include <CustomStyles.hpp>
```

Inheritance diagram for VerticalSliderLookAndFeel:



Collaboration diagram for VerticalSliderLookAndFeel:



Public Member Functions

- VerticalSliderLookAndFeel ()

  The constructor function for the `VerticalSliderLookAndFeel` class that creates an instance of the look and feel style.
- virtual ~**VerticalSliderLookAndFeel** ()=default

  The destructor function for the `VerticalSliderLookAndFeel` class which is currently set to default as there is no need for a deconstructor for this class.
- virtual void drawLinearSlider (juce::Graphics &g, int x, int y, int width, int height, float sliderPos, float min↩SliderPos, float maxSliderPos, const juce::Slider::SliderStyle style, juce::Slider &slider) override

  When this look and feel is attached to an object which has a vertical slider in it, this function sets the look and feel of the slider being used. This makes it look like an average equalizer slider.
- void setDialColour (const juce::Colour colour)

  A function to set the colour of the finger/dial of the slider.
- juce::Colour getDialColour (void)

  A function to get the current colour of the finger/dial of the slider.
- void setDialEdgeColour (const juce::Colour &colour)

  A function to set the colour of the edge of the finger/dial on the slider.
- juce::Colour getDialEdgeColour (void)

  A function to get the current colour of the edge of the finger /dial of the slider.
- void setTickColour (juce::Colour colour)

  A function to set the colour of the ticks on the slider.
- juce::Colour getTickColour (void)

  A function to get the current colour of the ticks on the slider.
- void setTrackColour (juce::Colour colour)

  A function to set the colour of the track of the slider.
- juce::Colour getTrackColour (void)

  A function to get the current colour of the track of the slider.
- void setBackgroundColour (juce::Colour colour)

  A function to set the colour of the background of the slider.
- juce::Colour getBackgroundColour (void)

  A function to get the current colour of the background of the slider.
- void setTextColour (juce::Colour colour)

  A function to set the colour of the text around the slider.
- juce::Colour getTextColour (void)

  A function to get the current colour of the text around the slider.
- void setTrackWidth (float width)

  A function to set the size of the track width of the slider.
- float getTrackWidth (void)

  A function to get the track width of the slider.
- void setTickThickness (float thickness)

  A function to set the thickness of the ticks to indicate what step the slider is positioned in.
- float getTickThickness (void)

  A function to get what the current tick thickness is for the object.
- void setTickWidth (float width)

  A function to set the width of the ticks to indicate what step the slider is positioned in.
- float getTickWidth (void)

  A function which may be used to find the current tick width of the object.
- void setFingerWidth (float width)

  A function to set the width of the slider's finger/knob when the slider is set to have a rectangular shape.
- float getFingerWidth (void)

A function to get the current width of the slider's finger/knob within the object when the slider is set to have a rectangular shape.

- void setFingerHeight (float height)

  A function to set the height of the slider's finger/knob when the slider is set to have a rectangular shape.

- float getFingerHeight (void)

  A function to get the current height of the slider's finger/knob within the object when the slider is set to have a rectangular shape.

9.44.1 Detailed Description

Creation of a class which can be used to change the look of the sliders used within the equalizer for the Trombone-less project.

9.44.2 Constructor & Destructor Documentation

9.44.2.1 VerticalSliderLookAndFeel()

```
VerticalSliderLookAndFeel::VerticalSliderLookAndFeel ( )
```

The constructor function for the `VerticalSliderLookAndFeel` class that creates an instance of the look and feel style.

Note

Currently there is not extra functionality attached to the contructor as the style of the vertical slider is defined in the `drawLinearSlider()` function and there are no other components being modified by this style currently.

9.44.3 Member Function Documentation

9.44.3.1 drawLinearSlider()

```
void VerticalSliderLookAndFeel::drawLinearSlider (
            juce::Graphics & g,
            int x,
            int y,
            int width,
            int height,
            float sliderPos,
            float minSliderPos,
            float maxSliderPos,
            const juce::Slider::SliderStyle style,
            juce::Slider & slider )  [override], [virtual]
```

When this look and feel is attached to an object which has a vertical slider in it, this function sets the look and feel of the slider being used. This makes it look like an average equalizer slider.

Parameters

g	A pointer to the `juce::Graphics` instance which is being drawn to. This is likely the main window or else the parent component.

Parameters

x	The integer representation of the slider's left corner in the global window.
y	The integer representation of the sliders top corner in the global window.
width	The integer representation of the width of the slider. i.e. the difference between the left most and right most part of the slider.
height	The integer representation of the width of the slider. i.e. the difference between the top most and bottom most parts of the slider's bounds.
sliderPos	The position of the sliders finger. The position represents x if the slider is horizontal and y if the slider is vertical.
minSliderPos	The minimum slider position.
maxSliderPos	The maximum slider position.
style	The style of the slider whose look and feel is getting modified. For example, LinearVertical or Rotary.
slider	A pointer to the slider which is being modified. This can allow for extraction of more data from the slider.

9.44.3.2 getBackgroundColour()

```
juce::Colour VerticalSliderLookAndFeel::getBackgroundColour (
            void  )
```

A function to get the current colour of the background of the slider.

Return values

Returns	the colour as a juce::Colour type.

9.44.3.3 getDialColour()

```
juce::Colour VerticalSliderLookAndFeel::getDialColour (
            void  )
```

A function to get the current colour of the finger/dial of the slider.

Return values

Returns	the colour as a juce::Colour type.

9.44.3.4 getDialEdgeColour()

```
juce::Colour VerticalSliderLookAndFeel::getDialEdgeColour (
            void  )
```

A function to get the current colour of the edge of the finger /dial of the slider.

Return values

Returns	the colour as a juce::Colour type.

9.44.3.5 getFingerHeight()

```
float VerticalSliderLookAndFeel::getFingerHeight (
            void  )
```

A function to get the current height of the slider's finger/knob within the object when the slider is set to have a rectangular shape.

Return values

The	floating point representation of the height of the object's finger/knob.

9.44.3.6 getFingerWidth()

```
float VerticalSliderLookAndFeel::getFingerWidth (
            void  )
```

A function to get the current width of the slider's finger/knob within the object when the slider is set to have a rectangular shape.

Return values

The	floating point representation of the width of the object's finger/knob.

9.44.3.7 getTextColour()

```
juce::Colour VerticalSliderLookAndFeel::getTextColour (
            void  )
```

A function to get the current colour of the text around the slider.

Return values

Returns	the colour as a juce::Colour type.

9.44.3.8 getTickColour()

```
juce::Colour VerticalSliderLookAndFeel::getTickColour (
            void  )
```

A function to get the current colour of the ticks on the slider.

Return values

Returns	the colour as a juce::Colour type.

9.44.3.9 getTickThickness()

```
float VerticalSliderLookAndFeel::getTickThickness (
            void  )
```

A function to get what the current tick thickness is for the object.

Return values

Returns	the floating point representation of the tick thickness of the object.

9.44.3.10 getTickWidth()

```
float VerticalSliderLookAndFeel::getTickWidth (
            void  )
```

A function which may be used to find the current tick width of the object.

Return values

Returns	the floating point representation of the tick's width.

9.44.3.11 getTrackColour()

```
juce::Colour VerticalSliderLookAndFeel::getTrackColour (
            void  )
```

A function to get the current colour of the track of the slider.

Return values

Returns	the colour as a juce::Colour type.

9.44.3.12 getTrackWidth()

```
float VerticalSliderLookAndFeel::getTrackWidth (
            void  )
```

A function to get the track width of the slider.

Return values

The	floating point representation of the width of the track in terms of the resoltion.

9.44.3.13 setBackgroundColour()

```
void VerticalSliderLookAndFeel::setBackgroundColour (
            juce::Colour colour )
```

A function to set the colour of the background of the slider.

Parameters

colour	The colour to which the background should be set to.

9.44.3.14 setDialColour()

```
void VerticalSliderLookAndFeel::setDialColour (
            const juce::Colour colour )
```

A function to set the colour of the finger/dial of the slider.

Parameters

colour	The colour to which the dial should be set to.

9.44.3.15 setDialEdgeColour()

```
void VerticalSliderLookAndFeel::setDialEdgeColour (
            const juce::Colour & colour )
```

A function to set the colour of the edge of the finger/dial on the slider.

Parameters

colour	The colour to which the edge of the dial should be set to.

9.44.3.16 setFingerHeight()

```
void VerticalSliderLookAndFeel::setFingerHeight (
            float height )
```

A function to set the height of the slider's finger/knob when the slider is set to have a rectangular shape.

Parameters

height	The floating point representation of the height of the finger/knob in terms of the resolution.

9.44.3.17 setFingerWidth()

```
void VerticalSliderLookAndFeel::setFingerWidth (
            float width )
```

A function to set the width of the slider's finger/knob when the slider is set to have a rectangular shape.

Parameters

width	The floating point representation of the width of the finger/knob in terms of the resolution.

9.44.3.18 setTextColour()

```
void VerticalSliderLookAndFeel::setTextColour (
            juce::Colour colour )
```

A function to set the colour of the text around the slider.

Parameters

colour	The colour to which the text should be set to.

9.44.3.19 setTickColour()

```
void VerticalSliderLookAndFeel::setTickColour (
            juce::Colour colour )
```

A function to set the colour of the ticks on the slider.

Parameters

colour	The colour to which the ticks should be set to.

9.44.3.20 setTickThickness()

```
void VerticalSliderLookAndFeel::setTickThickness (
            float thickness )
```

A function to set the thickness of the ticks to indicate what step the slider is positioned in.

Parameters

thickness	The thickess of the ticks (vertical height) in terms of the resolution.

9.44.3.21 setTickWidth()

```
void VerticalSliderLookAndFeel::setTickWidth (
            float width )
```

A function to set the width of the ticks to indicate what step the slider is positioned in.

Parameters

width	The width of the ticks (horizontal) in terms of the resolution.

9.44.3.22 setTrackColour()

```
void VerticalSliderLookAndFeel::setTrackColour (
            juce::Colour colour )
```

A function to set the colour of the track of the slider.

Parameters

colour	The colour to which the track should be set to.

9.44.3.23 setTrackWidth()

```
void VerticalSliderLookAndFeel::setTrackWidth (
            float width )
```

A function to set the size of the track width of the slider.

Parameters

width	The floating point representation of the width of the track in terms of the resolution.

The documentation for this class was generated from the following files:

- CustomStyles.hpp
- CustomStyles.cpp

Chapter 10

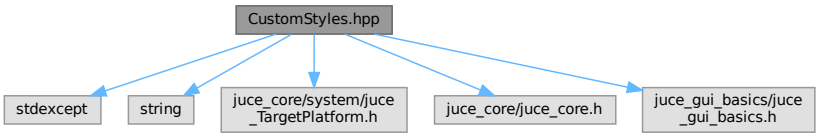
File Documentation

10.1 CustomStyles.hpp File Reference

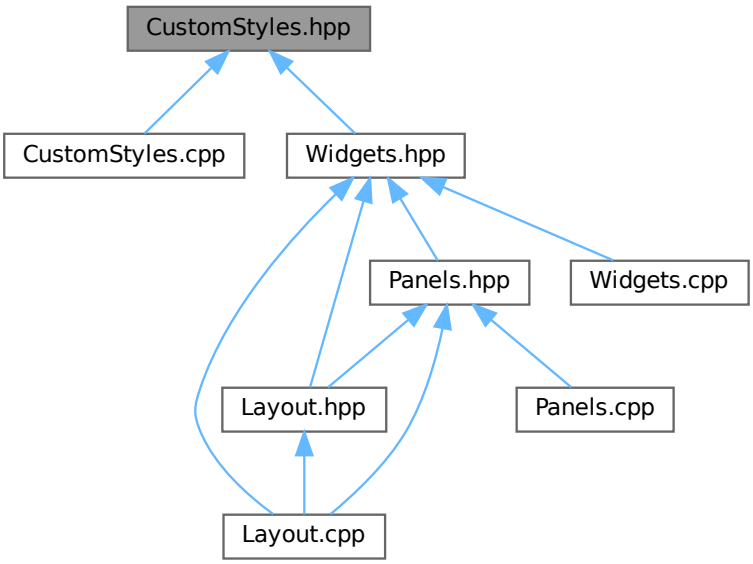
This file holds the declarations of all functions and variables needed by the Tromboneless project to change the looks and feel of various different widgets which are being displayed to the end user.

```
#include <stdexcept>
#include <string>
#include "juce_core/system/juce_TargetPlatform.h"
#include "juce_core/juce_core.h"
#include "juce_gui_basics/juce_gui_basics.h"
```

Include dependency graph for CustomStyles.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class VerticalSliderLookAndFeel

  Creation of a class which can be used to change the look of the sliders used within the equalizer for the Tromboneless project.
- class CalibrationSliderLookAndFeel

  A class to set the look and feel of a two finger linear slider to look like a trombone where the left most finger is the main body and the right most is the end of the trombones slide.
- class NeedleLookAndFeel

  A class which may be attached to a rotary slider from the juce framework to change its look and feel. This changes the look of the slider to have an arrow rotate about the centre of the slider pointing to the different values rather than a knob moving in an arc as the basic rotary slider.

Functions

- juce::Path PaintTrombone (float x=300, float y=300, float width=50, float height=20, bool incPipes=true)

  A function which returns a path matching the perimeter of a trombone.
- juce::Path PaintArc (float x, float y, float thickness, float gap, bool direction)

  A function that traces out the shape of an arc based upon the thickness of the arc and the gap between its inner sides. i.e the inner circle diameter that would remove an area from the larger outer circle.
- juce::Path RotaryArc (float x=100, float y=100, float thickness=5, float width=50, float height=50, float angularRange=5, float rotation=0)

  A function which can draw an arc with a specified thickness and nominal radius.
- juce::Path drawCircle (float x, float y, float xRadius, float yRadius)

  A simpified function for drawing a circle using the juce framework reducing the number of parameters compared to the ellipse function in juce.

10.1.1 Detailed Description

This file holds the declarations of all functions and variables needed by the Tromboneless project to change the looks and feel of various different widgets which are being displayed to the end user.

Author

Ryan McBride

10.1.2 Function Documentation

10.1.2.1 drawCircle()

```
juce::Path drawCircle (
            float x,
            float y,
            float xRadius,
            float yRadius )
```

A simpified function for drawing a circle using the juce framework reducing the number of parameters compared to the ellipse function in juce.

Parameters

x	The centre x position of the circle.
y	The centre y position of the circle.
xRadius	The radius of the circle.
yRadius	The radius of the circle.

Note

The reason there is an x and y radius is to compare which is smaller and set the circles radius to that size to stop the circle from being out of bounds or being distorted.

10.1.2.2 PaintArc()

```
juce::Path PaintArc (
            float x,
            float y,
            float thickness,
            float gap,
            bool direction )
```

A function that traces out the shape of an arc based upon the thickness of the arc and the gap between its inner sides. i.e the inner circle diameter that would remove an area from the larger outer circle.

Parameters

x	The x positon of the path, where this is the left most pixel of the area allowed for the path.
y	The y position of the path, where thes is the top most pixel of the area allowed by the path.

Parameters

thickness	The thickness of the arc itself.
gap	The inner diameter of the arc itself. The arcs center diameter would be the equivalent of gap + (thickness/2).
direction	A boolean option where 0 represents the arc having its outer face facing right, and 1 represents the arc having its outer face facing left.

Return values

Returns	a juce::Path which traces the perimeter of an arc.

10.1.2.3 PaintTrombone()

```
juce::Path PaintTrombone (
            float x = 300,
            float y = 300,
            float width = 50,
            float height = 20,
            bool incPipes = true )
```

A function which returns a path matching the perimeter of a trombone.

Parameters

x	The x coordinate from which the paths minimum x starts. The default value for this is 300.
y	The y coordinate from which the paths minimum y starts. The default value for this is 300.
width	The maximum width which the path is allowed to enclose. The default value for this is 50.
height	The maximum height which the path is allowed to enclose. The default value for this is 20.
incPipes	A boolean value determining if the trombones pipes should be included within the path or whether only the end of the pipe should be returned. The pipes are included by default.

Return values

Returns	a juce::Path which traces the perimeter of a trombone.

10.1.2.4 RotaryArc()

```
juce::Path RotaryArc (
            float x = 100,
            float y = 100,
            float thickness = 5,
            float width = 50,
            float height = 50,
            float angularRange = 5,
            float rotation = 0 )
```

A function which can draw an arc with a specified thickness and nominal radius.

Parameters

x	The x position of the centre point of the arc. This is set to be 100 by default
y	The y position of the centre point of the arc. This is set to be 100 by default.
thickness	The thickness of the arc which is being drawn. This is set to be 5 by default.
width	The radius in the x axis of the arc. This is set to be 50 by default.
height	the radius in the y axis of the arc. This is set to be 50 by default.
angularRange	This sets the difference in minimum versus maximum angle of the arc which is being drawn. This is measured in radians and is set to be 5 by default.
rotation	This parameter rotates the arc to be facing the orientated the preferred way by the user. This function splits the `angularRange` in half from the up direction and has the left and right arcs of equal length. By default, this parameter is set to 0.

Note

This is built upon using the juce::centredArc function and line functions to create a path. If the user chooses to draw the shape outline as opposed to fill, this will be visible.

10.2 CustomStyles.hpp

Go to the documentation of this file.

```
00001
00009 #pragma once
00010
00011 // #include <juce_gui_extra/juce_gui_extra.h>
00012 #include <stdexcept>
00013 #include <string>
00014 #include "juce_core/system/juce_TargetPlatform.h"
00015 #include "juce_core/juce_core.h"
00016 #include "juce_gui_basics/juce_gui_basics.h"
00017
00021 class VerticalSliderLookAndFeel : public juce::LookAndFeel_V4
00022 {
00023     public:
00031         VerticalSliderLookAndFeel();
00032
00037         virtual ~VerticalSliderLookAndFeel() = default;
00038
00065         virtual void drawLinearSlider(juce::Graphics& g, int x, int y, int width,
00066             int height, float sliderPos, float minSliderPos, float maxSliderPos,
00067             const juce::Slider::SliderStyle style, juce::Slider& slider) override;
00068
00072         void setDialColour(const juce::Colour colour);
00073
00078         juce::Colour getDialColour(void);
00079
00084         void setDialEdgeColour(const juce::Colour& colour);
00085
00090         juce::Colour getDialEdgeColour(void);
00091
00095         void setTickColour(juce::Colour colour);
00096
00101         juce::Colour getTickColour(void);
00102
00106         void setTrackColour(juce::Colour colour);
00107
00112         juce::Colour getTrackColour(void);
00113
00117         void setBackgroundColour(juce::Colour colour);
00118
00123         juce::Colour getBackgroundColour(void);
00124
00128         void setTextColour(juce::Colour colour);
00129
00134         juce::Colour getTextColour(void);
00135
00140         void setTrackWidth(float width);
00141
00146         float getTrackWidth(void);
```

```

        void setTickThickness(float thickness);

        float getTickThickness(void);

        void setTickWidth(float width);

        float getTickWidth(void);

#ifdef USE_CIRCLE_FINGER
        void setFingerRadius(float radius);

        float getFingerRadius(void);
#else
        void setFingerWidth(float width);

        float getFingerWidth(void);

        void setFingerHeight(float height);

        float getFingerHeight(void);

#endif

private:
    /*  Setting the main colour of the knob (dial) of the slider. */
    juce::Colour dialColour           = juce::Colours::white;

    /*  Setting the secondary colour of the knob (dial) of the slider.
        currently, this is for the lines within the knob. */
    juce::Colour dialEdgeColour       = juce::Colours::red;

    /*  Setting the colour of the ticks along the slider. */
    juce::Colour tickColour           = juce::Colours::darkgrey;

    /*  Setting the colour of the track which the knob will be moving
        across. */
    juce::Colour trackColour          = juce::Colours::darkgrey;

    /*  Setting the colour which will fill the bounds of the slider. */
    juce::Colour backgroundColour     = juce::Colours::black;

    /*  Setting the colour of the text which will be beside the slider. */
    juce::Colour textColour           = juce::Colours::white;

    /*          Sizes.       */
    /*  This sets the track width of the slider. */
    float trackWidth          = 5;

    /*  This sets the height of the ticks along the slider. */
    float tickThickness       = 3.0;

    /*  This sets the width of the ticks along the slider. */
    float tickWidth           = 10;

/*  Checking the he slider should use a circular finger or rectangular. */
#ifdef USE_CIRCLE_FINGER
    /*  This sets the radius of the finger of the dial position. */
    float fingerRadius        = 12;
#else
    /*  This sets the width of the finger of the slider. */
    float fingerWidth         = 25;

    /*  This sets the height of the finger of the slider. */
    float fingerHeight        = 10;
#endif
};

class CalibrationSliderLookAndFeel  :  public   juce::LookAndFeel_V4{
    public:
        CalibrationSliderLookAndFeel();

        virtual ~CalibrationSliderLookAndFeel() = default;

        virtual void drawLinearSlider(juce::Graphics& g, int x, int y,
            int width, int height, float sliderPos, float minSliderPos,
            float maxSliderPos, const juce::Slider::SliderStyle style,
            juce::Slider& slider);

        void setEmptyTrackColour(juce::Colour colour);

        juce::Colour getEmptyTrackColour(void);

        void setFullTrackColour(juce::Colour colour);

        juce::Colour getFullTrackColour(void);

```

```
        void setMinThumbColour(juce::Colour colour);

        juce::Colour getMinThumbColour(void);

        void setMaxThumbColour(juce::Colour colour);

        juce::Colour getMaxThumbColour(void);

        void setTrackWidth(float width);

        float getTrackWidth(void);

    private:
        juce::Colour  emptyTrackColour= juce::Colours::darkgrey;
        juce::Colour  fullTrackColour = juce::Colours::gold;
        juce::Colour  thumbOneColour  = juce::Colours::yellow;
        juce::Colour  thumbTwoColour  = juce::Colours::pink;

        float trackWidth        =   5;

};

juce::Path PaintTrombone(float x=300, float y=300, float width=50, float height = 20, bool incPipes =
    true);


juce::Path PaintArc(float x, float y, float thickness, float gap, bool direction);

class NeedleLookAndFeel :   public juce::LookAndFeel_V4
{
    public:
        NeedleLookAndFeel() = default;

        virtual ~NeedleLookAndFeel() = default;

        virtual void drawRotarySlider(juce::Graphics& g, int x, int y, int width,
            int height, float sliderPos, const float rotaryStartAngle,
                const float rotaryEndAngle, juce::Slider& slider) override;

        void setNeedleColour(juce::Colour colour);

        juce::Colour getNeedleColour(void);

    private:
        /*  Setting the colour of the needle/arrow which is being drawn. */
        juce::Colour needleColour =   juce::Colours::black;


};

juce::Path RotaryArc(float x = 100, float y = 100, float thickness = 5, float width = 50,
    float height = 50, float angularRange = 5, float rotation = 0);


juce::Path drawCircle(float x, float y, float xRadius, float yRadius);
```

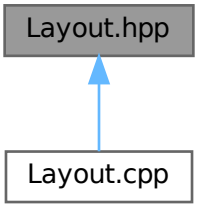
10.3 Layout.hpp File Reference

This file holds the declarations of all functions and variables needed by the Tromboneless project to define the layout of the window which is being displayed to the end user.

```
#include <stdexcept>
#include <string>
#include "Widgets.hpp"
#include "Panels.hpp"
#include "CoreWrapper.hpp"
#include <stdio.h>
```

Include dependency graph for Layout.hpp:

This graph shows which files directly or indirectly include this file:



Classes

- class Layout

  Main class that creates the window.

10.3.1 Detailed Description

This file holds the declarations of all functions and variables needed by the Tromboneless project to define the layout of the window which is being displayed to the end user.

Author

Ryan McBride

10.4 Layout.hpp

Go to the documentation of this file.

```
00001
00009 /*  Preventing recurssive definitons. */
00010 #pragma once
00011
00012 /*  Adding all the necessary header files to be included in the file */
00013 // #include <juce_gui_extra/juce_gui_extra.h>
00014 #include <stdexcept>
00015 #include <string>
00016 #include "Widgets.hpp"
00017 #include "Panels.hpp"
00018 #include "CoreWrapper.hpp"
```

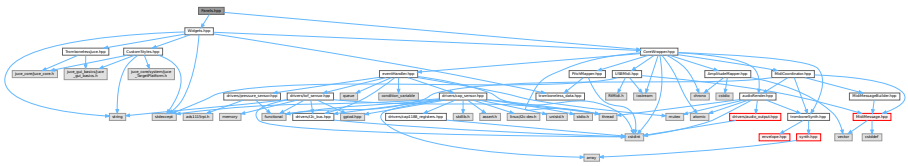
```
#include <stdio.h>

class Layout final : public juce::Component
{
    public:

        // Constructor
        Layout(CoreWrapper& coreWrapper);

        // Deconstructor
        ~Layout() = default;

        void paint (juce::Graphics& g) override;


        void resized() override;



    private:
        /*  Creating the reference to the core*/
        CoreWrapper& coreWrapper_ref;

        /* ========== This is where the buttons and sliders should be initialised and attached to the profiler. ========== */

        /* Shift keying drop down menu and label */
        DropDownMenus   dropDownMenus;

        /* Sliders and labels for calibrating the range. */
        Sliders sliders;

        /* Sliders and labels for calibrating the pressure maximum and minimum. */
        Barometer pressureSlider;

        /* Adding an option to calibrate the embouchure. */
        CalibrateEmbouchure calibrateEmbouchure;


        /* Creation of the equalizer. */
        EqualizerPanel equalizer;

        void showMessageBox (const juce::String& title, const juce::String& message)
        {
            juce::AlertWindow::showMessageBoxAsync(
                juce::MessageBoxIconType::InfoIcon,
                title,
                message
            );
        }

        JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR (Layout)
};


```

10.5 Panels.hpp File Reference

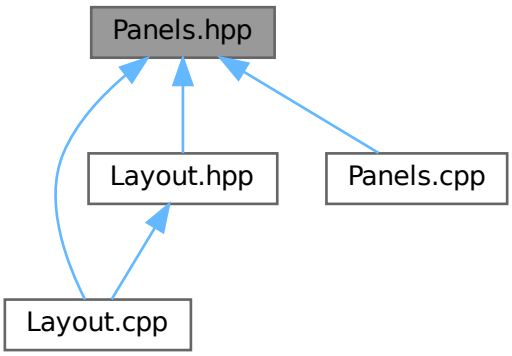
A header file to contain the panels used within the Tromboneless user interface window. This will allow for the encapsulation of different areas as opposed to having large amounts of repeat calls in the `Layout.hpp` file.

```
#include "Widgets.hpp"
#include "CoreWrapper.hpp"
```

Include dependency graph for Panels.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class Panels

  A class which is inherited from Juce:Component reponsible for setting the general style of a panel within the main window.
- class DropDownMenus

  A class which is created to display the drop down menus required by the Tromboneless UI to choose the chosen option for calibration settings.
- class Sliders

  A class which adds all the linear sliders being used within the calibration section to be within the same panel.
- class EqualizerPanel

  A class which may be called to add a panel to the window contains a series of sliders for the equalizer allowing for live editting of the synthesisers sound to ensure it sounds how the Trombonist wishes it to.

10.5.1 Detailed Description

A header file to contain the panels used within the Tromboneless user interface window. This will allow for the encapsulation of different areas as opposed to having large amounts of repeat calls in the `Layout.hpp` file.

Author

Ryan McBride

10.6 Panels.hpp

Go to the documentation of this file.

```
00001
00010     /* Preventing recurssion. */
00011  #pragma once
00012
00013  /* Adding any necessary include files. */
```

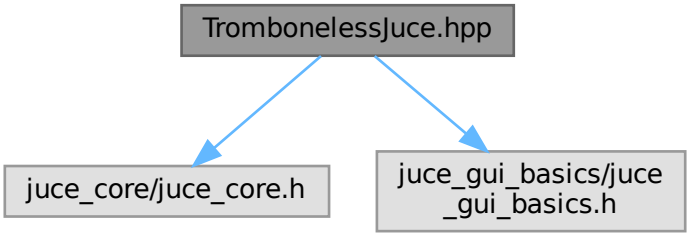
```
#include   "Widgets.hpp"
#include   "CoreWrapper.hpp"

/* Class definitions. */

class Panels   :   public juce::Component
{
   public:
       Panels() = default;

       ~Panels() = default;

       void paint(juce::Graphics& g) override;

       void setBackgroundColour(juce::Colour colour);

       juce::Colour getBackgroundColour(void);

   private:
       /*  Definition of the colour which the panel should be. Currently a dark
           grey colour with a slight transparency.*/
       juce::Colour backgroundColour         = juce::Colour((unsigned int) (0x43080810));

};

class DropDownMenus :   public Panels
{
   public:
       DropDownMenus(CoreWrapper& coreWrapper);

       ~DropDownMenus() = default;

       void resized() override;

       void shiftKeyChoiceChanged();

   private:
       /*  Shift keying drop down menu and label */
       DropDownMenu shiftKeyChoice;

       /*  Creating an instance of the CalibrateEmbouchure class. */
       CalibrateEmbouchure calibrateEmbouchure;
       CoreWrapper& coreWrapper_ref;
};

class Sliders : public Panels,
               public juce::Slider::Listener
{
   public:
       CalibrationSlider distanceSlider;

       Sliders(CoreWrapper& coreWrapper);

       ~Sliders();

       void resized() override;

       void sliderValueChanged(juce::Slider* sliderChanged) override;

       void setMinimumDistance(float distance);

       float getMinimumDistance(void);

       void setMaximumDistance(float distance);

       float getMaximumDistance(void);

       void setStepDistance(float step);

       float getStepDistance(void);

       void setDistanceRange(float range);

       float getDistanceRange(void);

   private:
       /*  Creating an instance of the CalibrationSliderLookAndFeel
           class which will later be applied to the slider to look
           like a trombone being actuated to its full length. */
       CalibrationSliderLookAndFeel LandF;


       /*  Sets the minimum distance the users hand may be from the Tromboneless for it to
           be detected and counted as a position on the pitch bend. */
       float minimumDistance = 5;
       /*  Sets the maxmimum distance the users hand may be from the Tromboneless for it to
           be detected and counted as a position on the pitch bend. */
```

```
        float maximumDistance = 60;
        /*  Sets the minimum change in distance the user can have when adjusting the slider.
            This is essentially the resolution of the slider. */
        float stepDistance = 0.1;
        /*  Setting the minimum distance that the maximum and minimum hand positions must be
            from each other. This reduces the chances of having ambiguity in the pitch bend. */
        float distanceRange = 5;
         CoreWrapper& coreWrapper_ref;
};

 class EqualizerPanel   :   public Panels,
                            public juce::Button::Listener
 {
    public:
        EqualizerPanel(CoreWrapper& coreWrapper);

        ~EqualizerPanel();

        void resized() override;

        void buttonClicked(juce::Button* buttonClicked) override;

    private:
        /*  Initialising the button which will be used to show or hide the equalizer. */
        juce::ToggleButton  button;
        /*  Initialising the equalizer which will allow the user to interact with the
            gains of the frequency bands for the synthesiser. */
        Equalizer equalizer;
        /*  Initialising the label which will appear beside the enable/disable button
            which will indicate what pressing the button will do. */
        juce::Label buttonLabel;
        CoreWrapper& coreWrapper_ref;

 };
```

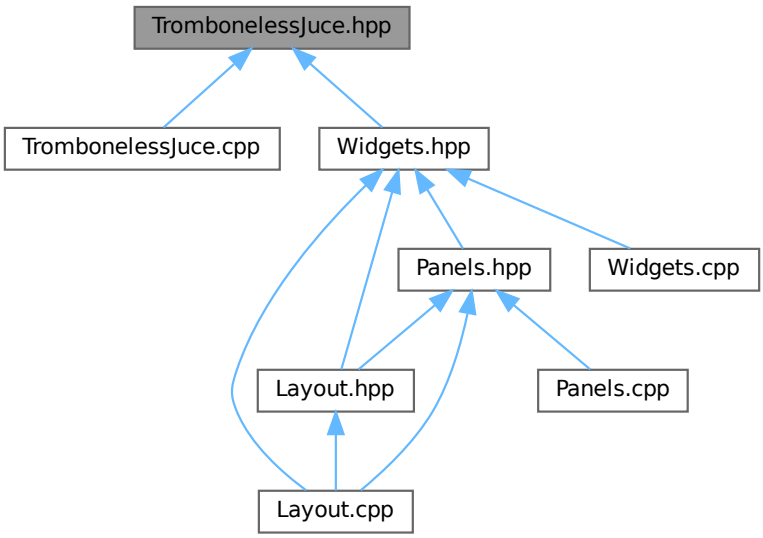
10.7 TrombonelessJuce.hpp File Reference

```
#include <juce_core/juce_core.h>
#include <juce_gui_basics/juce_gui_basics.h>
```

Include dependency graph for TrombonelessJuce.hpp:



This graph shows which files directly or indirectly include this file:



10.7.1 Detailed Description

Author

Ryan McBride

Adds all the necessary header files from Juce which have been used in the Tromboneless App to be made into a library.

10.8 TrombonelessJuce.hpp

Go to the documentation of this file.

```
00001
00007  /* Preventing recurssion. */
00008  #pragma once
00009
00010  #include <juce_core/juce_core.h>
00011  #include <juce_gui_basics/juce_gui_basics.h>
```

10.9 Widgets.hpp File Reference

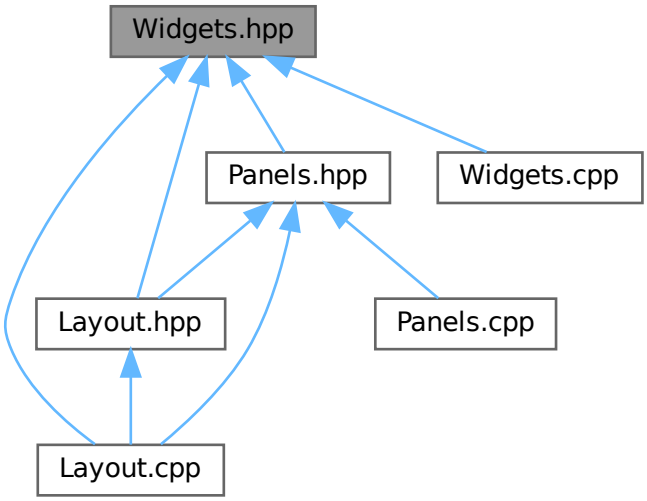
A file which sets the declarations of any custom widgets which have been built off of the juce framework. A large portion of these widgets have been created specifically for the Tromboneless project.

```
#include <stdexcept>
#include <string>
#include <TrombonelessJuce.hpp>
```

```
#include "CustomStyles.hpp"
#include "CoreWrapper.hpp"
#include <tromboneless_data.hpp>
```

Include dependency graph for Widgets.hpp:

This graph shows which files directly or indirectly include this file:



Classes

- class SliderWithLabel

  A class which inherits attributes from the juce::Component class to create a new component containing a 8 possible label locations around the slider.
- class CalibrationSlider

  A class that adds more functionality to the slider class from Juce.
- class DualRotarySlider

  A class that adds more functionality to the slider class from Juce.
- class Barometer

  A class which can allow for the look and feel of a set of rotary sliders be modified to look like a barometer (pressure gauge).
- class verticalMixSlider

  A class that adds more functionality to the slider class from Juce.
- class Equalizer

  A class which inherits its sliders from `verticalMixSlider` made to act as an equalizer.
- class DropDownMenu

A class that creates a drop down menu or 'comboBox' with a label attached to it. This takes up 95% of the available local area. The label is set to have a maxmimum width of 200, but scales as the window size is reduced.

- class CalibrationOnClick

  A class which allows for the additon of a button to the screen without inheriting the button directly. This allows for other widgets to be added to it later without compatibility issues.

- class CalibrateEmbouchure

  A class which is inherited from the CalibrationOnClick class that adds a dropdown menu next to the button for the user to select which option they wish to calibrate.

10.9.1 Detailed Description

A file which sets the declarations of any custom widgets which have been built off of the juce framework. A large portion of these widgets have been created specifically for the Tromboneless project.

Author

Ryan McBride

10.10 Widgets.hpp

Go to the documentation of this file.

```
00001
00010 #pragma once
00011
00012 // #include <juce_gui_extra/juce_gui_extra.h>
00013 #include <stdexcept>
00014 #include <string>
00015 #include <TromboneslessJuce.hpp>
00016 #include "CustomStyles.hpp"
00017 #include "CoreWrapper.hpp"
00018 #include <tromboneless_data.hpp>
00019
00020 /* A header file containing custom widgets to appear on the screen to reduce and repetitive
      definitions within the main Layout.cpp file. */
00021
00025 class SliderWithLabel : public juce::Component{
00026     public:
00027
00029         juce::Slider slider;
00030
00032         SliderWithLabel(juce::Slider::SliderStyle style){
00033             slider.setSliderStyle(style);
00034             slider.setTextBoxStyle(juce::Slider::TextEntryBoxPosition::NoTextBox, true, 0, 0);
00035             addAndMakeVisible(slider);
00036         }
00037
00039         typedef enum{
00040             UpperLeft, UpperCentre, UpperRight,
00041             MiddleLeft,              MiddleRight,
00042             LowerLeft, LowerCentre, LowerRight,
00043         } LabelPositions_t;
00044
00046         typedef enum{
00047             LabelOK,
00048             PositonNotAccepted,
00049         } InitialiseLabelErrors_t;
00050
00064         InitialiseLabelErrors_t CreateLabel(LabelPositions_t position,
00065             juce::String phrase,
00066             juce::NotificationType NotificationEnable = juce::dontSendNotification);
00067
00079         void resized() override;
00080
00086         void setTopLabelBounds(int bound);
00087
00092         int getTopLabelBounds(void);
00093
00099         void setLeftLabelBounds(int bound);
00100
00105         int getLeftLabelBounds(void);
```

```
        void setRightLabelBounds(int bound);

        int getRightLabelBounds(void);

        void setBottomLabelBounds(int bound);

        int getBottomLabelBounds(void);

    private:

        void labelSetup(juce::Label* Label, juce::String phrase,
                        juce::NotificationType NotificationEnable);

        /* Top label bound parameter (maximum height of the above the slider). */
        int topLabelBounds = 50;
        /* Left label bound parameter (maximum width of the label to the left of the slider). */
        int leftLabelBounds = 200;
        /* Right label bound parameter (maximum width of the label to the right of the slider). */
        int rightLabelBounds = 200;
        /* Bottom label bound parameter (maximum height of the label under the slider). */
        int bottomLabelBounds = 50;

        /* Creation of labels that may be used with the slider. */
        juce::Label UpperLeftLabel;
        juce::Label UpperCentreLabel;
        juce::Label UpperRightLabel;
        juce::Label MiddleLeftLabel;
        juce::Label MiddleRightLabel;
        juce::Label LowerLeftLabel;
        juce::Label LowerCentreLabel;
        juce::Label LowerRightLabel;
};


class CalibrationSlider :   public SliderWithLabel,
                            public juce::Slider::Listener
{
public:
    CalibrationSlider();

    ~CalibrationSlider();

    void setMinDifference(float difference);

    float getMinDifference(void);


    void sliderValueChanged(juce::Slider* sliderChanged) override;

    private:
        float minDistance = 1.0;
};

class DualRotarySlider : public juce::Component,
                         public juce::Slider::Listener
{
public:

    juce::Slider    maxSlider;
    juce::Slider    minSlider;
    juce::Label     rotaryLabel;

    DualRotarySlider();

    ~DualRotarySlider();

    void sliderValueChanged(juce::Slider* sliderChanged) override;

    void resized() override;

    void setMinDifference(float difference);

    float getMinDifference(void);

    void setMinSliderRadius(float radius);

    float getMinSliderRadius(void);

    void setMaxSliderRadius(float radius);

    float getMaxSliderRadius(void);

    void setLabelHeight(float height);

    float getLabelHeight(void);

```

```
    void setMinLimit(float limit);

    float getMinLimit(void);

    void setMaxLimit(float limit);

    float getMaxLimit(void);

    void setInterval(float step);

    float getInterval(void);

    void setMinAngle(float angle);

    float getMinAngle(void);

    void setMaxAngle(float angle);

    float getMaxAngle(void);

private:

    /* Setting the relative radius of the minimum slider. */
    float minSliderRadius =   0.6;

    /* Setting the relative radius of the maximum slider. */
    float maxSliderRadius =   0.9;

    /*  Setting the minimum limit of the slider. */
    float minLimit = 0.1;
    /*  Setting the maximum limit of the slider. */
    float maxLimit = 2.5;

    /*  Setting the minimum step to which the slider may be moved. */
    float interval = 0.05;

    /* This is the minimum difference between the min and max sliders with respect ot the non
    normalised values. */
    float minDifference = 0.01;

    /* Setting the relative height of the label. */
    float labelHeight;

    /*  These values should be moved to be private and set/get functions added. */
    /*  Setting the minimum angle which the slider can reach. */
    float minAngle      =   M_PI + 0.5;
    /*  Setting the maximum angle which the slider can reach. */
    float maxAngle      =   3*M_PI - 0.5;

    /* Finding the minimum difference in angle for the finger differences by
       normalising the minDifference with respect to the max and min limits
       and then mulitplying it by the difference betweent the max and min
       angles. */
    float minAngleDifference = (maxAngle - minAngle)*(minDifference/(maxLimit - minLimit));

    void updateMinAngleDifference(void);
};

class Barometer :   public DualRotarySlider
{
    public:
        Barometer(CoreWrapper& coreWrapper);

        ~Barometer();

        void sliderValueChanged(juce::Slider* sliderChanged) override;

        void paint (juce::Graphics& g) override;

        void setBackgroundColour(juce::Colour colour);

        juce::Colour getBackgroundColour(void);

        void setTextColour(juce::Colour colour);

        juce::Colour getTextColour(void);

        void setBoarderColour(juce::Colour colour);

        juce::Colour getBoarderColour(void);

        void setEdgeColour(juce::Colour colour);

        juce::Colour getEdgeColour(void);

        void setOuterRadius(float radius);

```

```
        float getOuterRadius(void);

        void setInnerRadius(float radius);

        float getInnerRadius(void);

        void setLabelRadius(float radius);

        float getLabelRadius(void);

    private:
        /*  Creation of an instance of the NeedleLookAndFeel class to be
            added to the slider knobs to have the correct aesthetic. */
        NeedleLookAndFeel arrow;

        /* Defining the colours used within the barometer. */
        juce::Colour backgroundColour   = juce::Colours::white;
        juce::Colour textColour         = juce::Colours::black;
        juce::Colour boarderColour      = juce::Colours::gold;
        juce::Colour edgeColour         = juce::Colour((unsigned int) (0x43080810));

        /* Dimensions */
        /* Setting the size of the outer arc on the barometer. */
        float relativeOuterRadius = 0.4;

        /* Setting the size of the inner arc on the barometer.*/
        float relativeInnerRadius = 0.35;

        /* Setting the distance of the gain labels relative to the centre point. */
        float relativeLabelRadius = 0.3;

        /*  Pointer to the core class functions. */
        CoreWrapper& coreWrapper_ref;
};

class verticalMixSlider : public SliderWithLabel,
                          public juce::Slider::Listener
{
public:
    verticalMixSlider();

    ~verticalMixSlider();

    void sliderValueChanged(juce::Slider* sliderChanged) override;

};


class Equalizer : public verticalMixSlider
{
    public:

        juce::OwnedArray <verticalMixSlider> eqSliders;

        Equalizer(char numberOfSliders = 10);

        ~Equalizer();

        void resized() override;

        void sliderValueChanged(juce::Slider* sliderChanged) override;

    private:
        /*  Creation of an array which stores the different labels which will be set
            as the frequency ranges of the sliders. */
        juce::Array <juce::String> frequencyLabels;

        /*  Stores the previous label which is being written to the bottom of the
            slider. i.e. the minimum frequency in the frequency band which is being
            adjusted by the slider. */
        float prevLabel;

        /*  Stores the current label which is being written to the top of the slider.
            i.e the maximum frequency in the frequency band which is being adjusted
            by the slider. */
        float newLabel;

        /*  Creating an instance of the VerticalSliderLookAndFeel class to apply a new
            look and feel to the sliders within this class to match that of a standard
            equalizer. */
        VerticalSliderLookAndFeel customLook;

        /* variable to store the number of sliders which are being used. */
        char nSliders;

};

```

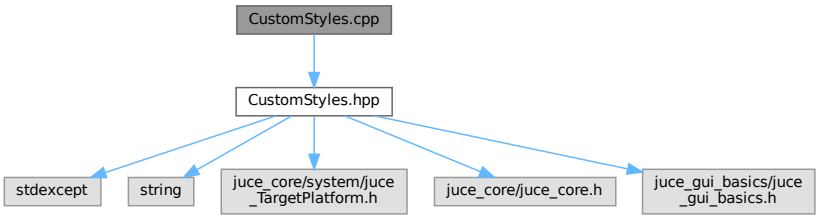
```

 class DropDownMenu :   public juce::Component
 {
     public:

         int getSelectedId() const{
             return dropDownChoice.getSelectedId();
         }

         template <typename Fn>
         void SetOnChange(Fn&& callback){
             dropDownChoice.onChange = std::forward<Fn>(callback);
         }

         DropDownMenu(juce::String labelPhrase = "ComboBox");

         ~DropDownMenu() = default;

         void resized() override;

         void AddItem(juce::String itemName, int itemValue);

         template <typename T> void OnChange(T* updateVariable){
             dropDownChoice.onChange = [updateVariable, this] {*updateVariable = static_cast<T>
(dropDownChoice.getSelectedId());};
         }

         void ChangeLabelText(juce::String text, juce::NotificationType notification =
juce::dontSendNotification);

         juce::String getLabelText(void);

     private:
         /* Adding the label which will be next to the drop down menu. */
         juce::Label dropDownLabel;

         /* Adding the drop down menu to the class. */
         juce::ComboBox dropDownChoice;
 };

 class CalibrationOnClick   :   public juce::Component,
                                public juce::Button::Listener
 {
     public:

         CalibrationOnClick();

         ~CalibrationOnClick();

         void resized() override;

         juce::TextButton button;

 };

class CalibrateEmbouchure    :   public CalibrationOnClick
{
    public:
        CalibrateEmbouchure();

        ~CalibrateEmbouchure() = default;

        void resized() override;

        void buttonClicked(juce::Button* thisButtonClicked) override;

    private:
        /*  Adding a dropdown menu next to the calibration button to select
            the embachure option which is being calibrated. */
        DropDownMenu embouchureChoice;

        /*  Adding a tootltipWindow to the class which may be displayed when
            the user runs their cursor over the button for at least 300ms. */
        juce::TooltipWindow tooltipWindow;

};

```

10.11 CustomStyles.cpp File Reference

This file holds the defintions of all functions and variables needed by the Tromboneless project to change the looks and feel of various different widgets which are being displayed to the end user.

```
#include "CustomStyles.hpp"
```

Include dependency graph for CustomStyles.cpp:



Functions

- juce::Path PaintTrombone (float x, float y, float width, float height, bool incPipes)

  A function which returns a path matching the perimeter of a trombone.
- juce::Path PaintArc (float x, float y, float thickness, float gap, bool direction)

  A function that traces out the shape of an arc based upon the thickness of the arc and the gap between its inner sides. i.e the inner circle diameter that would remove an area from the larger outer circle.
- juce::Path RotaryArc (float x, float y, float thickness, float width, float height, float angularRange, float rotation)

  A function which can draw an arc with a specified thickness and nominal radius.
- juce::Path drawCircle (float x, float y, float xRadius, float yRadius)

  A simpified function for drawing a circle using the juce framework reducing the number of parameters compared to the ellipse function in juce.

10.11.1 Detailed Description

This file holds the defintions of all functions and variables needed by the Tromboneless project to change the looks and feel of various different widgets which are being displayed to the end user.

Author

Ryan McBride

10.11.2 Function Documentation

10.11.2.1 drawCircle()

```
juce::Path drawCircle (
            float x,
            float y,
            float xRadius,
            float yRadius )
```

A simpified function for drawing a circle using the juce framework reducing the number of parameters compared to the ellipse function in juce.

Parameters

x	The centre x position of the circle.
y	The centre y position of the circle.
xRadius	The radius of the circle.
yRadius	The radius of the circle.

Note

The reason there is an x and y radius is to compare which is smaller and set the circles radius to that size to stop the circle from being out of bounds or being distorted.

10.11.2.2 PaintArc()

```
juce::Path PaintArc (
            float x,
            float y,
            float thickness,
            float gap,
            bool direction )
```

A function that traces out the shape of an arc based upon the thickness of the arc and the gap between its inner sides. i.e the inner circle diameter that would remove an area from the larger outer circle.

Parameters

x	The x positon of the path, where this is the left most pixel of the area allowed for the path.
y	The y position of the path, where thes is the top most pixel of the area allowed by the path.
thickness	The thickness of the arc itself.
gap	The inner diameter of the arc itself. The arcs center diameter would be the equivalent of gap + (thickness/2).
direction	A boolean option where 0 represents the arc having its outer face facing right, and 1 represents the arc having its outer face facing left.

Return values

Returns	a juce::Path which traces the perimeter of an arc.

10.11.2.3 PaintTrombone()

```
juce::Path PaintTrombone (
            float x = 300,
            float y = 300,
            float width = 50,
            float height = 20,
            bool incPipes = true )
```

A function which returns a path matching the perimeter of a trombone.

Parameters

x	The x coordinate from which the paths minimum x starts. The default value for this is 300.
y	The y coordinate from which the paths minimum y starts. The default value for this is 300.
width	The maximum width which the path is allowed to enclose. The default value for this is 50.
height	The maximum height which the path is allowed to enclose. The default value for this is 20.
incPipes	A boolean value determining if the trombones pipes should be included within the path or whether only the end of the pipe should be returned. The pipes are included by default.

Return values

Returns	a juce::Path which traces the perimeter of a trombone.

10.11.2.4 RotaryArc()

```
juce::Path RotaryArc (
            float x = 100,
            float y = 100,
            float thickness = 5,
            float width = 50,
            float height = 50,
            float angularRange = 5,
            float rotation = 0 )
```

A function which can draw an arc with a specified thickness and nominal radius.

Parameters

x	The x position of the centre point of the arc. This is set to be 100 by default
y	The y position of the centre point of the arc. This is set to be 100 by default.
thickness	The thickness of the arc which is being drawn. This is set to be 5 by default.
width	The radius in the x axis of the arc. This is set to be 50 by default.
height	the radius in the y axis of the arc. This is set to be 50 by default.
angularRange	This sets the difference in minimum versus maximum angle of the arc which is being drawn. This is measured in radians and is set to be 5 by default.
rotation	This parameter rotates the arc to be facing the orientated the preferred way by the user. This function splits the `angularRange` in half from the up direction and has the left and right arcs of equal length. By default, this parameter is set to 0.

Note

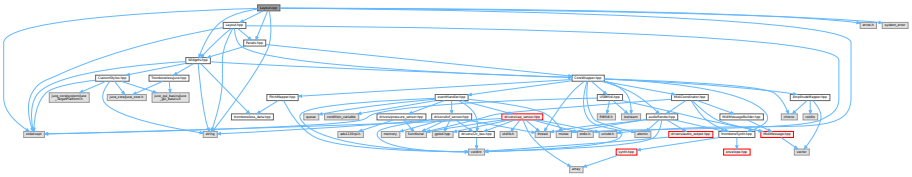
This is built upon using the juce::centredArc function and line functions to create a path. If the user chooses to draw the shape outline as opposed to fill, this will be visible.

10.12 Layout.cpp File Reference

This file holds the defintions of all functions and variables needed by the Tromboneless project to define the layout of the window which is being displayed to the end user.

```
#include "Layout.hpp"
#include "Widgets.hpp"
#include "Panels.hpp"
#include <errno.h>
#include <unistd.h>
#include <stdexcept>
#include <string>
#include <system_error>
```

Include dependency graph for Layout.cpp:



10.12.1 Detailed Description

This file holds the defintions of all functions and variables needed by the Tromboneless project to define the layout of the window which is being displayed to the end user.

Author

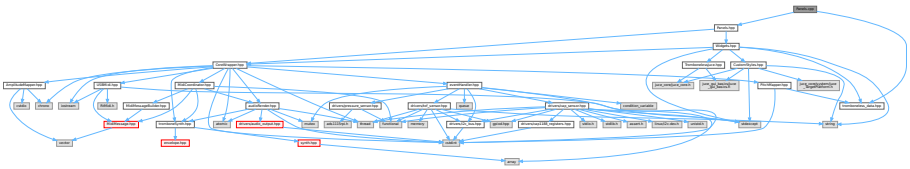
Ryan McBride

10.13 Panels.cpp File Reference

This file contains the function defintions for to create the necessary panels that may be used within the Tromboneless user interface.

```
#include "Panels.hpp"
#include <tromboneless_data.hpp>
```

Include dependency graph for Panels.cpp:



10.13.1 Detailed Description

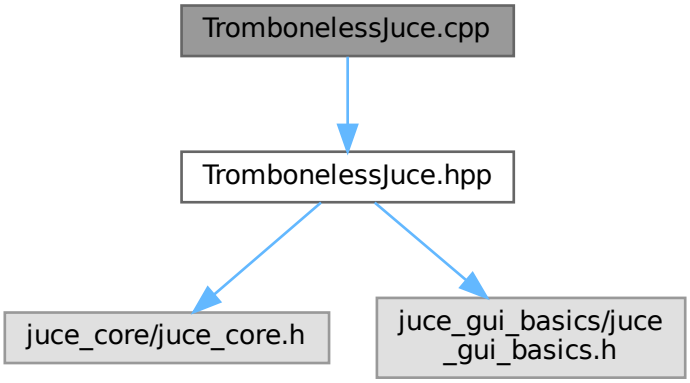
This file contains the function defintions for to create the necessary panels that may be used within the Tromboneless user interface.

Author

Ryan McBride

10.14 TrombonelessJuce.cpp File Reference

`#include "TrombonelessJuce.hpp"`
Include dependency graph for TrombonelessJuce.cpp:



10.14.1 Detailed Description

Author

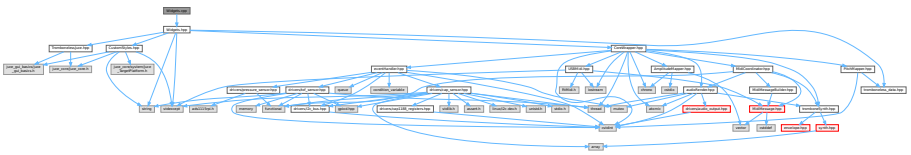
Ryan McBride

Adds all the necessary header files from Juce which have been used in the Tromboneless App to be made into a library.

10.15 Widgets.cpp File Reference

A file which defines the functions which have been declared within the `Widgets.hpp` file.

`#include "Widgets.hpp"`
Include dependency graph for Widgets.cpp:



10.15.1 Detailed Description

A file which defines the functions which have been declared within the `Widgets.hpp` file.

Author

Ryan McBride

10.16 AmplitudeMapper.hpp

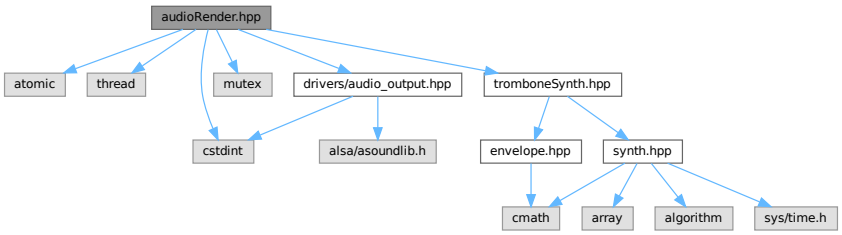
```
#pragma once
#include <cstdio>
#include <vector>
#include <chrono>

// This code defines a class that takes a voltage level sample from
// ADS1115 representing breath pressure
// and compares it against a baseline to determine whether it
// sufficiently high enough to switch note on.

class AmplitudeMapper {

    private:
        // Attributes
        float pressureSample = 0.0f;
        float pressureBaseline = 0.0f;
        float pressureThreshold;
        float gateFactor = 1.0f;
        bool pressureState = false;
        std::vector<float> buffer;
        std::chrono::steady_clock::time_point startTime;

    public:
        AmplitudeMapper();
        bool calculateBaseline(float pressureSample);
        float getBaseline();
        bool noteEdge(float pressureSample);

        float getGateFactor(void);

        void setGateFactor(float threshold);


    };
```

10.17 audioRender.hpp File Reference

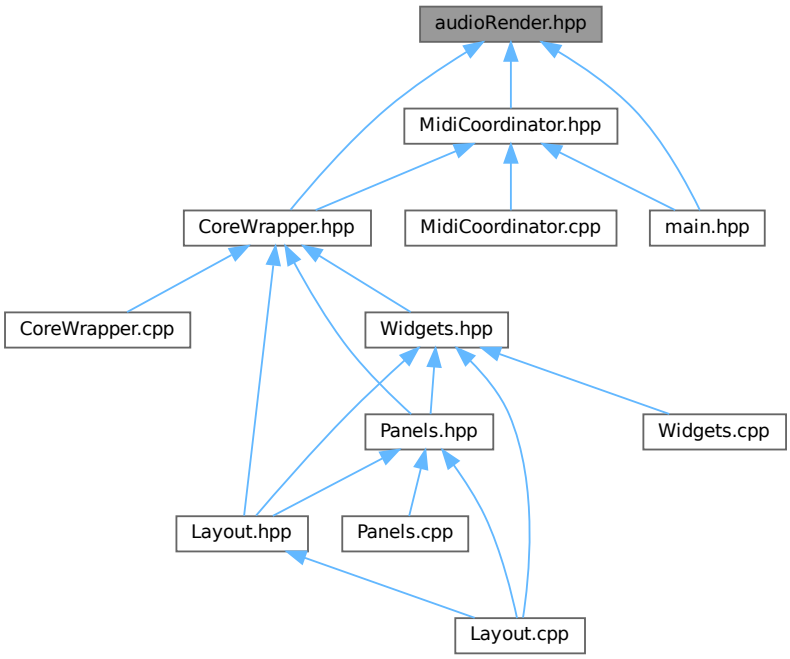
Declares the AudioRender class, responsible for managing the real-time audio rendering pipeline and interfacing with the synthesis engine.

```
#include <atomic>
#include <thread>
#include <cstdint>
#include <mutex>
#include "tromboneSynth.hpp"
#include "drivers/audio_output.hpp"
```

Include dependency graph for audioRender.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class AudioRender

  Handles real-time audio generation and output.

10.17.1 Detailed Description

Declares the AudioRender class, responsible for managing the real-time audio rendering pipeline and interfacing with the synthesis engine.

Author

Aidan McIntosh

Date

2026-04-20

10.18 audioRender.hpp

Go to the documentation of this file.

```
#pragma once

#include <atomic>
#include <thread>
#include <cstdint>
#include <mutex>

#include "tromboneSynth.hpp"
#include "drivers/audio_output.hpp"

class AudioRender : public TromboneSynth {
public:
    static constexpr int frames = 128;

    static constexpr int sample_rate = 44100;

    AudioRender(bool isTest);

    ~AudioRender();

    AudioRender(const AudioRender&) = delete;
    AudioRender& operator=(const AudioRender&) = delete;

    void start();

    void stop();

    bool isRunning() const { return running.load(); }

    void noteOnFromMidi(int midiNote);

    void noteOffFromMidi();

    void noteChangeFromMidi(int midiNote);

    void pitchBendFromMidi(int bend);

    void setDebugTone(bool enabled);

private:
    void renderLoop();

    void renderReleaseTail();

    AudioOutput output;

    std::thread renderThread;

    std::atomic<bool> running{false};

    std::atomic<bool> stopRequested{false};

    std::atomic<bool> debugToneEnabled{false};

    std::mutex synthMutex;

    int16_t buffer[frames]{0};

    float debugPhase{0.0f};

    bool isTestMode = false;
};
```

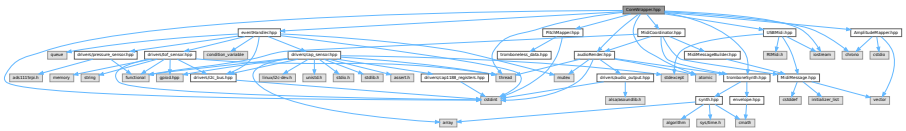
10.19 CoreWrapper.hpp File Reference

A file to allow for the app to view methods attached to the main functioanlity of the core.

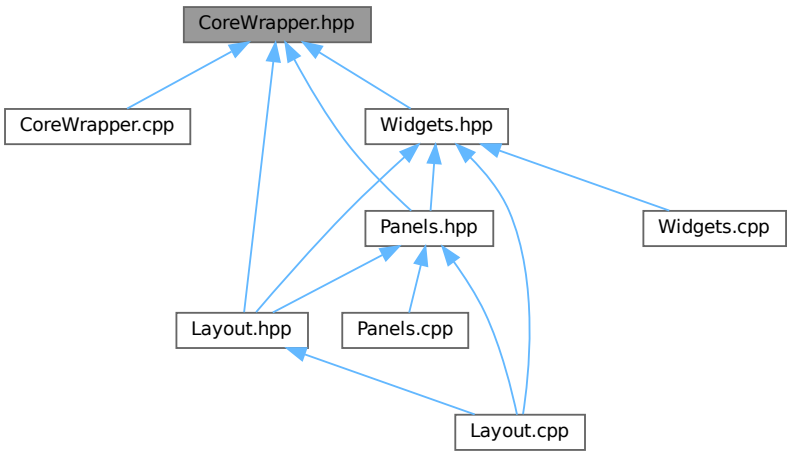
```
#include "eventHandler.hpp"
#include <cstdint>
#include <iostream>
#include <thread>
#include <chrono>
```

```
#include <atomic>
#include "MidiCoordinator.hpp"
#include "tromboneSynth.hpp"
#include "USBMidi.hpp"
#include "audioRender.hpp"
#include "PitchMapper.hpp"
#include "AmplitudeMapper.hpp"
```

Include dependency graph for CoreWrapper.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class CoreWrapper

10.19.1 Detailed Description

A file to allow for the app to view methods attached to the main functioanlity of the core.

Author

Ryan McBride Aidan McIntosh

10.20 CoreWrapper.hpp

Go to the documentation of this file.

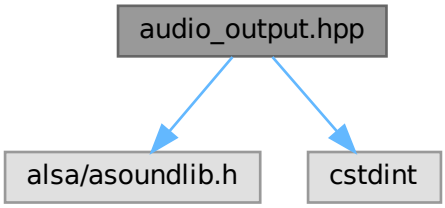
```
#pragma once

#include "eventHandler.hpp"
#include <cstdint>
#include <iostream>
#include <thread>
#include <chrono>
#include <atomic>

#include "MidiCoordinator.hpp"
#include "tromboneSynth.hpp"
#include "USBMidi.hpp"
#include "audioRender.hpp"
#include "PitchMapper.hpp"
#include "AmplitudeMapper.hpp"

class CoreWrapper{
    public:
        /*  ========================================  */
        /*  Adding the constructor and destructor.      */
        /*  ========================================  */

        CoreWrapper(bool isTest);
        ~CoreWrapper();

        /*  ========================================  */
        /*  Adding functions to start the threads.      */
        /*  ========================================  */
        void start();
        void stop();


        EventHandler* getEventHandler(void);
        RtMidiSink* getRtMidiSink(void);
        AudioRender* getAudioRender(void);
        AmplitudeMapper* getAmplitudeMapper(void);
        PitchMapper* getPitchMapper(void);
        MidiCoordinator* getMidiCoordinator(void);

        /*  ========================================  */
        /* Adding the getters for the classes' instances */
        /*  ========================================  */

    private:
        /*  Creating the instances. */

        EventHandler eventHandler;
        RtMidiSink midiSink;
        AudioRender render;
        AmplitudeMapper amplitudemapper;
        PitchMapper pitchmapper;
        MidiCoordinator coordinator;
        /*  A flag stored to inform the methods if it should ignore
            all hardware dependent functions. */
        bool isTestMode = false;

        /*  Core loop. */
        std::thread eventThread;
        std::atomic<bool> running{false};
        void eventLoop();
        bool externalDevicePresent = false;
};
```

10.21 audio_output.hpp File Reference

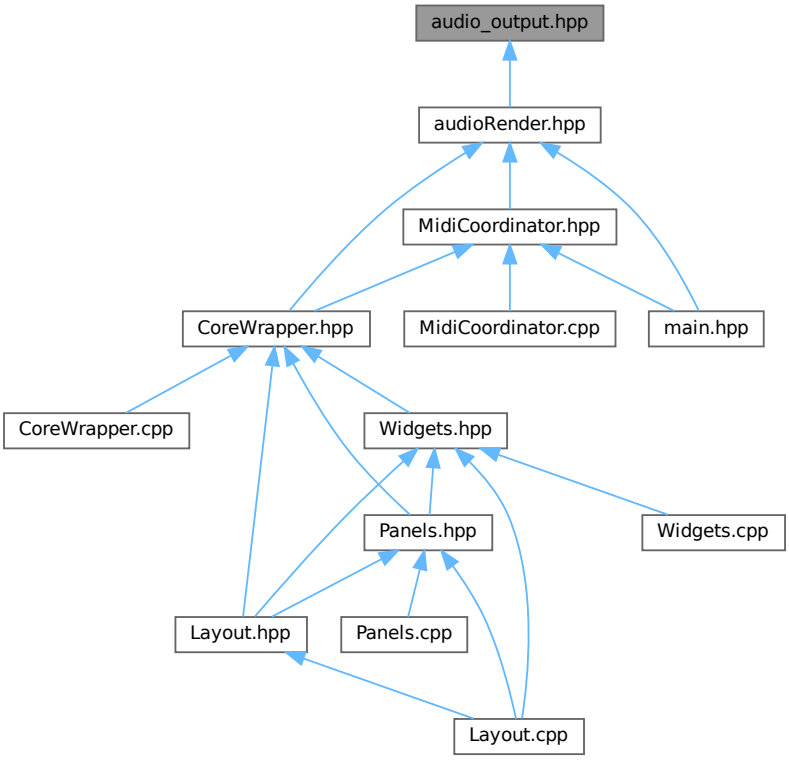
Declares the AudioOutput class, responsible for interfacing with the ALSA audio subsystem for playback.

```
#include <alsa/asoundlib.h>
#include <cstdint>
```

Include dependency graph for audio_output.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class AudioOutput

  Provides a low-level interface for audio playback via ALSA.

10.21.1 Detailed Description

Declares the AudioOutput class, responsible for interfacing with the ALSA audio subsystem for playback.

Author

Aidan McIntosh

Date

2026-04-20

10.22 audio_output.hpp

Go to the documentation of this file.

```
#pragma once

#include <alsa/asoundlib.h>
#include <cstdint>

class AudioOutput {
public:
    AudioOutput(bool isTest);

    ~AudioOutput();

    void writeSamples(const int16_t* buffer, int frames);

private:
    snd_pcm_t* handle = nullptr;

    bool isTestMode = false;
};
```

10.23 cap1188_registers.hpp

```
#pragma once

#include <cstdint>

// ============================================================
//  CAP1188 - 8-Channel Capacitive Touch Sensor
//  Raspberry Pi Linux I2C driver (i2c-dev / SMBus)
//
//  Register set sourced from Microchip datasheet DS00001620B
// ============================================================

// ------------------------------------------------------------
//  I2C address (set by AD pin wiring)
//  AD floating / 3V3    -> 0x29 (default)
//  AD -> 390K to GND    -> 0x28
//  AD -> 100K to GND    -> 0x2A
//  AD -> 680K to GND    -> 0x2B
//  AD -> GND            -> SPI mode only, not used here
// ------------------------------------------------------------
static constexpr uint8_t CAP1188_DEFAULT_ADDR   = 0x29;
static constexpr uint8_t CAP1188_390K_ADDR      = 0x28;         // Used for Tromboneless

// ------------------------------------------------------------
//  General Registers
// ------------------------------------------------------------
static constexpr uint8_t CAP1188_REG_MAIN_CTRL          = 0x00; // R/W  Main Control             - Controls general power states and power dissipation         Default: 0x00
static constexpr uint8_t CAP1188_REG_GENERAL_STATUS     = 0x02; // R    General Status            - Stores general status bits                                  Default: 0x00
static constexpr uint8_t CAP1188_REG_SENSOR_STATUS      = 0x03; // R    Sensor Input Status       - State of sampled capacitive touch sensor inputs             Default: 0x00
static constexpr uint8_t CAP1188_REG_LED_STATUS         = 0x04; // R    LED Status                - Stores status bits for LEDs                                 Default: 0x00
```

```
static constexpr uint8_t CAP1188_REG_NOISE_FLAG_STATUS  = 0x0A; // R    Noise Flag Status         - Stores the noise flags for sensor inputs                    Default: 0x00

// ------------------------------------------------------------
//  Delta Count Registers (read-only, one per channel)
//  Stores the difference between current count and base count.
//  A rising value indicates a touch. Compared against threshold
//  to determine if a touch event is flagged.
// ------------------------------------------------------------
static constexpr uint8_t CAP1188_REG_DELTA_COUNT_CS1    = 0x10; // R    Sensor Input 1 Delta Count - Delta count for CS1                                Default: 0x00
static constexpr uint8_t CAP1188_REG_DELTA_COUNT_CS2    = 0x11; // R    Sensor Input 2 Delta Count - Delta count for CS2                                Default: 0x00
static constexpr uint8_t CAP1188_REG_DELTA_COUNT_CS3    = 0x12; // R    Sensor Input 3 Delta Count - Delta count for CS3                                Default: 0x00
static constexpr uint8_t CAP1188_REG_DELTA_COUNT_CS4    = 0x13; // R    Sensor Input 4 Delta Count - Delta count for CS4                                Default: 0x00
static constexpr uint8_t CAP1188_REG_DELTA_COUNT_CS5    = 0x14; // R    Sensor Input 5 Delta Count - Delta count for CS5                                Default: 0x00
static constexpr uint8_t CAP1188_REG_DELTA_COUNT_CS6    = 0x15; // R    Sensor Input 6 Delta Count - Delta count for CS6                                Default: 0x00
static constexpr uint8_t CAP1188_REG_DELTA_COUNT_CS7    = 0x16; // R    Sensor Input 7 Delta Count - Delta count for CS7                                Default: 0x00
static constexpr uint8_t CAP1188_REG_DELTA_COUNT_CS8    = 0x17; // R    Sensor Input 8 Delta Count - Delta count for CS8                                Default: 0x00

// ------------------------------------------------------------
//  Sensitivity & Configuration Registers
// ------------------------------------------------------------
static constexpr uint8_t CAP1188_REG_SENSITIVITY        = 0x1F; // R/W  Sensitivity Control          - Controls sensitivity of threshold/delta counts and base count scaling  Default: 0x2F
static constexpr uint8_t CAP1188_REG_CONFIG             = 0x20; // R/W  Configuration                - Controls general functionality                          Default: 0x20
static constexpr uint8_t CAP1188_REG_SENSOR_ENABLE      = 0x21; // R/W  Sensor Input Enable          - Controls which capacitive touch sensor inputs are sampled              Default: 0xFF
static constexpr uint8_t CAP1188_REG_SENSOR_CFG1        = 0x22; // R/W  Sensor Input Configuration   - Controls max duration and auto-repeat delay (full power state)         Default: 0xA4
static constexpr uint8_t CAP1188_REG_SENSOR_CFG2        = 0x23; // R/W  Sensor Input Configuration 2 - Controls MPRESS controls for all sensor inputs          Default: 0x07
static constexpr uint8_t CAP1188_REG_AVG_SAMP_RATE      = 0x24; // R/W  Averaging and Sampling Config - Controls averaging and sampling window                 Default: 0x39
static constexpr uint8_t CAP1188_REG_CALIB_ACTIVATE     = 0x26; // R/W  Calibration Activate         - Forces re-calibration for capacitive touch sensor inputs               Default: 0x00
static constexpr uint8_t CAP1188_REG_INTERRUPT_ENABLE   = 0x27; // R/W  Interrupt Enable             - Enables interrupts associated with capacitive touch sensor inputs      Default: 0xFF
static constexpr uint8_t CAP1188_REG_REPEAT_RATE_ENABLE = 0x28; // R/W  Repeat Rate Enable           - Enables repeat rate for all sensor inputs               Default: 0xFF
static constexpr uint8_t CAP1188_REG_MULTI_TOUCH_CFG    = 0x2A; // R/W  Multiple Touch Configuration - Determines number of simultaneous touches to flag       Default: 0x80
static constexpr uint8_t CAP1188_REG_MULTI_TOUCH_PAT_CFG= 0x2B; // R/W  Multiple Touch Pattern Config- Determines MTP configuration                             Default: 0x00
static constexpr uint8_t CAP1188_REG_MULTI_TOUCH_PAT    = 0x2D; // R/W  Multiple Touch Pattern       - Determines pattern/number of sensor inputs used by MTP  Default: 0xFF
static constexpr uint8_t CAP1188_REG_RECALIB_CFG        = 0x2F; // R/W  Recalibration Configuration  - Determines re-calibration timing and sampling window    Default: 0x8A

// ------------------------------------------------------------
//  Threshold Registers (one per channel)
//  Delta count must exceed this value to register a touch.
//  Default value 0x40 = 64 counts.
// ------------------------------------------------------------
static constexpr uint8_t CAP1188_REG_THRESH_CS1         = 0x30; // R/W  Sensor Input 1 Threshold - Delta count threshold for touch on CS1                    Default: 0x40
static constexpr uint8_t CAP1188_REG_THRESH_CS2         = 0x31; // R/W  Sensor Input 2 Threshold - Delta count threshold for touch on CS2                    Default: 0x40
static constexpr uint8_t CAP1188_REG_THRESH_CS3         = 0x32; // R/W  Sensor Input 3 Threshold - Delta count threshold for touch on CS3                    Default: 0x40
static constexpr uint8_t CAP1188_REG_THRESH_CS4         = 0x33; // R/W  Sensor Input 4 Threshold - Delta count threshold for touch on CS4                    Default: 0x40
static constexpr uint8_t CAP1188_REG_THRESH_CS5         = 0x34; // R/W  Sensor Input 5 Threshold - Delta count threshold for touch on CS5                    Default: 0x40
static constexpr uint8_t CAP1188_REG_THRESH_CS6         = 0x35; // R/W  Sensor Input 6 Threshold - Delta count threshold for touch on CS6                    Default: 0x40
static constexpr uint8_t CAP1188_REG_THRESH_CS7         = 0x36; // R/W  Sensor Input 7 Threshold - Delta count threshold for touch on CS7                    Default: 0x40
static constexpr uint8_t CAP1188_REG_THRESH_CS8         = 0x37; // R/W  Sensor Input 8 Threshold - Delta count threshold for touch on CS8                    Default: 0x40
static constexpr uint8_t CAP1188_REG_NOISE_THRESHOLD    = 0x38; // R/W  Sensor Input Noise Threshold - Controls noise threshold for all sensor inputs          Default: 0x01

// ------------------------------------------------------------
//  Standby Configuration Registers
// ------------------------------------------------------------
static constexpr uint8_t CAP1188_REG_STANDBY_CHANNEL    = 0x40; // R/W  Standby Channel        - Controls which sensor inputs are enabled in standby          Default: 0x00
static constexpr uint8_t CAP1188_REG_STANDBY_CFG        = 0x41; // R/W  Standby Configuration  - Controls averaging and cycle time in standby                 Default: 0x39
```

```
static constexpr uint8_t CAP1188_REG_STANDBY_SENS       = 0x42; // R/W  Standby Sensitivity    - Controls sensitivity settings used in standby                Default: 0x02
static constexpr uint8_t CAP1188_REG_STANDBY_THRESH     = 0x43; // R/W  Standby Threshold      - Touch detection threshold for active inputs in standby       Default: 0x40
static constexpr uint8_t CAP1188_REG_CONFIG2            = 0x44; // R/W  Configuration 2        - Additional configuration controls for the device             Default: 0x40

// ------------------------------------------------------------
//  Base Count Registers (read-only, one per channel)
//  Stores the reference (baseline) capacitance for each pad,
//  established during calibration. Delta count = current - base.
// ------------------------------------------------------------
static constexpr uint8_t CAP1188_REG_BASE_COUNT_CS1     = 0x50; // R    Sensor Input 1 Base Count - Reference count value for sensor input 1                  Default: 0xC8
static constexpr uint8_t CAP1188_REG_BASE_COUNT_CS2     = 0x51; // R    Sensor Input 2 Base Count - Reference count value for sensor input 2                  Default: 0xC8
static constexpr uint8_t CAP1188_REG_BASE_COUNT_CS3     = 0x52; // R    Sensor Input 3 Base Count - Reference count value for sensor input 3                  Default: 0xC8
static constexpr uint8_t CAP1188_REG_BASE_COUNT_CS4     = 0x53; // R    Sensor Input 4 Base Count - Reference count value for sensor input 4                  Default: 0xC8
static constexpr uint8_t CAP1188_REG_BASE_COUNT_CS5     = 0x54; // R    Sensor Input 5 Base Count - Reference count value for sensor input 5                  Default: 0xC8
static constexpr uint8_t CAP1188_REG_BASE_COUNT_CS6     = 0x55; // R    Sensor Input 6 Base Count - Reference count value for sensor input 6                  Default: 0xC8
static constexpr uint8_t CAP1188_REG_BASE_COUNT_CS7     = 0x56; // R    Sensor Input 7 Base Count - Reference count value for sensor input 7                  Default: 0xC8
static constexpr uint8_t CAP1188_REG_BASE_COUNT_CS8     = 0x57; // R    Sensor Input 8 Base Count - Reference count value for sensor input 8                  Default: 0xC8

// ------------------------------------------------------------
//  LED Control Registers
// ------------------------------------------------------------
static constexpr uint8_t CAP1188_REG_LED_OUTPUT_TYPE    = 0x71; // R/W  LED Output Type              - Controls output type for LED outputs                  Default: 0x00
static constexpr uint8_t CAP1188_REG_LED_LINKING        = 0x72; // R/W  Sensor Input LED Linking     - Controls linking of sensor inputs to LED channels     Default: 0x00
static constexpr uint8_t CAP1188_REG_LED_POLARITY       = 0x73; // R/W  LED Polarity                 - Controls output polarity of LEDs                      Default: 0x00
static constexpr uint8_t CAP1188_REG_LED_OUTPUT_CTRL    = 0x74; // R/W  LED Output Control           - Controls the output state of the LEDs                 Default: 0x00
static constexpr uint8_t CAP1188_REG_LED_LINKED_TRANS   = 0x77; // R/W  LED Linked Transition Control - Controls transition when LEDs are linked to CS channels Default: 0x00
static constexpr uint8_t CAP1188_REG_LED_MIRROR_CTRL    = 0x79; // R/W  LED Mirror Control           - Controls mirroring of duty cycles for the LEDs        Default: 0x00
static constexpr uint8_t CAP1188_REG_LED_BEHAVIOUR1     = 0x81; // R/W  LED Behavior 1               - Controls behaviour and response of LEDs 1-4           Default: 0x00
static constexpr uint8_t CAP1188_REG_LED_BEHAVIOUR2     = 0x82; // R/W  LED Behavior 2               - Controls behaviour and response of LEDs 5-8           Default: 0x00
static constexpr uint8_t CAP1188_REG_LED_PULSE1_PERIOD  = 0x84; // R/W  LED Pulse 1 Period           - Controls the period of each breathe during a pulse    Default: 0x20
static constexpr uint8_t CAP1188_REG_LED_PULSE2_PERIOD  = 0x85; // R/W  LED Pulse 2 Period           - Controls the period of breathing during breathe/pulse Default: 0x14
static constexpr uint8_t CAP1188_REG_LED_BREATHE_PERIOD = 0x86; // R/W  LED Breathe Period           - Controls the period of an LED breathe operation       Default: 0x5D
static constexpr uint8_t CAP1188_REG_LED_CONFIG         = 0x88; // R/W  LED Config                   - Controls LED configuration                            Default: 0x04
static constexpr uint8_t CAP1188_REG_LED_PULSE1_DUTY    = 0x90; // R/W  LED Pulse 1 Duty Cycle       - Min and max duty cycle for pulse operation            Default: 0xF0
static constexpr uint8_t CAP1188_REG_LED_PULSE2_DUTY    = 0x91; // R/W  LED Pulse 2 Duty Cycle       - Min and max duty cycle for breathe and pulse operation Default: 0xF0
static constexpr uint8_t CAP1188_REG_LED_BREATHE_DUTY   = 0x92; // R/W  LED Breathe Duty Cycle       - Min and max duty cycle for breathe operation          Default: 0xF0
static constexpr uint8_t CAP1188_REG_LED_DIRECT_DUTY    = 0x93; // R/W  LED Direct Duty Cycle        - Min and max duty cycle for Direct mode LED operation  Default: 0xF0
static constexpr uint8_t CAP1188_REG_LED_DIRECT_RAMP    = 0x94; // R/W  LED Direct Ramp Rates        - Rising and falling edge ramp rates of the LEDs        Default: 0x00
static constexpr uint8_t CAP1188_REG_LED_OFF_DELAY      = 0x95; // R/W  LED Off Delay                - Determines the off delay for all LED behaviours       Default: 0x00

// ------------------------------------------------------------
//  Calibration Registers (read-only, one per channel)
// ------------------------------------------------------------
static constexpr uint8_t CAP1188_REG_CALIB_CS1          = 0xB1; // R    Sensor Input 1 Calibration - Upper 8-bit calibration value for sensor input 1    Default: 0x00
static constexpr uint8_t CAP1188_REG_CALIB_CS2          = 0xB2; // R    Sensor Input 2 Calibration - Upper 8-bit calibration value for sensor input 2    Default: 0x00
static constexpr uint8_t CAP1188_REG_CALIB_CS3          = 0xB3; // R    Sensor Input 3 Calibration - Upper 8-bit calibration value for sensor input 3    Default: 0x00
static constexpr uint8_t CAP1188_REG_CALIB_CS4          = 0xB4; // R    Sensor Input 4 Calibration - Upper 8-bit calibration value for sensor input 4    Default: 0x00
static constexpr uint8_t CAP1188_REG_CALIB_CS5          = 0xB5; // R    Sensor Input 5 Calibration - Upper 8-bit calibration value for sensor input 5    Default: 0x00
static constexpr uint8_t CAP1188_REG_CALIB_CS6          = 0xB6; // R    Sensor Input 6 Calibration - Upper 8-bit calibration value for sensor input 6    Default: 0x00
static constexpr uint8_t CAP1188_REG_CALIB_CS7          = 0xB7; // R    Sensor Input 7 Calibration - Upper 8-bit calibration value for sensor input 7    Default: 0x00
static constexpr uint8_t CAP1188_REG_CALIB_CS8          = 0xB8; // R    Sensor Input 8 Calibration
```

```
- Upper 8-bit calibration value for sensor input 8    Default: 0x00
static constexpr uint8_t CAP1188_REG_CALIB_LSB1         = 0xB9; // R    Sensor Input Calibration LSB 1
- 2 LSBs of calibration value for sensor inputs 1-4  Default: 0x00
static constexpr uint8_t CAP1188_REG_CALIB_LSB2         = 0xBA; // R    Sensor Input Calibration LSB 2
- 2 LSBs of calibration value for sensor inputs 5-8  Default: 0x00

// ------------------------------------------------------------
//  Identification Registers (read-only)
// ------------------------------------------------------------
static constexpr uint8_t CAP1188_REG_PRODUCT_ID         = 0xFD; // R    Product ID      - Fixed value
identifying this product                                Default: 0x50
static constexpr uint8_t CAP1188_REG_MANUFACTURER_ID    = 0xFE; // R    Manufacturer ID - Fixed value
identifying Microchip                                   Default: 0x5D
static constexpr uint8_t CAP1188_REG_REVISION           = 0xFF; // R    Revision        - Fixed value
representing the chip revision                          Default: 0x83

// ------------------------------------------------------------
//  Known identification values
// ------------------------------------------------------------
static constexpr uint8_t CAP1188_PRODUCT_ID_VALUE       = 0x50;
static constexpr uint8_t CAP1188_MANUFACTURER_ID_VALUE  = 0x5D;
static constexpr uint8_t CAP1188_REVISION_VALUE         = 0x83;

// ------------------------------------------------------------
//  Main Control register bit masks (REG_MAIN_CTRL 0x00)
// ------------------------------------------------------------
static constexpr uint8_t CAP1188_MAIN_INT_BIT           = (1 « 0); // Interrupt flag - clear to
acknowledge a touch interrupt
static constexpr uint8_t CAP1188_MAIN_DSLEEP_BIT        = (1 « 4); // Deep sleep enable
static constexpr uint8_t CAP1188_MAIN_STBY_BIT          = (1 « 5); // Standby enable
static constexpr uint8_t CAP1188_MAIN_GAIN1_BIT         = (1 « 6); // Gain bit 1 (GAIN[1:0] at bits
7:6)
static constexpr uint8_t CAP1188_MAIN_GAIN2_BIT         = (1 « 7); // Gain bit 2

// ------------------------------------------------------------
//  Sensitivity control values (bits 6:4 of REG_SENSITIVITY 0x1F)
//  Lower value = more sensitive (smaller delta required)
// ------------------------------------------------------------
static constexpr uint8_t CAP1188_SENSITIVITY_128X       = 0x00;
static constexpr uint8_t CAP1188_SENSITIVITY_64X        = 0x01;
static constexpr uint8_t CAP1188_SENSITIVITY_32X        = 0x02; // default
static constexpr uint8_t CAP1188_SENSITIVITY_16X        = 0x03;
static constexpr uint8_t CAP1188_SENSITIVITY_8X         = 0x04;
static constexpr uint8_t CAP1188_SENSITIVITY_4X         = 0x05;
static constexpr uint8_t CAP1188_SENSITIVITY_2X         = 0x06;
static constexpr uint8_t CAP1188_SENSITIVITY_1X         = 0x07;

static constexpr std::array<uint8_t, 8> CAP1188_SENSITIVITY_OPTIONS = {128, 64, 32, 16, 8, 4, 2, 1};
static constexpr std::array<uint8_t, 8> CAP1188_AVERAGING_OPTIONS = {1, 2, 4, 8, 16, 32, 64, 128};
static std::array<std::string, 4> CAP1188_SAMP_TIME_OPTIONS = {"320us", "640us", "1.28ms", "2.56ms"};
static std::array<std::string, 4> CAP1188_CYCLE_TIME_OPTIONS = {"35ms", "70ms", "105ms", "140ms"};
```

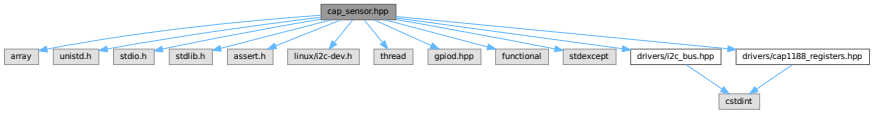
10.24 cap_sensor.hpp File Reference

Base driver class for the Adafruit CAP1188 sensor breakout board, specific to Raspberry Pi 5 over Linux kernel with I2C communication.

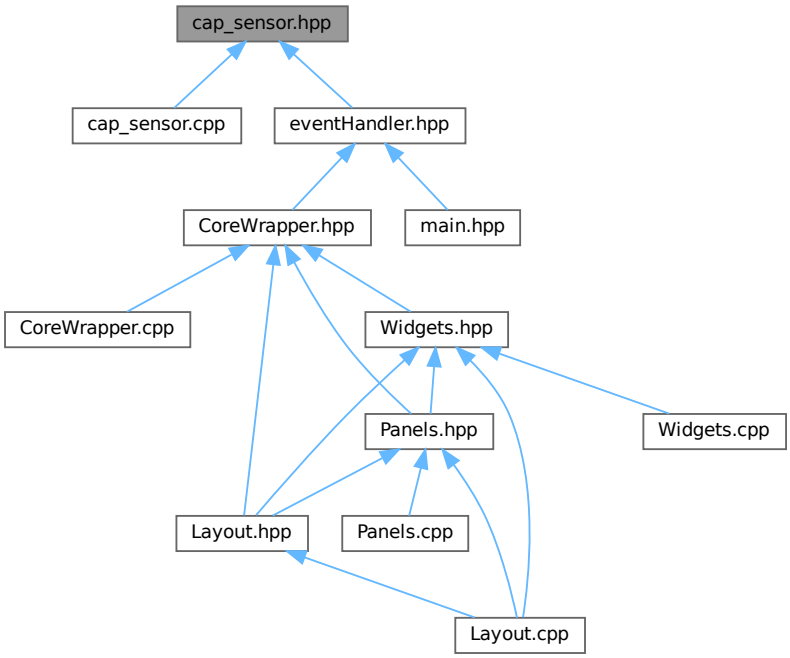
```
#include <array>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <assert.h>
#include <linux/i2c-dev.h>
#include <thread>
#include <gpiod.hpp>
#include <functional>
#include <stdexcept>
#include "drivers/i2c_bus.hpp"
```

```
#include "drivers/cap1188_registers.hpp"
```

Include dependency graph for cap_sensor.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class CAP1188Channel

  Helper class unique to one sensor channel.
- class CAP1188

  Base driver class for the CAP1188.

10.24.1 Detailed Description

Base driver class for the Adafruit CAP1188 sensor breakout board, specific to Raspberry Pi 5 over Linux kernel with I2C communication.

Author

Ben Allen

Version

0.1

Date

2026-04-19

Copyright

Copyright (c) 2026

10.25 cap_sensor.hpp

Go to the documentation of this file.

```
00001
00014 #pragma once
00015
00016 #include <array>
00017 #include <unistd.h>
00018 #include <stdio.h>
00019 #include <stdlib.h>
00020 #include <assert.h>
00021 #include <linux/i2c-dev.h>
00022 #include <thread>
00023 #include <gpiod.hpp>
00024 #include <functional>
00025 #include <stdexcept>
00026 #include "drivers/i2c_bus.hpp"
00027 #include "drivers/cap1188_registers.hpp"
00028
00029 class CAP1188;
00036 class CAP1188Channel{
00037     public:
00046         bool isPinTouched();
00054         int8_t pinDelta();
00059         int getThreshold();
00064         void setThreshold(int threshold);
00070         void pinRecalibrate();
00071     private:
00072         friend class CAP1188;
00073         CAP1188Channel();
00074         ~CAP1188Channel();
00075         void initialise(CAP1188* cap1188, int pin);
00076         CAP1188* cap1188_;
00077         int pin_;
00078 };
00086 class CAP1188
00087 {
00088     public:
00089         using CAP1188CallbackInterface = std::function<void(uint8_t)>;
00090
00107         CAP1188(I2CBus& bus,
00108                 uint8_t address                 = 0x28,
00109                 const std::string& gpioChipPath = "/dev/gpiochip0",
00110                 unsigned int gpioLine = 27,
00111                 uint8_t enabledChannels = 0x01,
00112                 uint8_t interruptEnabledChannels = 0x01,
00113                 bool isTest = false);
00114
00118         ~CAP1188();
00129         bool initialise();
00134         void start();
00138         void stop();
00142         void worker();
00148         void handleDataReady();
00154         void registerCallback(CAP1188CallbackInterface ci);
00166         uint8_t getSensitivity();
00177         void setSensitivity(uint8_t sensitivityVal);
00185         uint8_t getAveraging();
00194         void setAveraging(uint8_t averagingVal);
00200         std::string getSampleTime();
00206         void setSampleTime(std::string newSampleTime);
00213         std::string getCycleTime();
00221         void setCycleTime(std::string newCycleTime);
```

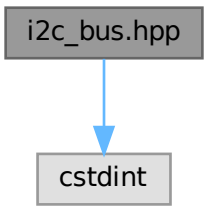
```
        int touched();
        int8_t deltaCount(int pin);
        void recalibratePins(uint8_t pins);
        std::array<uint8_t, 8> getThresholds();
        void setThresholds(std::array<uint8_t, 8> newThresholds);
        void linkLEDs(uint8_t pinmask);
        void setInterruptRepeatRate(uint8_t rate);
        void enableChannels();
        void enableInterruptChannels();
        void disableStandby();
        void clearInterrupt();

    private:

        I2CBus& bus_;
        uint8_t address_;
        std::string gpioChipPath_;
        unsigned int gpioLine_;
        unsigned int gpioReset_;
        uint8_t enabledChannels_;
        uint8_t interruptEnabledChannels_;
        CAP1188Channel channels[8];

        uint8_t interruptPolarity_ = 0;
        bool running_ = false;

        // Stored callback
        CAP1188CallbackInterface cap1188CallbackInterface_;

        std::shared_ptr<gpiod::chip> chip_;
        std::shared_ptr<gpiod::line_request> request_;
        std::thread thread_;

};
```

10.26 i2c_bus.hpp File Reference

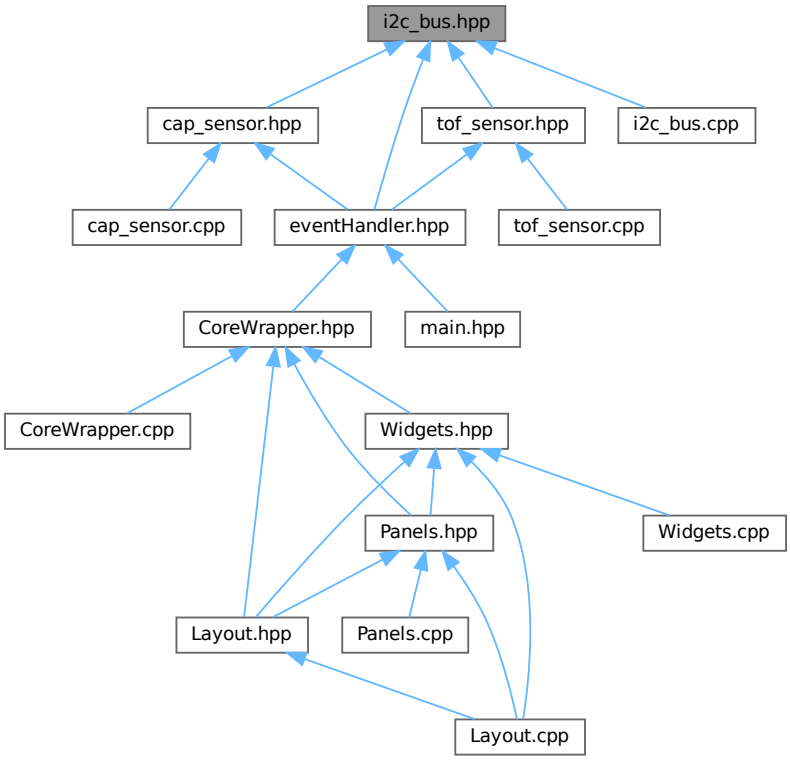
Minimal Linux I2C bus wrapper.

`#include <cstdint>`

Include dependency graph for i2c_bus.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class I2CBus

  Minimal Linux I2C bus wrapper for register-based communication.

10.26.1 Detailed Description

Minimal Linux I2C bus wrapper.

This class provides basic functionality to open/close an I2C device and perform register-based read/write transactions. It does not contain any sensor-specific logic, making it reusable for various I2C devices.

10.27 i2c_bus.hpp

Go to the documentation of this file.

```
00001 #pragma once
00002
00013 #include <cstdint>
00014
00027 class I2CBus
00028 {
00029 public:
00035     explicit I2CBus(const char* devicePath = "/dev/i2c-1", bool isTest = false);
```

```

    ~I2CBus();

    bool openBus();

    void closeBus();

    int writeBlock16(uint8_t address, uint16_t reg, const uint8_t* data, uint8_t length);

    int readBlock16(uint8_t address, uint16_t reg, uint8_t* data, uint8_t length);


    int writeBlock8(uint8_t address, uint8_t reg, const uint8_t* data, uint8_t length);


    int readBlock8(uint8_t address, uint8_t reg, uint8_t* data, uint8_t length);

private:

    const char* devicePath_;

    int fd_;

    /*  A flag stored to inform the methods if it should ignore
        all hardware dependent functions. */
    bool isTestMode = false;
};
```

10.28 Pin_Definitions.h

```
/* Defining main.h to have so there is no recursion*/

#ifndef __PIN_DEF_
#define __PIN_DEF_
#ifdef C_PLUS_PLUS
C{
#endif



/* Pin definitions*/
#define LED 2



#ifdef C_PLUS_PLUS
}
#endif
#endif
```

10.29 pressure_sensor.hpp

```
#pragma once

#include <functional>
#include "ads1115rpi.h"  // From ADS1115 submodule

class PressureSensor {
public:
    using PressureCallback = std::function<void(float)>;

    PressureSensor() = default;
    ~PressureSensor() { stop(); }

    PressureSensor(const PressureSensor&) = delete;
    PressureSensor& operator=(const PressureSensor&) = delete;


    void registerCallback(PressureCallback cb) {
        callback = cb;
        adc.registerCallback([this](float voltage) {
            if (callback) {
                callback(voltage);
            }
        });
    }


```

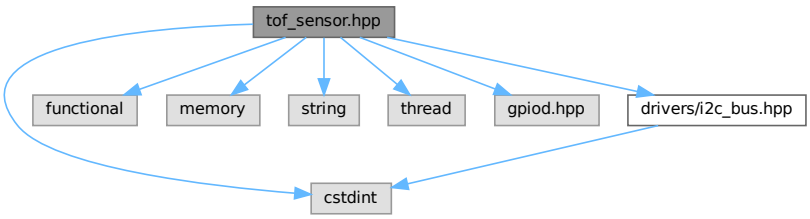
```
    void start(ADS1115settings settings = getDefaultSettings()) {
        adc.start(settings);
    }


    void stop() {
        adc.stop();
    }

    static ADS1115settings getDefaultSettings() {
        ADS1115settings s;
        s.samplingRate = ADS1115settings::FS128HZ;
        s.channel = ADS1115settings::AIN0;
        s.pgaGain = ADS1115settings::FSR0_512;
        return s;
    }

    ADS1115rpi& getADC() { return adc; }

private:
    ADS1115rpi adc;
    PressureCallback callback;
};
```

10.30 tof_sensor.hpp File Reference

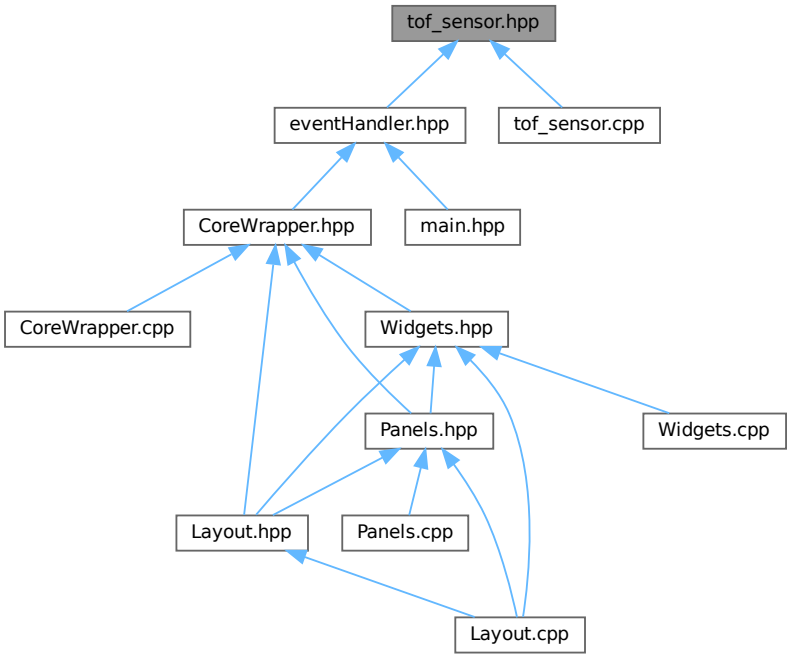
Interface for VL53L1X Time-of-Flight sensor.

```
#include <cstdint>
#include <functional>
#include <memory>
#include <string>
#include <thread>
#include <gpiod.hpp>
#include "drivers/i2c_bus.hpp"
```

Include dependency graph for tof_sensor.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class ToFSensor

  Time-of-Flight sensor interface using I2C and GPIO interrupt handling.

10.30.1 Detailed Description

Interface for VL53L1X Time-of-Flight sensor.

Defines the ToFSensor class, which provides interrupt-driven distance measurement using the VL53L1X sensor over I2C and GPIO (libgpiod).

10.31 tof_sensor.hpp

Go to the documentation of this file.

```
00001 #pragma once
00002
00012 #include <cstdint>
00013 #include <functional>
00014 #include <memory>
00015 #include <string>
00016 #include <thread>
00017
00018 #include <gpiod.hpp>
00019
00020 #include "drivers/i2c_bus.hpp"
00021
00031 class ToFSensor
```

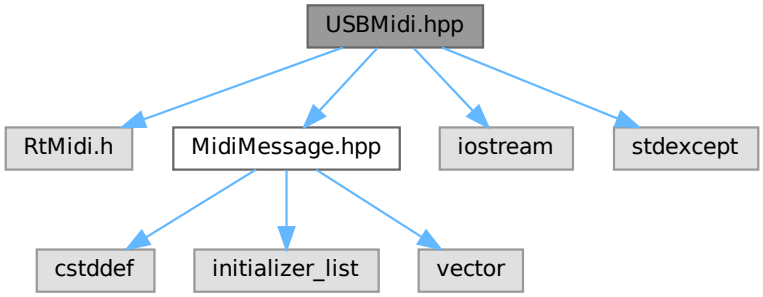
```
{
public:

    using DistanceCallback = std::function<void(uint16_t)>;


    ToFSensor(I2CBus& bus,
              uint8_t i2cAddress7Bit = 0x29,
              const std::string& gpioChipPath = "/dev/gpiochip0",
              unsigned int gpioLine = 4,
              bool isTest=false);

    ~ToFSensor();

    bool initialise();

    void registerCallback(DistanceCallback cb);

    void start();

    void stop();

private:

    void softReset();

    void startRanging();

    void stopRanging();

    uint16_t readDistanceMm();

    void worker();

    void handleDataReady();

    I2CBus& bus_;

    uint8_t i2cAddress7Bit_;

    uint16_t stDevAddress_;

    std::string gpioChipPath_;

    unsigned int gpioLine_;

    DistanceCallback distanceCallback_;

    bool running_ = false;

    std::thread thread_;

    std::shared_ptr<gpiod::chip> chip_;

    std::shared_ptr<gpiod::line_request> request_;

    /*  A flag stored to inform the methods if it should ignore
        all hardware dependent functions. */
    bool isTestMode = false;
};
```

10.32 USBMidi.hpp File Reference

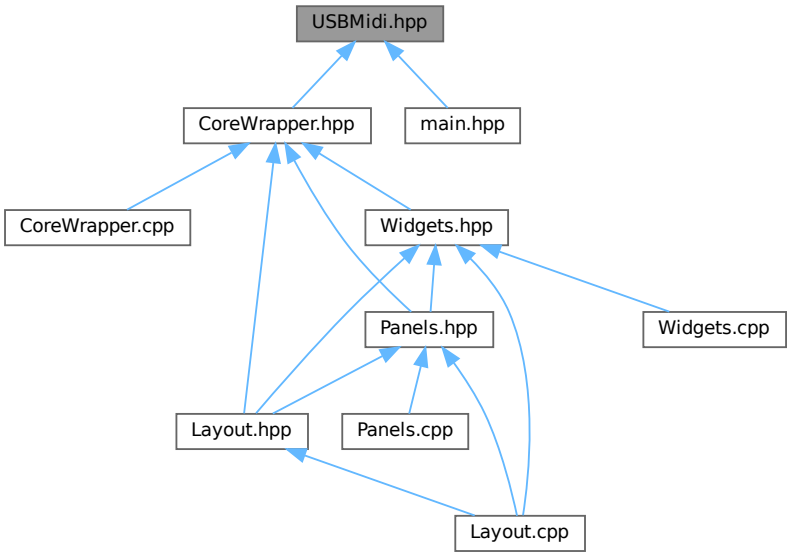
Declares the RtMidiSink class, responsible for sending MIDI messages to an external device via RtMidi.

```
#include <RtMidi.h>
#include "MidiMessage.hpp"
#include <iostream>
#include <stdexcept>
```

Include dependency graph for USBMidi.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class RtMidiSink

  Handles MIDI output to external devices using RtMidi.

10.32.1 Detailed Description

Declares the RtMidiSink class, responsible for sending MIDI messages to an external device via RtMidi.

Author

Aidan McIntosh

Date

2026-04-20

10.33 USBMidi.hpp

Go to the documentation of this file.

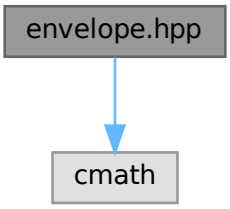
```
#pragma once

#include <RtMidi.h>
#include "MidiMessage.hpp"
#include <iostream>
#include <stdexcept>

class RtMidiSink {
public:
    RtMidiSink(bool isTest);

    void send(const MidiMessage& message);

    void DetectUSBMidiOutput();

    bool GetDeviceStatus();

private:
    RtMidiOut midiOut;

    bool EXTERNAL_DEVICE_PRESENT = false;

    int externalPortIndex = -1;
};
```

10.34 envelope.hpp File Reference

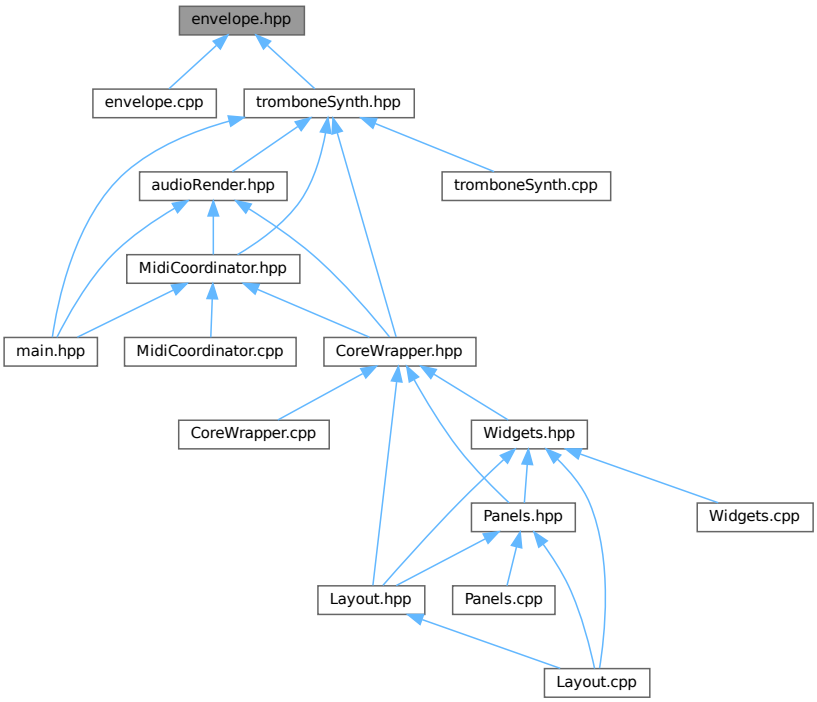
A header file containing all the necessary declarations and references to create a functional envelope class using the ADSR standard.

`#include <cmath>`

Include dependency graph for envelope.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class Envelope

  A class which can allow for the create of an envelope function which outputs a value between 0 and 1 depending on what stage the envelope is in. This uses the ADSR standard in music synthesis where A is the Attack, D is the decay, S is the saturation and R is the rest stage.

10.34.1 Detailed Description

A header file containing all the necessary declarations and references to create a functional envelope class using the ADSR standard.

Author

Ryan McBride

10.35 envelope.hpp

Go to the documentation of this file.

```
00001
00007 /* Preventing recursion. */
00008 #pragma once
00009
00010 /* Addition of all necessary header files. */
00011 #include <cmath>
```

```
00012
00021 class Envelope{
00022     public:
00023
00025         typedef enum{
00026             no_stage,
00027             attack_stage,
00028             decay_stage,
00029             sustain_stage,
00030             rest_stage,
00031         }   envelope_stages_t;
00032
00053         Envelope(int attack_in = 10, int decay_in = 20, float sustain_in = 0.98, int rest_in = 10);
00054
00058         ~Envelope() = default;
00059
00068         float getAmplitude(void);
00069
00071         void startEnvelope(void);
00072
00076         void endEnvelope(void);
00077
00084         void setAttack(int attack_in);
00085
00093         int getAttack(void);
00094
00101         void setDecay(int decay_in);
00102
00111         int getDecay(void);
00112
00119         void setSustain(float sustain_in);
00120
00125         float getSustain(void);
00126
00133         void setRest(int rest_in);
00134
00142         int getRest(void);
00143
00149         float attackFunction(void);
00150
00159         float decayFunction(void);
00160
00169         float restFunction(void);
00170
00174         envelope_stages_t getStage(void);
00175
00179         void setStage(envelope_stages_t stage_in);
00180
00181     private:
00182         /*  ADSR Variables */
00183         int attack;
00184         int decay;
00185         float sustain;
00186         int rest;
00187
00193         float clamp01(float value);
00194         /*  Timing Variables */
00195         int counter = 0;
00196         envelope_stages_t stage = no_stage;
00197 };
```

10.36 eventHandler.hpp File Reference

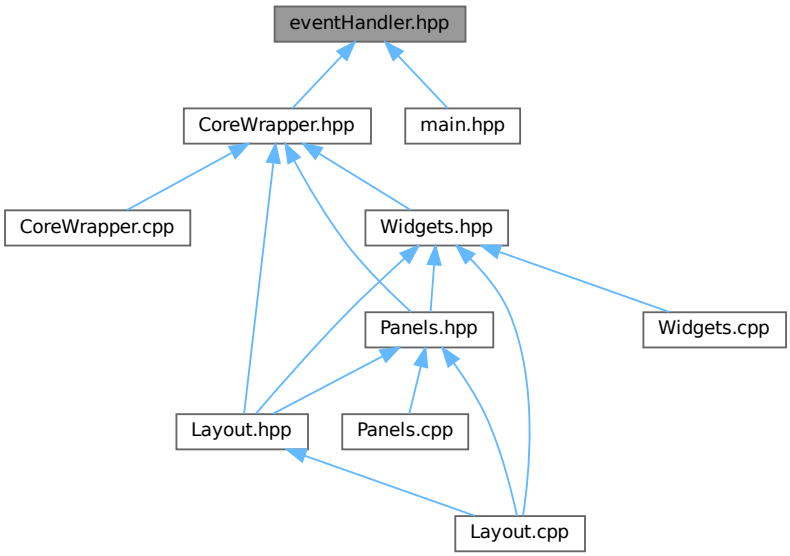
Declares the EventHandler class, responsible for acquiring and normalising raw sensor inputs into a unified event stream.

```
#include <cstdint>
#include <condition_variable>
#include <mutex>
#include <queue>
#include "drivers/i2c_bus.hpp"
#include "drivers/pressure_sensor.hpp"
#include "drivers/tof_sensor.hpp"
```

```
#include "drivers/cap_sensor.hpp"
```

Include dependency graph for eventHandler.hpp:

This graph shows which files directly or indirectly include this file:



Classes

- struct RawInputEvent

  Represents a generic input event produced by hardware sensors.
- class EventHandler

  Handles acquisition and dispatch of raw sensor input events.

10.36.1 Detailed Description

Declares the EventHandler class, responsible for acquiring and normalising raw sensor inputs into a unified event stream.

Author

Aidan McIntosh

10.37 eventHandler.hpp

Go to the documentation of this file.

```
#pragma once

#include <cstdint>
#include <condition_variable>
#include <mutex>
#include <queue>

#include "drivers/i2c_bus.hpp"
#include "drivers/pressure_sensor.hpp"
#include "drivers/tof_sensor.hpp"
#include "drivers/cap_sensor.hpp"

struct RawInputEvent {
    enum class Type {
        ToFDistance,
        PressureReading,
        MouthpieceReading,
        Keycontrol
    };

    Type type{};
    uint16_t tofDistance{};
    float pressureReading{};
    int8_t mouthpieceReading{};
    char keycontrol{};
};

class EventHandler {
public:
    using EventQueue = std::queue<RawInputEvent>;

    EventHandler(bool isTest);

    bool initialise();

    void start();

    RawInputEvent waitForEvent();

private:
    void pushEvent(const RawInputEvent& event);

    void handleToFDistance(uint16_t distance);

    void handleKeyControl(char key);

    void handleEmbouchure(int8_t embouchure);

    void handlePressureReading(float pressure);

    EventQueue eventQueue;

    std::mutex eventQueueMutex;

    std::condition_variable eventQueueCv;

    I2CBus bus;

    ToFSensor tofSensor;

    PressureSensor pressureSensor;

    CAP1188 cap1188;

    bool isTestMode = false;
};
```

10.38 main.hpp

```
/* Defining main.h to have so there is no recursion */

#ifndef __MAIN_H_
#define __MAIN_H_
#ifdef C_PLUS_PLUS
C{
#endif

```

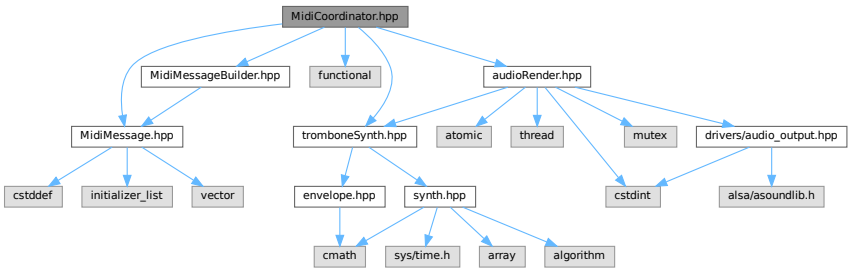
```
/* Including header files */

#include "eventHandler.hpp"
#include <condition_variable>
#include <cstdint>
#include <iostream>
#include <thread>
#include <mutex>
#include <queue>
#include <chrono>
#include "MidiCoordinator.hpp"
#include "tromboneSynth.hpp"
#include "USBMidi.hpp"
#include "audioRender.hpp"
#include "PitchMapper.hpp"
#include "AmplitudeMapper.hpp"
#include <iostream>
#include "tromboneless_data.hpp"

#ifdef C_PLUS_PLUS
}
#endif
#endif
```

10.39 MidiCoordinator.hpp File Reference

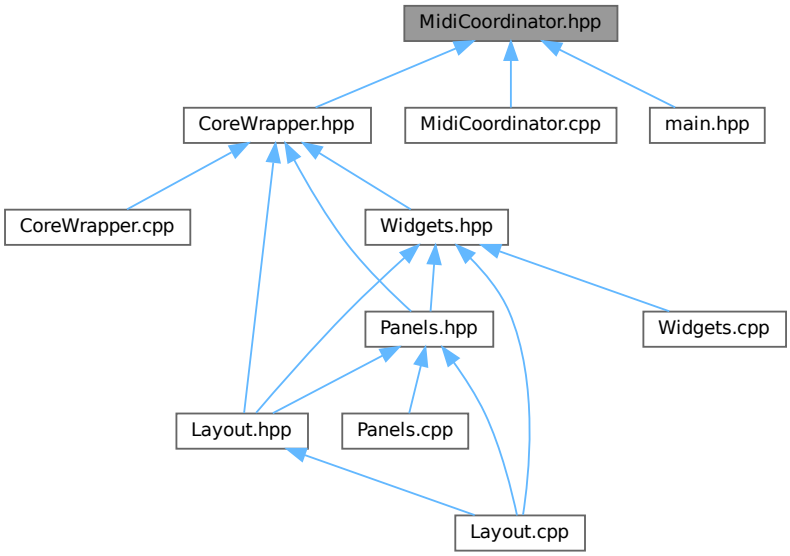
Declaration of class for high-level musical coordination behaviour. Takes premapped sensor inputs and either internal or external synth.

```
#include "MidiMessage.hpp"
#include "MidiMessageBuilder.hpp"
#include "functional"
#include "tromboneSynth.hpp"
#include "audioRender.hpp"
```

Include dependency graph for MidiCoordinator.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class MidiCoordinator

  Class to coordinate Midi message construction based on output from sensor mappers. This class is stateful and responsible for translating high-level sensor data into Midi messages. Utilises MidiMessageBuilder for external synths and directly controls internal synth.

10.39.1 Detailed Description

Declaration of class for high-level musical coordination behaviour. Takes premapped sensor inputs and either internal or external synth.

Author

Aidan McIntosh

10.40 MidiCoordinator.hpp

Go to the documentation of this file.

```
00001
00008 #pragma once
00009 #include "MidiMessage.hpp"
00010 #include "MidiMessageBuilder.hpp"
00011 #include "functional"
00012 #include "tromboneSynth.hpp"
00013 #include "audioRender.hpp"
00014
00019 class MidiCoordinator{
00020     private:
00025         MidiMessage message;
00026         MidiMessageBuilder builder;
```

```
        AudioRender& synthRef;

        using CallbackInterface = std::function<void(const MidiMessage&)>;
        CallbackInterface callback;

        int latestNote = -1; // Updated by mouthpiece. [0:127]
        int latestExpr = -1; // Updated by pressure. [0:127]
        int latestBend = -1; // Updated by slide. [0:8192]

        int currentNote = -1; // Playing note
        int lastSentExpr = -1; // Saves last expression sent to device
        int lastSentBend = -1; // Saves last bend sent to device

        const int velocity = 127; // Clamped to max
        typedef enum OutputType{
            EXTERNAL,
            INTERNAL
        } outputtype;

        outputtype current_output = INTERNAL;

        enum State{
            IDLE,
            PLAYING,
            ERROR
        };

        enum State current_state = IDLE;
        bool isTestMode = false;

    public:

        MidiCoordinator(AudioRender& synth, bool isTest);
        void RegisterCallback(CallbackInterface cb);

        void PressureEdge(bool on);
        void ChangeNote(int note);
        void setBend(int bend);
        void setExpr(int expr);

        void setState(State newstate);

        void setDevice(bool external_device_present);

        AudioRender& getSynth(void);
};
```

10.41 MidiMessage.hpp

```
#pragma once

#include <cstddef>
#include <initializer_list>
#include <vector>

class MidiMessage
{
public:
    using Byte = unsigned char;

    MidiMessage() = default;

    explicit MidiMessage(std::vector<Byte> bytes);

    MidiMessage(std::initializer_list<Byte> bytes);

    const std::vector<Byte>& bytes() const noexcept;

    std::size_t size() const noexcept;

    bool empty() const noexcept;

private:
    std::vector<Byte> data_;
};
```

10.42 MidiMessageBuilder.hpp

```
#pragma once
```

```

#include "MidiMessage.hpp"

class MidiMessageBuilder
{
public:
    using Byte = MidiMessage::Byte;

    MidiMessage noteOn(int channel,
                       Byte note,
                       Byte velocity) const;

    MidiMessage noteOff(int channel,
                        Byte note,
                        Byte velocity = 0) const;

    MidiMessage pitchBend(int channel,
                          int value) const;

    MidiMessage expr(int channel,
                     int value) const;

private:
    static Byte clampChannel(int channel);
};
```

10.43 PitchMapper.hpp

```
    /*** @file PitchMapper.hpp
     *   @author Aidan McIntosh
    */
#pragma once
#include <cstdint>
#include "tromboneless_data.hpp"

class PitchMapper{
    private:



        typedef enum{
            piccolo = 17,
            soprano = 12,
            alto = 5,
            tenor = 0,
            bass = -5,
            contrabass = -12
        } Trombone_type_t;


        int slide_max_limit_mm = 600; /* Max limit of virtual trombone slide in mm.*/
        int slide_min_limit_mm = 0; /* Min limit of virtual trombone slide in mm.*/
        int mouthpiece_MIDI_note = 34; /* MIDI note produced by mouthpiece.*/
        int partial = 0;


        int trombone_type = Trombone_type_t::piccolo;

    public:

        struct PartialDef
        {
            int minDelta;
            int hysteresisDelta;
            int partial;
            int midiNote;
        };

        PitchMapper();

        int tof_to_MIDI_bend(uint16_t tof_distance);

        int mouthpiece_to_MIDI_note(int8_t delta);


        void SetSlideMaxLimit(int new_slide_max_limit_mm);

        int GetSlideMaxLimit();

        void SetSlideMinLimit(int new_slide_min_limit_mm);

        int GetSlideMinLimit();

```

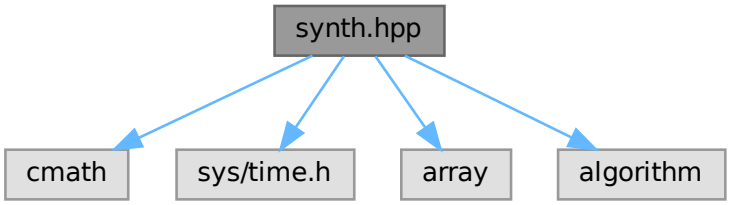
```
        void SetMouthMIDI(int new_MIDI_note);

        int GetMouthMIDI();

        void SetTromboneType(ShiftKeyingOptions_t trombone_choice);
};
```

10.44 synth.hpp File Reference

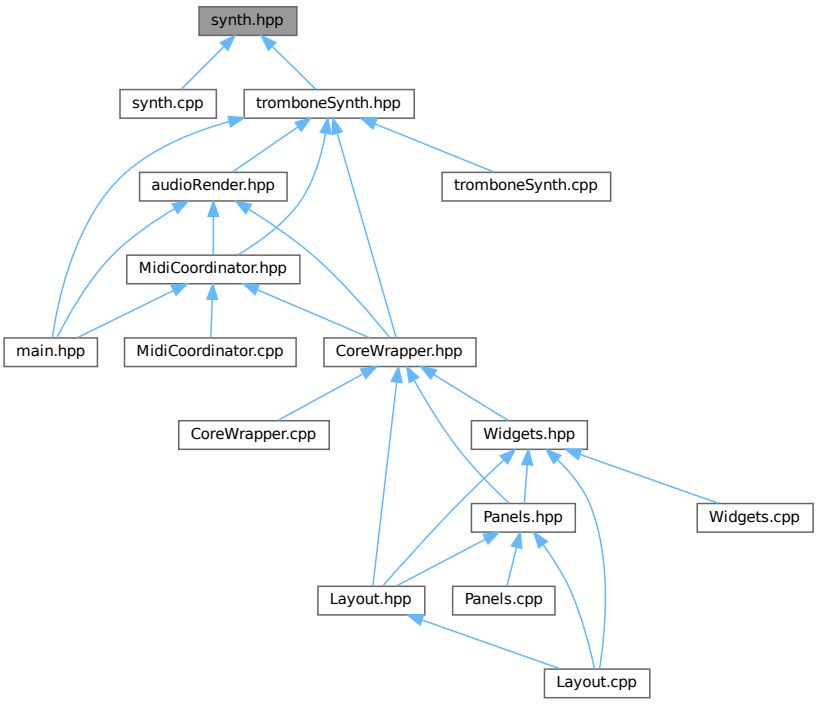
A header file containing all the necessary declarations and references to allow for the Tromboneless to synthesise the noise a trombone would make when playing the same notes.

```
#include <cmath>
#include <sys/time.h>
#include <array>
#include <algorithm>
```

Include dependency graph for synth.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class Notes

  A class to store the frequency of the notes which may be referenced. The notes are stored in an enum `Notes_t` which allow for the higher level functions to call upon the specific note as opposed to frequency.
- class Octaves

  Creation of a class to store the frequencies of notes at different octaves. Inherits from `Notes` class.

- class OctavesWithHarmonics

  A class which inherits functions and variables from the `Octaves` class allowing for the notes being output to have a set of harmonic frequencies attached to them too.

10.44.1 Detailed Description

A header file containing all the necessary declarations and references to allow for the Tromboneless to synthesise the noise a trombone would make when playing the same notes.

Author

Ryan McBride

Note

The envelope class lacks set/get functionality for its initialised parameters such as the attack, decay and rest times. This should be added in the next revision to allow for the sound to be modified without needing to create a new object each time.

10.45 synth.hpp

Go to the documentation of this file.

```
/* Preventing recursion. */
#pragma once

/* Addition of all necessary header files. */
#include <cmath>
#include <sys/time.h>
#include <array>
#include <algorithm>

/* ========================================================================================== */
/*                                    Class definitions                                       */
/* ========================================================================================== */

class Notes{
    public:

        typedef enum{
            note_C,
            note_Db,
            note_D,
            note_Eb,
            note_E,
            note_F,
            note_Gb,
            note_G,
            note_Ab,
            note_A,
            note_Bb,
            note_B,

            /* Addition of sharp notes. */
            note_C_sharp = note_Db,
            note_D_sharp = note_Eb,
            note_F_sharp = note_Gb,
            note_G_sharp = note_Ab,
            note_A_sharp = note_Bb,
        }   Notes_t;

        Notes(float C_in     =   16.35, float Db_in    =   17.32,
            float D_in       =   18.35, float Eb_in    =   19.45,
            float E_in       =   20.60, float F_in     =   21.83,
            float Gb_in      =   23.12, float G_in     =   24.50,
            float Ab_in      =   25.96, float A_in     =   27.50,
            float Bb_in      =   29.14, float B_in     =   30.87);

        ~Notes() = default;

        /* ========================================================================================== */
        /*                                  Note setters                                              */
        /* ========================================================================================== */

        void setC(float frequency);

        void setC_sharp(float frequency);

        void setDb(float frequency);

        void setD(float frequency);

        void setD_sharp(float frequency);

        void setEb(float frequency);

        void setE(float frequency);

        void setF(float frequency);

        void setF_sharp(float frequency);

        void setGb(float frequency);

        void setG(float frequency);

        void setG_sharp(float frequency);

        void setAb(float frequency);

        void setA(float frequency);
```

```
        void setA_sharp(float frequency);

        void setBb(float frequency);

        void setB(float frequency);

        float setNote(Notes_t note, float freq);

        /* ==========================================================================================
        /*                                Note getters
        /* ==========================================================================================

        float getC(void);

        float getC_sharp(void);

        float getDb(void);

        float getD(void);

        float getD_sharp(void);

        float getEb(void);

        float getE(void);

        float getF(void);

        float getF_sharp(void);

        float getGb(void);

        float getG(void);

        float getG_sharp(void);

        float getAb(void);

        float getA(void);

        float getA_sharp(void);

        float getBb(void);

        float getB(void);

        float getNote(Notes_t note);

    private:
        /* Initialisation of the possible note frequencies. */
        float   C, Db, D, Eb, E, F,
                Gb, G, Ab, A, Bb, B;

};


class Octaves   : public Notes
{
    public:

        static const char nOctaves = 9;

        Notes octaves[nOctaves];

        /* Constructor for the class*/
        Octaves();

        ~Octaves() = default;

        float PlayingNote(int octave, Notes_t note, float phase);

        float PlayingFrequency(float phase);


    private:
};

class OctavesWithHarmonics :    public Octaves
{
    public:

        OctavesWithHarmonics();

```

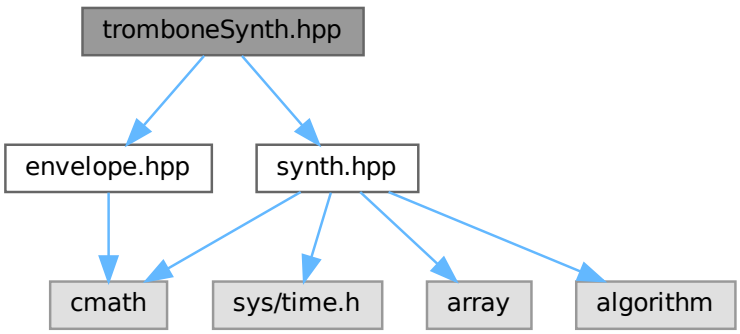
```
        ~OctavesWithHarmonics() = default;

        float PlayingNoteWithHarmonics(int n, int octave, Notes_t note, float phase);

        float PlayingFrequencyWithHarmonics(int n, float phase);

        float HarmonicDecay(int n);

        float getHarmonicDecayMax(int n);

        void setDecayConstant(float decayC);

        float getDecayConstant(void);

    protected:

        static constexpr int maxHarmonics = 64;

        float decayConstant = 0.1f;

        std::array<float, maxHarmonics> harmonicWeights{};
        int cachedNHarmonics = 0;
        bool harmonicCacheDirty = true;

        void updateHarmonicCache(int n);
};
```

10.46 tromboneSynth.hpp File Reference

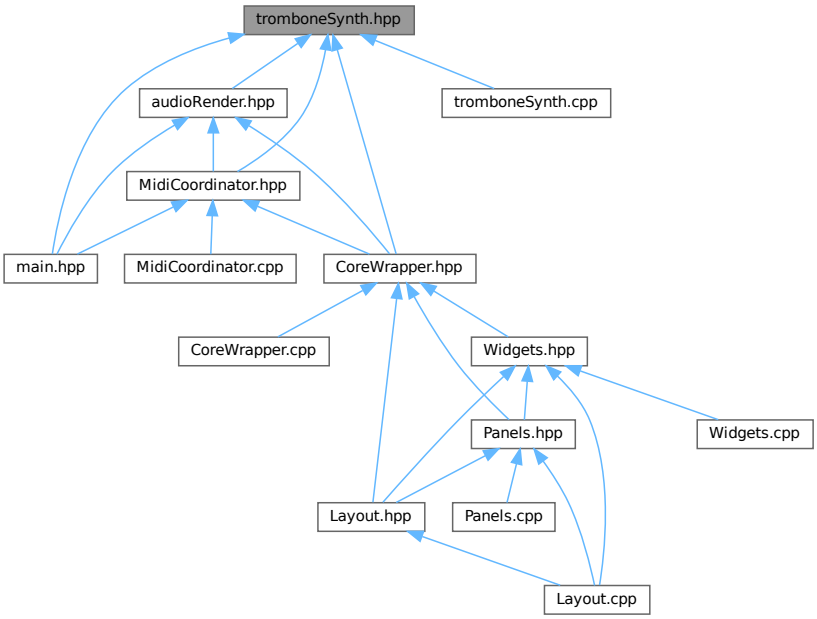
A file to declare the synthesiser parameters and functions for the trombone specifically using the classes and methods declared in the `synth.hpp` file.

```
#include "synth.hpp"
#include "envelope.hpp"
```

Include dependency graph for tromboneSynth.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class TromboneSynth

  A class which may be used to create a custom synth with an envelope attached to it.

10.46.1 Detailed Description

A file to declare the synthesiser parameters and functions for the trombone specifically using the classes and methods declared in the `synth.hpp` file.

Author

Ryan McBride

10.47 tromboneSynth.hpp

Go to the documentation of this file.

```
00001
00008     /* Preventing recurssion */
00009  #pragma once
00010
00011  /* Adding the necessary header files to be included. */
00012  #include "synth.hpp"
00013  #include "envelope.hpp"
00014
00018  class TromboneSynth     :   public OctavesWithHarmonics
00019  {
00020     public:
00035         TromboneSynth(int sampleRate = 44100, float attack = 10.0f, float decay = 50.0f,
00036              float sustain = 1.0f, float rest = 10.0f);
```

```

        void StartTromboneNote(Notes::Notes_t note_in, int octave_in);

        void ChangeTromboneNote(Notes::Notes_t note_in, int octave_in);

        float ReadTromboneAudio(void);

        void StopTromboneNote(void);

        void NewTromboneNoteMIDI(int MIDINote);

        void HandleMIDINoteOn();

        void HandleMIDINoteOff();

        void HandleMIDIPitchBend(int midipitchbend);

        void HandleMIDINoteChange(int new_note);

        void setAttackMS(float attack_time);

        float getAttackMS(void);

        void setDecayMS(float decay_time);

        float getDecayMS(void);

        void setRestMS(float rest_time);

        float getRestMS(void);

        void setSustain(float sustain_in);

        float getSustain(void);

        void setNHarmonics(int n);

        int getNHarmonics(void);

        int getOctave(void);

        Notes::Notes_t getNote(void);

        void setPitchBend(int bend);

        int getPitchBend(void);

        float getAdjustedFrequency(void);

        Envelope getEnvelope(void);

    private:
        /* Initialising the envelope class which will be used to create the various
        different noises for the trombone. This sets the sound of the trombone
        permanently. The ADSR values may need to be adjusted but this should sound
        brassy. */

        Envelope tromboneEnvelope = Envelope(10, 50, 1.0f, 10);

        float attack_ms = 10.0f;        /*  The attack time in milliseconds of the envelope. */
        float decay_ms = 50.0f;         /*  The decay time in milliseconds of the envelope. */
        float rest_ms = 10.0f;          /*  The rest time in milliseconds of the envelope. */
        float sustain = 1.0f;          /*  The sustain level of the envelope. */

        float cachedFrequency = 0.0f;
        float cachedPhaseIncrement = 0.0f;
        bool pitchCacheDirty = true;

        void updatePitchCache();


        float phase = 0.0f; /* Phase of oscillator wave */

        /*  synth   */
        int nHarmonics = 17;    /*  The number of harmonics which will be present in the final signal. */
        int octave = 4;         /*  The octave which the note will be present in. */
        Notes::Notes_t note = Notes::Notes::note_A;  /*  The note which will be played. */
        int pitchBend   =   8192;  /*  The MIDI message pitch bend dependent upon the position of the slider. */

        unsigned int samples = 0;
        unsigned int sampleRate = 44100;

        const unsigned int pitchBendMax = 8192;    /*  The maximum number which may be passed to the the pitch bending function by the MIDI input. */
        const unsigned int pitchBendSpread = 7;    /*  The maximum number of semitones that the pitch
```

```
bend may change the playing note by*/
        const float semitoneSpread = pitchBendMax/pitchBendSpread;  /*   The number of data points
between each adjacent semitone. */
        const int nNotes = 12;          /*  The number of notes per octave. */
};
```

10.48 CoreWrapper.cpp File Reference

A file which encapsulations the main funcationality to allow for the app to act independently of it.

```
#include "CoreWrapper.hpp"
#include <stdexcept>
```

Include dependency graph for CoreWrapper.cpp:

10.48.1 Detailed Description

A file which encapsulations the main funcationality to allow for the app to act independently of it.

Author

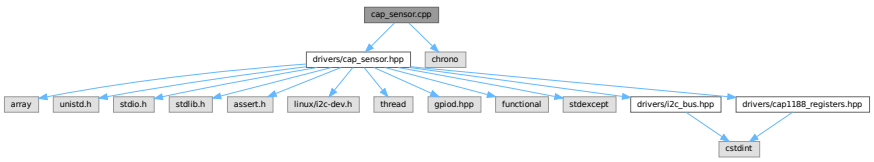
Ryan McBride Aidan McIntosh

10.49 cap_sensor.cpp File Reference

Implementation of CAP11888 driver for Tromboneless.

```
#include "drivers/cap_sensor.hpp"
#include <chrono>
```

Include dependency graph for cap_sensor.cpp:



10.49.1 Detailed Description

Implementation of CAP11888 driver for Tromboneless.

Author

Ben Allen

Version

0.1

Date

2026-04-20

Copyright

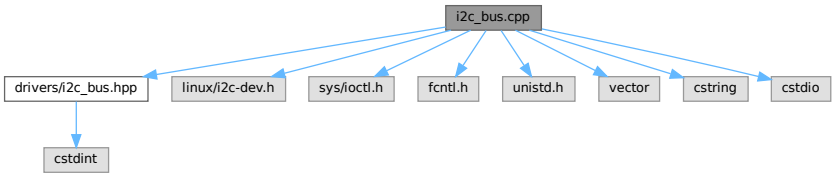
Copyright (c) 2026

10.50 i2c_bus.cpp File Reference

Implementation of the I2CBus class for Linux I2C communication.

```
#include "drivers/i2c_bus.hpp"
#include <linux/i2c-dev.h>
#include <sys/ioctl.h>
#include <fcntl.h>
#include <unistd.h>
#include <vector>
#include <cstring>
#include <cstdio>
```

Include dependency graph for i2c_bus.cpp:



10.50.1 Detailed Description

Implementation of the I2CBus class for Linux I2C communication.

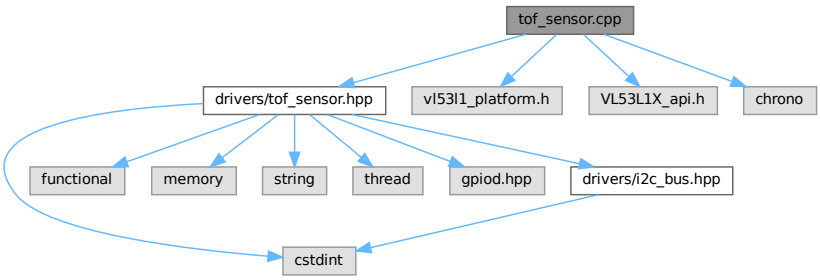
Implements low-level Linux I2C device access using file descriptors, ioctl device selection, and register-based read/write operations.

10.51 tof_sensor.cpp File Reference

Implementation of the ToFSensor class.

```
#include "drivers/tof_sensor.hpp"
#include "vl53l1_platform.h"
#include "VL53L1X_api.h"
#include <chrono>
```

Include dependency graph for tof_sensor.cpp:



10.51.1 Detailed Description

Implementation of the ToFSensor class.

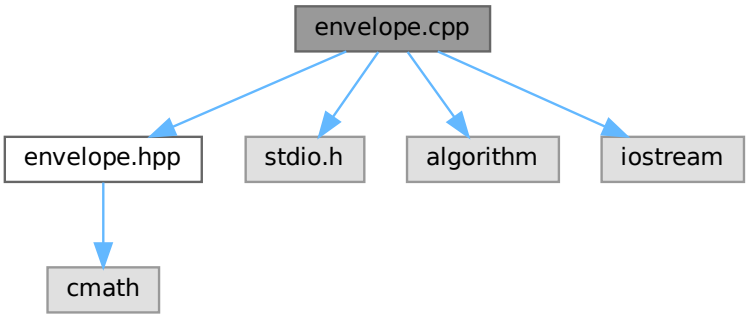
Implements VL53L1X initialisation, GPIO interrupt handling via libgpiod, and callback-based publication of distance measurements.

10.52 envelope.cpp File Reference

Contains the defintions of the functions and variables needed to create an envelope.

```
#include "envelope.hpp"
#include <stdio.h>
#include <algorithm>
#include <iostream>
```

Include dependency graph for envelope.cpp:



10.52.1 Detailed Description

Contains the defintions of the functions and variables needed to create an envelope.

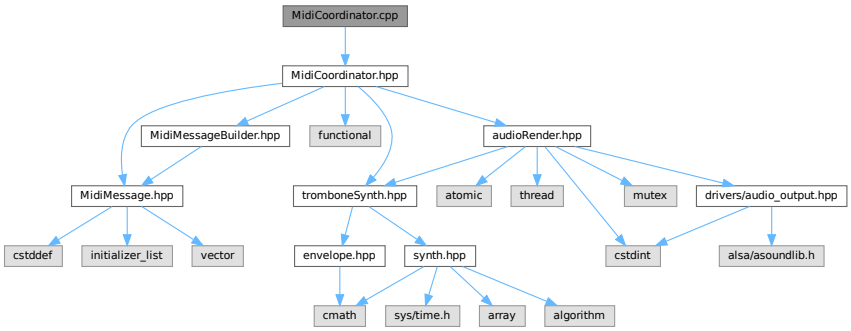
Author

RyanMcBride

10.53 MidiCoordinator.cpp File Reference

Definition file for MIDICoordinator class.

`#include "MidiCoordinator.hpp"`
Include dependency graph for MidiCoordinator.cpp:



10.53.1 Detailed Description

Definition file for MIDICoordinator class.

Author

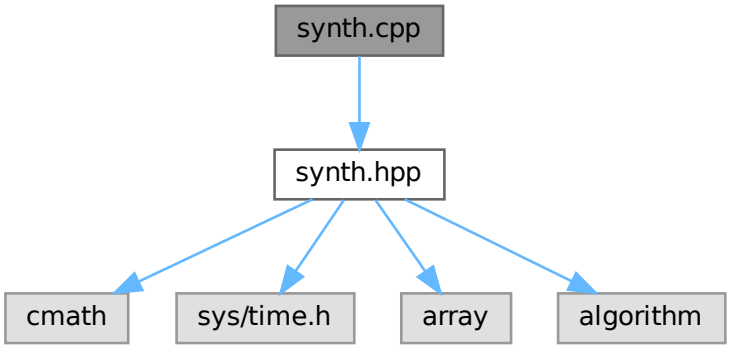
Aidan McIntosh

10.54 synth.cpp File Reference

C file which contains the defintions for the functions declared in the `synth.hpp` file.

```
#include "synth.hpp"
```

Include dependency graph for synth.cpp:



10.54.1 Detailed Description

C file which contains the defintions for the functions declared in the `synth.hpp` file.

Author

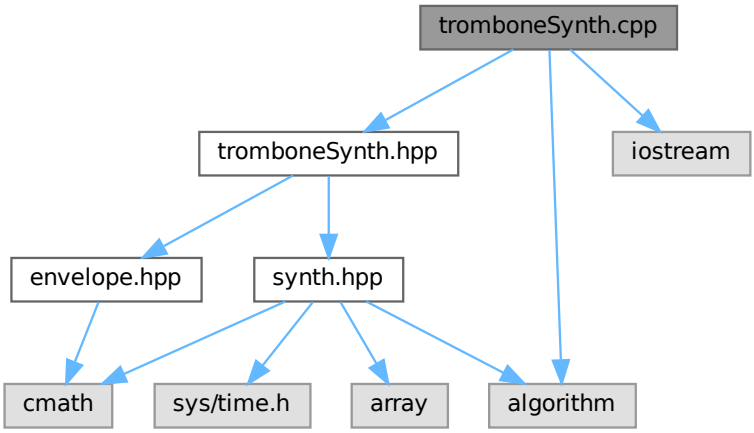
Ryan McBride

10.55 tromboneSynth.cpp File Reference

A file to define the synthesiser parameters and functions for the trombone specifically using the classes and methods declared in the `synth.hpp` file.

```
#include "tromboneSynth.hpp"
#include <algorithm>
#include <iostream>
```

Include dependency graph for tromboneSynth.cpp:



10.55.1 Detailed Description

A file to define the synthesiser parameters and functions for the trombone specifically using the classes and methods declared in the `synth.hpp` file.

Author

Ryan McBride

10.56 tromboneless_data.hpp

```
/* Header file to pass functions which may have an effect on both the app and core functionality. */
#ifndef _TROMBONELESS_DATA_H_
#define _TROMBONELESS_DATA_H_

/* ============================ This is where the types are defined ============================ */

typedef enum{
    SKOpt_EMPTY,
    SKOpt_BASS,
    SKOpt_PICCOLO,
    SKOpt_ALTO,
    SKOpt_TENOR,
    SKOpt_CONTRABASS,
    SKOpt_SOPRANO,
} ShiftKeyingOptions_t;

typedef enum{
    EMBCR_EMPTY,
    EMBCR_LowPlacement,
    EMBCR_MediumPlacement,
    EMBCR_HighPlacement,
} EmbachureOptions_t;

typedef struct{
    ShiftKeyingOptions_t shiftKeyingOption;

    double nearDistance;

    double farDistance;

```

```
    double lowPressure;
    double highPressure;
    EmbachureOptions_t embouchureOption;
    bool triggerEmbouchureCalibrate;
} Tromboneless Parameters_t;

typedef struct{
    bool synthEnable;
    float gains[10] = {1, 1, 1, 1, 1, 1, 1, 1, 1, 1};
} SynthesiserParameters_t;

/*  Creating an instance of the tromboneless Parameters_t struct to
    store the active parameter variables. */
extern Tromboneless Parameters_t tromboneless Parameters;
/*  Creating an instance of the SynthethiserParameters_t struct to
    store the active parameter variables. */
extern SynthesiserParameters_t synthesiserParameters;
#endif
```